

★ Overview



Project Julius
Instruction Guide

Welcome to Project Julius

Before You Begin

Read through every item on this page carefully. Working through everything will take time, but it is the most important time you will spend on this project.

This is rare, specialized research, and only a small number of people in the world have both the expertise and the opportunity to do it. You're one of them. We're thrilled to have you onboard!

Project Overview

Project Julius is a unique opportunity to take AI agents out of the chat interface and into real professional environments, where they produce actual file outputs, interact with real tools, and complete the type of work you do every day.

AI models have become capable thought partners — they're able to search the web, write code, and synthesize information at a high level. But being a strong thought partner isn't the same as doing the job. The harder frontier is whether a model can operate as an *agent*: plan across many steps, act inside real software and files, and deliver work a domain expert would actually accept.

That's the gap Project Julius is built to test. We recreate realistic work environments — "worlds" — and write challenging, multi-step tasks inside them, so we can measure where today's best models still fall short of a professional's standard. Your expertise is what makes that judgment possible.

Our Core Mission

Expert Experience

Expert experience is top priority for this project and we want all experts to feel they are on the best project at Mercor.

We will work to make the best systems, processes, staff, and guidelines to make this happen.

Quality Data

This project pushes the boundaries of AI capability. We prioritize quality data and thoughtful work over data that is incorrect or low-effort.

The work of this project will directly impact how systems behave and this project's shared responsibility to align models correctly.

Pushing the Frontier

We want experts on this project to understand they are at the frontier and contribute in their specialized fields.

We encourage all experts to ask thoughtful questions on edge cases, new research, or clarifications that might apply specifically to their domain.

Workstreams

The project is divided into two workstreams: World Building and Task Writing. In Project Julius, each expert owns a world end-to-end — these are not separate roles handed off between people. This sequencing matters: because you built the world yourself, you know exactly where its complexity lives, which makes for richer, more realistic tasks.

World Building

World Building creates simulated environments that mirror the file systems and apps professionals use every day.

The work includes populating folders with artifacts. These artifacts will differ by domain, but they may include public data sets, contracts, financial analyses, meeting notes, presentations, and research. They should contain everything a real team folder would contain.

World builders also generate synthetic data to seed databases such as QuickBooks, ADP Payroll, and Workday. Video walkthrough: [Setting up Agents Playground Golden Trajectories](#) covers configuring golden trajectories in the Agents Playground as you build your world.

Task Writing

Once their world is built, experts design realistic, multi-step tasks within it, based on how work actually gets done and how an LLM should execute it.

Tasks may involve searching for information, running calculations, creating documents, updating databases, analyzing data, or coordinating multiple actions in sequence.

A given task should take an industry expert at least 3-8 hours to execute unassisted by an AI agent.

Tasks are designed at the frontier of the given domain to push AI models to the limits of their reasoning and deep research capabilities. Video walkthrough: [How to Capture Golden Agent Trajectories](#) shows how to record golden agent trajectories for your tasks.

Project Expectations

The following summarizes what we expect from you as a contributor on Project Julius.

Minimum Activity	All experts must check in on Slack most days to see the latest updates. Because each expert owns a world end-to-end — building it, then writing tasks in it — the minimum world building commitment is 30 hours a week. Task writing has a minimum commitment of 20 hours a week. This project's tasks take many hours with guidelines that may change. Staying in touch on the latest and frequently contributing helps your work go smoothly. Notify an EPM or domain lead in advance if you can't meet this in a given week. Repeated misses may put your contract at risk.
Quality and Consistency	Deliver clear, accurate, and well-supported work across every task and workstream.
Onboarding Timeline	Complete onboarding within 48 hours of becoming active. Your spec creation should be submitted within 24 hours of getting access to your domain. Feedback loops start early and you improve with each iteration!
Engagement and Community	Stay responsive in Slack and within task interfaces. Prompt communication keeps workflows smooth and supports coordination between writers, reviewers, and leads. Treat all experts and staff with respect; we are one team.
Efficiency	Work efficiently. Please ensure that you spend the necessary time to complete the quality needed, but do not sit idly on tasks. We expect ~10 hours per task (including all review and revision cycles). Submission means a task sent for review. Completion means a task that has passed our quality standards.

When Logging Time In Insightful, Select The Timer That Matches Your Current Phase

- Project Julius - World Building, if you are actively building or refining your world.
- Project Julius - Task Writing, if you are writing tasks within your world.

Hours, Caps and Scheduling

- Your total hours across all active hourly projects must not exceed 80 hours per week. When you first join you start with a limited hour cap during onboarding; once you submit your plan creation that cap is raised.
- The pay period runs Saturday at 12:00 AM IST through Friday at 11:59 PM IST. A Thursday evening in the Americas can already be Friday in IST, so plan logging near the boundary carefully.
- What counts as paid time: in-scope task and world-building work, Mercor webinars, and approved coordination meetings

or office hours while you are actively engaged. What does not count: idle time, off-scope or unrelated activity, policy violations, and spam.

- Scheduling is flexible; the cap is weekly, not daily, so you can work in whatever chunks suit you. For time off, contact your point of contact through Slack or email; requests are handled case by case.
- See <https://talent.docs.mercor.com/policies/time-tracking-pay>

Strong performance opens opportunities for larger roles on the project, such as reviewer or team lead, along with more consistent work and recognition as a high performer for future Mercor projects.

← Navigate the tabs on the left!



Quick Start Onboarding & Resources

Project Julius
Instruction Guide

Quick Start Guide

You're working at the frontier of AI, testing and improving models on highly complex, long-horizon tasks. The full instruction guide is long, so this guide exists to get you oriented, set up, and confident before you claim your first task.

Day 1 Checklist

Get these things done before you claim your first task. The most common onboarding issue is logging in with the wrong email or browser session, so use a private or incognito window for Mercor resources and confirm your @mercor.expert email is the active account.

Mercor Account Support

Have questions about payments, accounts, Okta, or something else? Mercor maintains centralized talent docs that might be useful to you!

What is Mercor

Mercor is a talent marketplace that connects you (the contractor) with companies that need expert work. Mercor handles your contract, onboarding, time tracking, and payments, so you can focus on the work itself. The company you support on this project relies on Mercor to vet experts, run the workflow, and pay you on a predictable schedule.

Payment

Payouts run every Wednesday for the prior pay period, paid through Stripe (Wise where Stripe is unavailable in your country). See <https://talent.docs.mercor.com/policies/time-tracking-pay>
Time tracking: hourly work is tracked through Insightful (the desktop app is branded Workpuls). See <https://talent.docs.mercor.com/how-to/use-insightful>

Referrals

You can refer other experts from the Referrals tab in your Mercor dashboard. See <https://talent.docs.mercor.com/policies/referrals>

Social Media

You may mention that you work with Mercor using only information that is already public, and you must keep all project details confidential under your CIAA. You are an independent contractor, not a Mercor employee. See <https://talent.docs.mercor.com/policies/social-media>

Taxes

You complete a W-9 (US persons) or W-8BEN (non-US persons) during onboarding. See <https://talent.docs.mercor.com/policies/taxes-work-authorization>

Payment Schedule and Disputes

Payouts run on Wednesdays for the previous pay period and are processed by the end of Wednesday (PST), through Stripe as the primary method and Wise where Stripe is unavailable in your country. Your first payout may sit on a 7-day Stripe hold.

The pay period runs Saturday at 12:00 AM IST through Friday at 11:59 PM IST. Note that a Thursday evening in the Americas can already be Friday in IST, which is the last day of the period.

Common reasons a payout is missed: Stripe or Wise details are out of date, your background check is still pending, or there is a normal bank delay.

Deductions can be triggered by idle timer activity, activity unrelated to your scope, off-scope work, duplicate or double tracking, an AI-policy violation, or spam. Every deduction is reviewed by a human and is not automated.

If you believe a deduction is wrong, reply to the deduction email within the same pay period in which it was issued. Decisions are final once made.

A manual adjustment may be possible case by case if you contact your point of contact with documentation.

Work that is not tracked through Insightful is not eligible for payment.

Project Timeline and End-of-Project

Project duration is variable and there is no guaranteed end date; timelines can shift. We give notice of an end when possible.

You will hear about status changes by email and in Slack. Transitions are not automatic.

Contracts on the project end on the same date unless your individual contract is extended.

Completion badges are finalized once either the project itself officially ends or your individual contract for the project has ended, whichever comes first.

Before any deadline, submit your outstanding hours, finish your deliverables, and keep your Stripe or Wise payout details current.

See <https://talent.docs.mercor.com/support/project>

→ [General Talent Docs](#) | support@mercor.com ←

Onboarding Time Limit

You will have a seven hour cap when you start during which you must read the instructions document, start your world build plan creation, and attend any additional calls/live syncs. Once you progress through the world building stages your hours cap will increase incrementally.

Get Set Up

1

Set Up Okta and Your @mercor.expert Email

Log in at c-mercor.okta.com, then access Gmail from your Okta Dashboard. Use a private or incognito window if you have other Google accounts open.

Account Access details (Okta, Slack, Email)

Your personal email logs you into work.mercor.com, Okta, and Insightful. Your @mercor.expert Google Workspace identity is accessed through Okta; the @mercor.expert contractor email cannot itself be used to log into Okta.

Use Google Chrome, logged into your personal Gmail with sync on, and not an incognito window, when you set up your Okta passkey through Google Password Manager. Firefox is not supported because it does not support the Okta WebAuthn configuration.

You will not receive a separate email invitation. Access is granted through your contract, and Okta single sign-on covers Slack, RL Studio, and Google Workspace.

Slack may take up to 24 hours to synchronize after you complete the Okta setup, so do not worry if it is not immediately available.

If you were issued a @c-mercor.com address, note it is being deprecated after a transition period (with at least a week of notice); you keep both addresses during the transition. Slack usernames are auto-prefixed with [EXP].

2

Set Up Slack

Slack access is auto-provisioned via Okta after you accept the contract. Check to make sure you have joined the all-project channels and your domain channel as soon as you are in.

Onboarding Checklist

Pre-acceptance, in order: set up your Okta account, complete ID verification through Persona (government ID plus a selfie), pass the background check (run through Certn or Zinc; most Certn checks complete within 3 to 7 days), accept the Terms of Work, sign the CIAA (this protects the client's intellectual property), and submit your tax form (W-9 for US persons or W-8BEN for non-US persons).

You cannot begin work until your background check clears.

On acceptance: sign your offer letter and set up payments, with Stripe as the primary method and Wise as the fallback.

Post-acceptance: join Slack through Okta, install Insightful (this is gated to hourly and mixed contracts), and set up your project tools.

You are onboarded when your background check has cleared, your Terms of Work, CIAA, and tax form are submitted, your payout method is set, Slack and Insightful are working, and you have access to your project tools.

Install and Configure Insightful

Insightful tracks your time. Download it, enable screenshots, and confirm it captures the screen you are actively using. It does not run on Chromebooks, iPads, or mobile devices. If you need help, try our Mercor documents or support team email.

Time Tracking with Insightful

You will receive an Insightful invitation from support@insightful.io. If it does not arrive within 48 hours, you can download the app directly at <https://app.insightful.io/#/token/download>. The desktop app is branded Workpuls.

Supported operating systems: Windows 10 or later (.NET 4.8 or later), macOS 11 Big Sur or later (12 recommended), and Linux Ubuntu or Mint 20.04 or later, or Fedora 32 or later (Beta).

Chromebooks, iPads, and phones are not supported.

Three Rules

- clock only one device at a time,
- an internet connection is required (offline or airplane hours do not sync)
- always use the personal email on your Mercor profile.

Common Errors

- a 400 error means your account is still being created, so wait and retry.
- "User not found" message or a prompt to "create organization" usually means your login email changed; do not create a new organization.

- Invitation links expire after 48 hours.

All time is logged in IST. Clock all of your Mercor activity for this project, including active task work and approved calls.

On Project Julius you choose the timer that matches your workstream: select Project Julius - World Building when you are a designated world builder actively building a world, and Project Julius - Task Writing when you are working on tasks.

See <https://talent.docs.mercor.com/how-to/use-insightful>

→ [General Talent Docs](#) | support@mercor.com ←

4

Read the Instruction Guide

Log into your **@mercor.expert** email, then open the full instruction doc. Plan on about an hour reading the relevant tabs before you start work.

We also have a general introduction video here 🙌 [video](#) 🙌

Once You're Assigned to a World

6

Check Your Placement

Please refer to the Slack channels you have been added to and check-in with your Domain Leads and Jason Benoit regarding your specific assignments.

In Project Julius, every expert owns their world end-to-end — you first build your own world, then write the tasks within it. There are no separate world-builder and task-writer roles; the same expert does both.

We recommend saying **hello** 🙌 in your Slack channels for your world and domain!

7

Learn About Your World

**Note: This only applies if you are tasking in legacy worlds. If you have built your own world, there is no knowledge transfer process.*

Please reach out to your Domain Lead regarding the following in any of the legacy tasking worlds:

1. **One Sentence Summary** — high-level framing.
2. **Scouting Report** — per-world context doc covering scenario, key entities, and files.
3. **World Spec ("Map")** — the source of truth. Focus on Entity Profile, Scenario, Consistent Facts, and Artifact Registry.

4. **World Slack channel** (e.g. #consult-w001) — introduce yourself and skim recent threads.
5. **World Google Drive folder /RL Studio files** — the generated files you'll be working with.

Ask your Domain Lead in the world Slack channel if anything is unclear.

Find Your First Task

Once setup is green, head to your domain channel and look for the latest task batch announcement. If you are not sure where to start, post in [#julius-questions](#).

Before you write anything, review the [Task Creation Primer](#) - it walks you through the full workflow from prompt to golden output to rubric.

Meet the Project Julius Team

The reporting chain runs Independent Contractor, then Reviewer, then Domain Lead, then PC, then EPM, then SPL. Knowing who owns what saves a lot of time when something gets stuck.

PROJECT LEADS + MERCOR RESEARCH

Michael Bennett Vaibhav Mittal David Ostrofsky Spencer Sheff
Connor Rooney Ben Curtis Ricardo Haegler Pilnik

Work directly with the client and the broader Mercor team. Follow [#julius-core-announcements](#) for project-wide updates.

PROJECT CONSULTANT (PC)

Jason Benoit

End-to-end teams and talent management & Law Domain.

EXPERT PROJECT MANAGERS (EPM)

MERCOR TECHNICAL PMs (TPM)

Manages day-to-day coordination of the project.

[EXP] Danielle Le... Financial Services & World Building Ops	[EXP] Kylie P Engineering Domain & World Building Ops	Kshitij T. - STPM Platform provisioning, contracts, batch delivery coordination, technical escalations, infrastructure blockers, and high-leverage operational oversight.
[EXP] Matt G Knowledge Domain, Task Quality & Review	[EXP] Chichi Ife... Design Domain & Task Quality	Potta Sai Chaitanya- TPM Technical escalations, infrastructure blockers, back-end support.

DOMAIN LEADS
<i>Own the full pipeline of their domain (throughput, quality, and efficiency). They activate writers, run midpoint reviews, enforce domain standards, and are the primary escalation point for ICs. Newly added for Project Julius: Healthcare (domain lead TBD), in addition to the domains listed below.</i>

Sales + Marketing Rajat G. & Carlos P. & Jason P.	Accounting Joseph B. & Nnenna O.	Human Resources Bart E. & Lade A.
Legal Angela W. & Elizabeth K. & Saulo	Financial Planning & Analysis Ryan W. & Mark G.	Personal Consumer Luke K. & Amanda W.
	Engineering MechE: Lee E. EE: Richard M. CivE: Mushtaq	Healthcare Eliza P & Kanwar

REVIEWERS (TASK WRITING PHASE)
Assigned per domain <i>Final review layer for all task-writing output. Detailed checks on task prompts, golden outputs, calculations, and rubric alignment before approval. Work closely with Domain Leads and the Quality and Review team.</i>

IC Writers: World Builders (artifacts, databases, world spec) and Task Writers (prompts, verifiers, golden output, failure analysis).

Escalation chain: Writer → Domain Lead → EPM → Mercor Team.

● Please Avoid DM'ing Project Leads

Open communication is very important! Post in your world channel and escalate → domain channel → a general channel in order of support! Our team will loop in whoever is needed if even more escalation is needed.

This is the fastest path to a response and makes it easier to track open issues!

Slack Channels: Where to Go for What

Post in open channels rather than DMs whenever possible.

The whole team benefits, and your question gets a faster answer from whoever has context. DMs are best saved for sensitive subjects.

All-Project and Operations Channels

Channel	Who It's For
#julius-core-announcements	Admin-only broadcasts. All-project updates. Read daily.
#julius-questions	Open Q&A for tasks. All experts welcome.
#julius-tech-support	Tech support and Insightful hour-increase requests.
#julius-social	Open fun topics or cat photos. Optional but encouraged.
#julius-domain-leads	Domain Leads and EPMs.
#julius-ops-leads	TPMs, EPMs, Domain Leads, Quality Leads, and Mercor.
#julius-wb-questions	Open Q&A for all world building tasks. All experts welcome.

Domain Channels

Each domain has a top-level channel for domain-wide Q&A.

Domain	Top-Level Channel
Human Resources	#julius-hr

Legal	#julius-law
Financial Planning & Analysis	#julius-fpa
Sales and Marketing	#julius-sm
Accounting	#julius-accounting

Who to Contact for What

Use this decision table before you DM anyone. Almost every question has a channel that gets you a faster, better answer.

If You...	Go Here
Have a question about your task or domain	Post in your domain or world channel and tag your world or Domain Lead. Use the channel rather than a DM so others can chime in.
Hit a tech issue (login, access, broken platform)	Post in #julius-tech-support . Include a screenshot if you can and what you've tried!
Need an Insightful hour increase	Post in #julius-tech-support with your full name, @mercor.expert email, and how much time you have remaining. Ask when you are down to about 2 hours, not earlier.
Have a payment or bonus question	Email support@mercor.com or reach out to a TPM. Include your name, @mercor.expert email, and a screenshot if applicable.
Need live help during work	Join office hours (see below) or post in #julius-questions , or your world or domain channels.
Have something serious that needs escalation	Tag your Domain Lead in the channel thread. They will loop in an EPM if needed. This is the fastest path. Support and Escalation: Point of contact (POC): David Ostrofsky Project Consultant: Jason Benoit Other project leads you may be routed to: Kshitij Tiwari Connor Rooney Ben Curtis Vaibhav Mittal Spencer Sheff Potta Sai Chaitanya For general platform questions and payments, email

	support@mercor.com. Background-check questions also go to support@mercor.com. Escalation order: search the instruction guide and Slack first, then ask in your world or domain channel, then your Reviewer, then your Domain Lead, then the EPM, and finally email support@mercor.com. Response times: daily office hours are held at 11am PST, and your domain also runs its own office hours; check #julius-announcements for the schedule. For anything urgent, post in the relevant channel and tag your Domain Lead.
Are not sure where to ask	Post in #julius-questions . Someone will route you.

How-To Video Library

Short Loom walkthroughs for the most common setup tasks. Bookmark them; you will reach for them again.

How-To	Video Links!
<u>This is an archive of videos, the UI might slightly change</u>	
Welcome Video	Welcome Video
Overview of Project Julius & Kickoff Meeting	Project Overview, Objectives, and Trajectory
Project Julius Onboarding Session	Onboarding Session
Overview of Studio & Prompts	Studio x& Prompts
Overview of Trajectories	Trajectory Annotations
Overview of Steps, Golden Answer, Rubric	Steps, Golden, Rubrics

Ground Rules

These are non-negotiable. Violating any of these will result in removal from the project.

No Sensitive Content

Do not write or request NSFW content, personally identifiable information (PII), or anything harmful,

dangerous, or illegal. If a task prompt seems to ask for it, flag it in your domain channel and do not proceed.

Confidentiality

Do not discuss the project, the models, or any project data outside designated Slack channels. Do not share screenshots, data, or results externally. Mentioning Mercor publicly is fine using only information that is already public; no project specifics.

Treating Others with Respect

Expert experience is #1 on this project, as a result we expect *all experts* to treat each other with respect. Our goal is to be the best project and malicious behavior or attitudes will not be tolerated.

Project Norms and Reminders

A handful of habits that make life easier for you and the team.

What We Expect From You

- Check Slack at least once a day, even on days you are not working. A few minutes to review announcements keeps you in the loop.
- Commitment of 20 hours per week, in whatever chunks work for you. Folks below that tend to miss details and end up with rework. If you will be off for a stretch, give your lead a heads up.
- Only log active hours. If you are sitting idly waiting for something (a trajectory run, etc.), clock out and take a break. **Only log time when you are actively working on a task or on a call with us.**
- Inactive or unresponsive experts **will be removed from this project frequently.**
- Post in open channels, not DMs. Faster answers, and chances are someone else has the same question.
- Stay on task when tasking. Clock out for breaks. Manage timers carefully so we can project capacity and put support where it is needed.

Key Resources (Save These!)

The links you will reach for most often. Bookmark them now and you will not have to dig through Slack later.

Link	Usage
Shared Project Calendar	This is our shared calendar we use that lists all office hours and additional meetings
LLM Reimbursement Form	Please fill this out when purchasing LLM subscriptions, as noted in the LLM Purchase Guide .

Referrals

We do allow for referrals onto this project if you feel a contact would be a good fit. Ideally, they have already been on Mercor projects before and are familiar with rubrics.

Referral Process

- Please fill out this form for referrals
- If it seems like they would be a good fit for our needs and headcount projections, we'll incorporate their review into the next time we onboard more experts.

 [Referral Form Click Here](#) 



Changelog

Change Log & Key Links

Updates to the project guidelines, with most recent entries at the top.

Date	Domain(s)	Change	Author
Sep 23	Task Writing	DB Diff LLM Update	& Person
Sep 14	Task Writing	Step Plan section instructions added	[EXP] Matt G
Sep 11	Task Writing	Added the 100+ step requirement and clarified steps vs. tool calls, production-only scope, the Work Trial exemption, and the Law exception. Updated all model references to Gemini 3.8.	Ricardo Haegler Pilnik
Sept 3	World Building	Included the apps tool in vercel as a link in the Spec section	[EXP] Danielle Lemky
Aug 30	World Building	Added clarity about spec skill and file remediation skill	[EXP] Danielle Lemky
Aug 28	Task Writing	Added Output Dependencies to Rubric Creation	Ben Curtis
Aug 18	World Building	Added Claude Code setup and file remediation skill setup to "Studio Workflow"	[EXP] Danielle Lemky
Aug 17	World Building	Added Banned AI-Default Names to "Key Restrictions" tab	Ben Curtis

Aug 17	Task Writing	Removed Opus 4.8; trajectories and failure analysis now Gemini 3.8 Flash and GPT Sol 5.6 only (5x per Gemini, 3x per Sol)	Ben Curtis
Aug 17	Task Writing	Updated 3.6 Flash to 3.8 Flash, updated failure bar	Ben Curtis
Aug 15	Task Writing	Added trajectory requirement of Sol 5.6 xHigh 5x runs	Ben Curtis
Aug 15	World Building	Naming & realism: hand-authored seed names are now mandatory; pinned the canonical Banned AI-Default Names list (synced with the AutoQC name check); clarified the 555 rule (555-01xx numbers are correct, flag uniform/round data, not the prefix). Restrictions tab → AI-Generated Content Policy	Ben Curtis
Aug 14	World Building	Updated policy on usage of AI-generated names during world building. All AI-generated names must be updated and replaced.	Jason Benoit
Aug 13	World Building	Updated Studio Workflow tab to include app data screenshots and common troubleshooting	[EXP] Danielle Lemky
Aug 13	World Building	Updated the App Glossary to the current 50-item scope and clarified required app use in task ideas. Aligned builder and reviewer checklists and removed stale no-app guidance.	Spencer Sheff
Aug 13	World Building	Clarified Apps usage and app data workflow	[EXP] Danielle Lemky
Aug 12	World Building	Plan removed from world building instructions. World building now begins with SPEC.	[EXP] Danielle Lemky
Aug 12	Task Writing	Excel boilerplate language removed	Ben Curtis
Aug 12	Task Writing	Opus scoring clarification	Ben Curtis
Aug 11	World Building	Update to World building AHT expectations	David Ostrofsky
Aug 10	Engineering	Updated allowed task output file types	Ricardo Haegler Pilnik

Aug 7	LLMs	Removed Gemini from acceptable LLM to use	Ricardo Haegler Pilnik
Aug 6	Engineering	Added International Standards Track exception for the Plan AutoQC "U.S. Scenario" dimension	Ricardo Haegler Pilnik
Aug 5	World Building	Updated reviewer checklists	Ricardo Haegler Pilnik
Aug 5	World Building	Edited Engineering regional requirements	Ricardo Haegler Pilnik
Aug 4	World Building	Added additional tickboxes to spec creation page.	Ricardo Haegler Pilnik
Jul 30	All	Put rubric step before golden	⚙ Person
Jul 30	World Building	Inserted spec link into Spec page.	Ricardo Haegler Pilnik
Jul 29	Task Writing	Insert Task Writing Calibration Session Recording Link	Ben Curtis
Jul 29	Studio Workflow	Added a screenshot clarifying world selection	Ricardo Haegler Pilnik
Jul 29	Task Writing	Clarified there is no difficulty bar for Opus	Ben Curtis
Jul 28	Engineering	Added engineering world of worlds links to: "GETTING STARTED: Claim an Available World" "Claim your world in your domain's" Added Engineering regional requirements	Ricardo Haegler Pilnik
Jul 25	Task Writing	Deprecated "Message in Console" as a valid expected output within Prompting	Ben Curtis
Jul 24	World Building	Added S&M Golden Plan and Spec	David Ostrofsky
Jul 23	World Building	Clarified regional restrictions for experts without U.S. regional expertise. Google specialization and regional form links added for Accounting and Law domains.	Jason Benoit
Jul 22	Task Writing World Building	Removed references and links to the deprecated Global Atlas. Updated onboarding steps in instructions to reflect current workflow.	Jason Benoit
Jul 22	Task Writing World Building	Updated instructions to reflect the deprecated role of World Lead as role no longer activated in current workflow.	Jason Benoit
Jul 21	Task Writing, World Building	Relinked 27 deprecated instruction links to the equivalent tabs within this guide	Ben Curtis

Jul 20	Task Writing	Updated trajectory annotations, failure analysis, moving from long-form to short-form	Ben Curtis
Jul 16	World Building	Updated guidance on Artifact Count, Task Support Requirements,	David Ostrofsky
Jul 11	World Building	Updated workflow to reflect v4 spec / Updated workflow to reflect removal of Defect Log and Playground Walkthrough Annotation / Updated workflow to reflect new Ops team ownership of app data schema checks and Playground verification	[EXP] Danielle Lemky
Jul 7	World Building	Added more guidance on Reference / Anchor Files in the Spec	David Ostrofsky
Jul 7	World Building	Domain specific App Count Requirements clarified	David Ostrofsky
Jul 6	World Building	Added Guidance	Spencer Sheff
Jul 3	World Building	AHT	[EXP] Danielle Lemky
Jul 1	World Building	Spec V4 & updated Plan + Spec AI Skill	David Ostrofsky
Jun 27	World Building	Added this guidance	Spencer Sheff
Jun 26	World Building	Reviewer checklist added	David Ostrofsky
Jun 24	World Building	Added deep dives for: - Spec Creation - File Validation -Playground Validation	Spencer Sheff
Jun 22	World Building	Added Video Curriculum (World Building Basics, Spec Creation Walkthrough, World Creation Walkthrough, Knowledge Transfer)	David Ostrofsky
Jun 22	World Building	Added Checklist for Spec, File Creation, App Creation, and Knowledge Transfer	David Ostrofsky
Jun 22	World Building	Added Change Log tab to track future edits to this guide.	Spencer Sheff
Jun 17	All	Prompt Realism + Verifier Requirements Clarified prompt realism standards and verifier requirements. Prompts do not require personas, should avoid storytelling, reviewer-only context, and	

		artificial time references. Clarified that numeric verifiers must test exact values and may not use acceptable ranges.	
Jun 10	Law	Memo outputs Broadened guidance for Corporate Law to include instructions for Memo outputs.	
Jun 9	All	HR and FP&A ERPNext Guides added along with a general ERPNext Preamble	
Jun 5	S&M	Sales and Marketing ERPNext Guide added	
Jun 3	All	Revamp World Building guidance for multi-app and updated process	
Jun 1	All	Revamp of all project instructions for readability and guidance	
May 26	All	Prompt review failure analysis Updated failure analysis to require: (1) one short paragraph per trajectory for prompt review stage, and (2) expanding the failure analysis to the full four-box structure only after final approval.	
May 22	Law	Verifier guidance for redlining tasks Removed references to 'Visual Grading' in the Corporate Law section. Removed duplicated legal verifier guidance which appeared in the 'Rewards / Verifiers' section.	
May 21	All	Task input files allowed Writers may upload input files as part of their prompt; they should be realistic & not copyrighted	
May 21	All	Prompts do not need to specify files used Prompts do not need to specify specific file names or app tables; they should still be highly specified, however, so that the agent doesn't need to guess what you're looking for as the answer	
May 21	All	Trap and noise files in worlds Trap and noise files are allowed to be part of World filesystems, and are added by World Builders	
May 20	All	Reviewer Resources	

		Added 'Giving Feedback' section to Reviewer Resources, to aid reviewers in structuring their feedback.	
May 19	All	Failure Analysis Updated 'Failure Analysis' tab to reflect the new failure analysis structure (using How, What, Why).	
May 18	All	General Updates Updates to our Quick Start Guide Rolled out referral process More clarity on task creation and onboarding process	
May 13	Investment Banking	Artifacts composition in Investment Banking Worlds Clarified that an ideal IB world combines real-world data from public companies (10-Ks, public statements, stock price history) with synthetically generated supplementary artifacts for a fictitious company (company overviews, press releases, share prices, trading comps). Supplementary artifacts exclude detailed financial documents and models.	
May 12	All	Real World Files Removed requirements for 33% of files to be real world files. Added more specific guardrails on allowed real world file sources to avoid IP issues.	
May 12	All	World Building Updated 'Admin steps for world lead' section to reflect the world building structure.	
May 12	All	Task Examples Updated tasks in 'Examples' section to replace outdated task elements.	
May 11	Investment Banking	Supporting Files Clarified that supporting files are not needed for IB tasks since the main gold output (i.e., the workbook) should contain all calculations.	
May 7	All	Verifier Design Clarified guidance on designing verifiers based on golden output (in 'Rewards / Verifiers'), and tips for crafting lower-weight verifiers and their	

		corresponding prompt requests.	
May 6	Consulting	Supporting Calculations Outputs Updated the Consulting-specific guidance to include instructions for creating Supporting Calculations workbooks.	
May 5	All	LLMaaJ Verifiers Only; Exclude Excel Evals as a Verifier Updated to not allow Excel evals as a verifier (i.e., not programmatic verification). We are only using LMAAJ going forward. Updated the Domain-specific guidance to match.	
May 4	Consulting	Slide Deck Outputs to Require Workbooks Added the requirement to have workbooks alongside the slide deck outputs in Consulting. See the domain-specific tab.	
May 4	Law	Task Type Allowance and Visual Grading Clarified the type of tasks allowed and visual grading. See the domain-specific tab.	
May 4	Investment Banking, Consulting, Law	Updated the Domain-specific guidance to match the old instructions (now deprecated).	

Resources

Project Julius | World Building

Instruction Guide

Video Curriculum

These videos below contain all the information you need to complete a world build end to end.

Topic	Run time	Link to Video
World Building Basics	~11 mins	 World Building Basics
Spec Creation Walkthrough	~11 mins	 Spec Creation Walkthrough
Skill Training - How to Build a World Spec	~5 mins	 Skill Training→ World Spec
World Creation Walkthrough	~11 mins	 World Creation Walkthrough
Knowledge Transfer	~5 mins	 Knowledge Transfer
Golden Spec Walkthrough	~10 mins	 Golden Spec Walkthrough

Key Links

Building

[World Spec Template V5](#)

[App Universe](#) | Password: **ErgonAppsAreGreat!**

- Use to explore your assigned apps and test task idea feasibility

[Skill Library](#)

[Task Seed Log Template](#)

[World Building Blocker Escalation Form](#)

Tracking

[RLS | World of World](#)

[Defect Log \(optional\)](#): only use if requested by world building ops team or domain lead

Knowledge

[World Overview Presentation HTML](#)

Golden Examples

Spec

 **Sales & Marketing**

Worlds

- Law 17 → [RL Studio](#) · [Universe page](#)
- FP&A 4 → [RL Studio](#) · [Universe page](#)
- Accounting 6 → [RL Studio](#) · [Universe page](#)
- S&M 7 → [RL Studio](#) · [Universe page](#)
- HR 6 → [RL Studio](#) · [Universe page](#)



Glossary

Glossary

Reference as you go!

This is a complicated project, but here is a quick list of terms you might see thrown around! Reference below as you move through the instructions!

1. The Project

Project Julius	Mercor's frontier research project building self-contained simulated professional environments and tasks that challenge AI agents to do real expert work.
Frontier-class project	Mercor's internal classification for projects that push the boundary of what current AI models can do. Julius is one. The bar for engagement and quality is higher than typical annotation work, and the volume of writers globally working on this style of data is small.

2. Core AI Agent vocabulary

Agent	An AI system that takes actions in an environment to complete a task. Different from a chatbot in that it operates in a loop: read state, decide, act, observe, repeat.
Model	The underlying large language model that powers the agent's decision-making. On Julius: Gemini 3.8 Flash, GPT Sol 5.6.
Harness	The code wrapper around the model that turns it into an agent. Defines the agent loop, the tools the agent can call, the system prompt, the step budget, and how outputs are emitted. On Julius the default harnesses are 'React Toolbelt Agent (250) - no BUA' and 'React Toolbelt Response api agent (250) - no BUA'. The '(250)' is the maximum step count. 'no BUA' means browser-use capabilities are disabled. Different harnesses on the same model produce meaningfully different agent behavior.

Platform	The set of tools available to the agent inside a world. Different platforms expose different toolsets. Julius examples: 'PDF+Filesystem+Code (no MCP, no BUA)', 'PDF+Filesystem+Code+Nextcloud', 'PDF+Filesystem+Code+Nextcloud+BUA'. The platform is chosen per world based on what tools the agent needs to complete tasks in that domain.
System prompt	Instructions stored on the harness telling the agent how to behave in general (use tools when needed, complete tasks independently, etc.). Writers do not edit this. It is configured by Studio engineering at the harness level.
User prompt	The actual task instruction the writer authors and passes to the agent. This is what you write in the Prompt field on a Julius task.
Trajectory	One complete agent run on one task. Includes the initial messages, every tool call the agent made, every response the model produced, and the final outputs.
Tool call	An action the agent took, for example 'read file X', 'run code Y', 'query table Z'. Each tool call shows up as a step in the trajectory.
Step	One iteration of the agent loop: a single decision, tool call, and observation. The harness step budget caps how many of these the agent gets before it is forced to produce a final answer.
3. Worlds	
World	A self-contained slice of a simulated professional environment. Contains the documents, data, apps, and tools the agent can interact with. Each world is built for one specific domain and persists across all the tasks created against it.
World files	The documents (PDF, .docx, .xlsx, .eml, .pptx, etc.) that live in the world's filesystem. Realistic artifacts the agent can read.
World data / world data tables	Structured data living inside apps within the world, for example records in a Salesforce table, transactions in QuickBooks, employees in ADP. The agent can query these the way a real professional would.
Domain lead	The expert who owns the spec for an entire domain across all its worlds (all Law worlds, all FP&A worlds, etc.). Updates instructions as patterns emerge.
Pod	The team of writers and reviewers attached to one world. Your pod is who you collaborate with day to day.
4. Tasks	
Task	A prompt, set of annotations, trajectories, gold answer, and rubric representing one unit of work. The atomic unit Julius produces.

Prompt (user prompt)	The instruction text the writer authors that the agent receives. Should read like an challenge/ask from someone in the relevant professional role.
Gold response / golden response / gold output	The expert-authored correct answer for the task. Writers create this from scratch by doing the task themselves.
Task input files	Files the writer uploaded specifically for this task, in addition to the world files. The agent can access these the same way it accesses world files.
Rubric	The collection of criteria (verifiers) on a task. Each verifier is one row of the rubric.
Verifier	A single rubric criterion attached to a task that defines a specific thing the agent's output must satisfy. Each verifier has criteria text, a difficulty (Low / Medium / High), and an expected file type. Stored as a Studio record (ver_...). The set of verifiers on a task is the task's grading rubric.
Difficulty	A field on each verifier (Low / Medium / High). High-difficulty verifiers carry more weight in scoring.
Expected output	A custom field on each task indicating what kind of deliverable the agent should produce (e.g. 'Make New Doc', 'Edit Existing Doc', 'Make New Sheet', 'Make New Slide Deck').
Show Your Work	A custom field where the writer documents their reasoning while building the gold response. Used by reviewers and sometimes by downstream training pipelines.

5. Task status lifecycle

Drafting Prompt	Writer is composing the user prompt.
Awaiting Prompt Review	Writer has finished the prompt and submitted for first-pass review.
In Prompt Review	A reviewer has picked it up and is evaluating the prompt only (not the gold or rubric yet).
Crafting	Prompt passed review; writer is building out the golden response and verifiers.
Awaiting Review	Full task submitted for second-pass review.

In Review	A reviewer is evaluating the complete task.
Approved	Task passed both reviews and shipped.
Rejection loop	Rejection at either review stage loops the task back to the prior writing stage for rework. A task only ships once it clears both reviews.

6. People and roles

Writer	Expert who authors tasks: prompt, gold response, verifiers.
Reviewer	Expert who QAs other writers' tasks at the prompt-review and final-review stages.
Domain lead	Owns a domain's spec across all its worlds.
EPM (Engagement / Project Manager)	Mercor staff member running project operations: ramp, throughput, communications.
DRI (Directly Responsible Individual)	The single person accountable for a given workstream.
Mercor Research	The internal team that sets the research direction Julius serves.

7. Process and ops vocabulary

AHT (Average Handling Time)	How long, on average, a writer or reviewer spends per task.
Insightful	The time-tracking tool. Use the 'Project Julius - World Building' timer when world building and 'Project Julius - Task Writing' when writing tasks. Clock in when you start, clock out when you stop.
Time fraud	Logging time on Insightful while not actually working. Cause for removal.
Pacing	The expected weekly volume of completed tasks. Driven by the project's milestone targets.

Office hours	Daily live sessions for questions and walk-throughs. Times and links live in the master instructions doc.
8. Acronyms	
BUA	Browser Use Agent
MCP	Model Context Protocol
AHT	Average Handling Time
EPM	Expert Project Manager
DRI	Directly Responsible Individual
QC	Quality Control
RLS	RL Studio (the platform's earlier name; still the Linear team key)
PII	Personally Identifiable Information
NSFW	Not Safe For Work (nudity, profanity, etc.)
LLM	Large Language Model

App Glossary

App Glossary

Current scope

This glossary is the current production and active testbed scope: 8 general services and 42 domain apps, for 50 services/apps in total.

Use only the apps named in App Configuration for the task. Do not add, remove, or replace an app. If a required app does not start or its data does not load, tell Apps Ops and wait.

How interfaces work

- Normal domain apps use MCP. The agent reads and acts through MCP.
- A seeded platform may also contain a BUA service only because the CSV loader lives there. That BUA is not an agent interaction surface.
- Personal/Consumer apps are the exception. Chatwoot, Easy!Appointments, OpenTripPlanner, and WooCommerce use BUA.
- FreeCAD and KiCad are no-seed file apps. They use MCP against files included in the world.
- Loader success is not app proof. For a normal seeded app, confirm the loaded data through the app's MCP.

What App Configuration tells you

- App name: the exact app to include.
- Version: the approved app build.
- Seed or file source: the world data to load or the files to attach.
- Credentials: only when a Personal/Consumer browser app needs them.

Apps by area

- General services: File system; Email; Calendar; PDF; Word; Excel; PowerPoint; Code execution.
- Shared: Wiki.js; Nextcloud.

- Financial services and accounting: ERPNext; QuickBooks; Fineract; OpenInsure; FASLR; Caseware OnPoint; FloQast; FP&A Forecaster; GoSystem Tax RS; NetSuite; ONESOURCE Tax Provision; Optro; Workiva.
- Law: DocuSeal; Docassemble.
- HR and recruiting: BambooHR; SAP SuccessFactors Recruiting; SAP SuccessFactors Onboarding; Workday; Greenhouse.
- Projects and operations: Atlassian; OpenProject; Odoo; Gitea; NocoDB; CKAN.
- Personal/Consumer: Chatwoot; Easy!Appointments; OpenTripPlanner; WooCommerce.
- Data and analytics: JupyterHub; Metabase; dbt; MySQL; Grafana.
- Education and research: Canvas; OpenEduCat; Zotero.
- Engineering: FreeCAD; KiCad.

General services

File system

Interface: MCP

What it is: The shared files and folders inside the world.

What the agent can do: List, search, read, create, rename, move, edit, and delete world files.

Example task: Find the source files for a report, update the report, and save the result.

Login: None.

Email

Interface: MCP

What it is: The world's seeded mailbox. This is the standard Mail app, not Gmail.

What the agent can do: Read messages and threads, search mail, manage drafts, and send mail when the task requires it.

Example task: Find the latest customer message and draft a reply using the facts in the world.

Login: None. The mailbox is pre-authenticated.

Calendar

Interface: MCP

What it is: The world's seeded calendar.

What the agent can do: Read, search, create, update, and delete calendar events.

Example task: Find an open time and schedule a review meeting.

Login: None. The calendar is pre-authenticated.

PDF

Interface: MCP

What it is: Tools for reading and producing PDF documents.

What the agent can do: Extract text, inspect pages, combine information, and create a final PDF when requested.

Example task: Read a policy PDF and produce a short PDF summary.

Login: None.

Word, Excel, and PowerPoint

Interface: MCP

What it is: The three Office document services used for .docx, .xlsx, and .pptx files.

What the agent can do: Read, create, and update documents, workbooks, and presentations.

Example task: Update an Excel model, write a Word memo, and prepare a PowerPoint summary.

Login: None.

Code execution

Interface: MCP

What it is: A controlled runtime for scripts and data work inside the world.

What the agent can do: Run code to inspect data, calculate results, transform files, and support the task.

Example task: Calculate monthly totals from CSV files and save a checked output.

Login: None.

Domain apps

Atlassian

Interface: MCP

What it is: A combined Jira, Confluence, and Bitbucket work system.

What the agent can do: Manage issues, search and edit wiki pages, and inspect repositories and code.

Example task: Trace a product issue from its Jira ticket to the related Confluence plan and Bitbucket code.

Login: None. The MCP is pre-authenticated.

BambooHR

Interface: MCP

What it is: A human resources information system.

What the agent can do: Read and manage employees, time off, benefits, documents, and HR records.

Example task: Review an employee record and check the status of a time-off request.

Login: None. The MCP is pre-authenticated.

Canvas

Interface: MCP

What it is: A learning management system for courses, assignments, grades, and discussions.

What the agent can do: Read and manage course content, enrollments, assignments, submissions, and grades.

Example task: Review late submissions and update the course assignment plan.

Login: None. The MCP is pre-authenticated.

Caseware OnPoint

Interface: MCP

What it is: An audit engagement and workpaper platform.

What the agent can do: Manage engagements, trial balances, journals, materiality, risks, controls, work programs, signoffs, statements, and auditor reports.

Example task: Review an engagement's risk and materiality data, then identify incomplete audit work.

Login: None. The MCP uses a fixed, pre-authenticated identity.

Chatwoot

Interface: BUA

What it is: A Personal/Consumer customer messaging inbox.

What the agent can do: Use the visible app to review contacts, conversations, and messages and respond when the task allows it.

Example task: Find the customer conversation about a delivery problem and send the approved reply.

Login: Use the credentials in App Configuration.

Important: Chatwoot is a Personal/Consumer exception. The browser UI is the interaction surface.

CKAN

Interface: MCP

What it is: A data catalog for datasets, organizations, groups, and resources.

What the agent can do: Search, inspect, create, and update catalog records and their resources.

Example task: Find the authoritative dataset for a city metric and report its latest resource.

Login: None. The MCP is pre-authenticated.

dbt

Interface: MCP

What it is: An analytics engineering system for models, tests, sources, and lineage.

What the agent can do: Inspect projects, models, dependencies, runs, tests, and documentation.

Example task: Trace a failed data model to its upstream source and explain the impact.

Login: None. The MCP is pre-authenticated.

Docassemble

Interface: MCP

What it is: A guided legal interview and document automation system.

What the agent can do: Manage interviews, sessions, answers, templates, and generated legal documents.

Example task: Complete a client intake and generate the requested legal form.

Login: None. The MCP is pre-authenticated.

DocuSeal

Interface: MCP

What it is: An electronic signature and document workflow system.

What the agent can do: Manage templates, submissions, recipients, fields, and signing status.

Example task: Prepare a signature packet and report which recipient still needs to sign.

Login: None. The MCP is pre-authenticated.

Easy!Appointments

Interface: BUA

What it is: A Personal/Consumer appointment scheduling app.

What the agent can do: Use the visible app to book, reschedule, or cancel appointments and manage services, providers, customers, and calendars.

Example task: Book a service with the requested provider at an available time.

Login: Use the credentials in App Configuration.

Important: Easy!Appointments is a Personal/Consumer exception. The browser UI is the interaction surface.

ERPNext

Interface: MCP

What it is: An enterprise resource planning system for accounting, sales, purchasing, inventory, and operations.

What the agent can do: Read and manage customers, suppliers, items, invoices, payments, stock, and accounting records.

Example task: Check an unpaid invoice, its customer, and the related payment status.

Login: None. The MCP is pre-authenticated.

FASLR

Interface: MCP

What it is: A financial accounting standards and ledger reporting system.

What the agent can do: Work with ledgers, accounts, entries, periods, reports, and supporting accounting records.

Example task: Review period activity and explain the entries behind a balance.

Login: None. The MCP is pre-authenticated.

Fineract

Interface: MCP

What it is: A core banking and lending system.

What the agent can do: Manage clients, accounts, loans, savings, repayments, transactions, and branches.

Example task: Review a borrower's loan status and recent repayments.

Login: None. The MCP is pre-authenticated.

FloQast

Interface: MCP

What it is: A financial close management platform.

What the agent can do: Manage close checklists, reconciliations, journal entries, variance analysis, entity consolidation, statements, signoffs, and review notes.

Example task: Find overdue close work and summarize the items blocking signoff.

Login: None. The MCP is pre-authenticated.

FP&A Forecaster

Interface: MCP

What it is: A planning, budgeting, forecasting, and scenario analysis system.

What the agent can do: Manage accounts, assumptions, dimensions, models, scenarios, data imports, and forecast outputs.

Example task: Compare the base and downside scenarios and explain the main revenue change.

Login: None. The MCP is pre-authenticated.

FreeCAD

Interface: MCP

What it is: A computer-aided design tool for 3D engineering files.

What the agent can do: Inspect and modify supported CAD files through MCP file operations.

Example task: Open a world CAD file, change a named dimension, and save the revised file.

Login: None.

Important: FreeCAD is a no-seed file app. It works from world files, not seeded app records.

Gitea

Interface: MCP

What it is: A source code hosting and collaboration system.

What the agent can do: Read and manage repositories, files, commits, branches, issues, pull requests, and comments.

Example task: Inspect a pull request and connect its code changes to the reported issue.

Login: None. The MCP is pre-authenticated.

GoSystem Tax RS

Interface: MCP

What it is: A corporate tax compliance system.

What the agent can do: Work with entities, returns, federal and state forms, schedules, fixed assets, credits, payments, diagnostics, tie-outs, workpapers, and e-file status.

Example task: Review a return's diagnostics and identify the items that block filing.

Login: None. The MCP is pre-authenticated.

Grafana

Interface: MCP

What it is: A monitoring and observability system for dashboards, metrics, and alerts.

What the agent can do: Search dashboards, inspect panels and data, review alerts, and explain operational trends.

Example task: Find the dashboard behind an alert and summarize the affected service and time window.

Login: None. The MCP is pre-authenticated.

Greenhouse

Interface: MCP

What it is: A recruiting and applicant tracking system.

What the agent can do: Manage jobs, candidates, applications, interviews, scorecards, offers, and hiring activity.

Example task: Review a candidate's interview feedback and report the next hiring step.

Login: None. The MCP is pre-authenticated.

JupyterHub

Interface: MCP

What it is: A notebook environment for analysis and reproducible data work.

What the agent can do: Inspect notebooks and files, run supported analysis, and save results.

Example task: Analyze a dataset in a notebook and save the checked findings.

Login: None. The MCP is pre-authenticated.

KiCad

Interface: MCP

What it is: An electronic design automation tool for circuit and PCB files.

What the agent can do: Inspect and modify supported schematic and board files through MCP file operations.

Example task: Read a world schematic, update a component value, and save the revised design.

Login: None.

Important: KiCad is a no-seed file app. It works from world files, not seeded app records.

Metabase

Interface: MCP

What it is: A business intelligence system for questions, dashboards, and saved analysis.

What the agent can do: Search data, inspect questions and dashboards, run analysis, and report results.

Example task: Find the dashboard for customer retention and explain the largest month-over-month change.

Login: None. The MCP is pre-authenticated.

MySQL

Interface: MCP

What it is: A relational database used for structured application data.

What the agent can do: Inspect schemas and run safe queries to read and analyze data.

Example task: Query order data and calculate revenue by customer segment.

Login: None. The MCP is pre-authenticated.

NetSuite

Interface: MCP

What it is: An enterprise accounting and ERP system.

What the agent can do: Work with general ledger, AP, AR, cash, bank reconciliation, inventory, fixed assets, revenue, payroll, close, budgets, tax, reports, and audit trails.

Example task: Trace a balance from the financial report to its transactions and audit history.

Login: None. The MCP is pre-authenticated.

Nextcloud

Interface: MCP

What it is: A collaborative file storage and sharing system.

What the agent can do: Search, read, upload, organize, share, and manage files and folders.

Example task: Find the latest project file, update its folder, and share the correct version.

Login: None. The MCP is pre-authenticated.

NocoDB

Interface: MCP

What it is: A database-style workspace with tables, records, fields, and views.

What the agent can do: Read and manage bases, tables, columns, records, and views.

Example task: Find incomplete records in a project table and update their status.

Login: None. The MCP is pre-authenticated.

Odoo

Interface: MCP

What it is: A business operations suite for CRM, sales, purchasing, inventory, accounting, and projects.

What the agent can do: Read and manage operational records across the enabled business modules.

Example task: Follow a sales order from the customer record through delivery and invoicing.

Login: None. The MCP is pre-authenticated.

ONESOURCE Tax Provision

Interface: MCP

What it is: An ASC 740 corporate income tax provision and close system.

What the agent can do: Manage units, datasets, permanent and temporary differences, NOLs, valuation allowances, journals, payable and return-to-provision data, footnotes, and provision reports.

Example task: Review the effective tax rate and explain the largest reconciling item.

Login: None. The MCP is pre-authenticated.

OpenEduCat

Interface: MCP

What it is: An education management system for students, courses, faculty, attendance, and academic records.

What the agent can do: Read and manage education records and academic operations.

Example task: Review a student's enrollment and attendance, then identify missing coursework.

Login: None. The MCP is pre-authenticated.

OpenInsure

Interface: MCP

What it is: An insurance policy and claims management system.

What the agent can do: Manage customers, policies, products, premiums, claims, payments, and related records.

Example task: Review a claim against its policy and report the current status and next action.

Login: None. The MCP is pre-authenticated.

OpenProject

Interface: MCP

What it is: A project management system for projects, work packages, schedules, and teams.

What the agent can do: Read and manage projects, tasks, status, assignments, dates, comments, and relationships.

Example task: Find blocked work packages and summarize what must happen next.

Login: None. The MCP is pre-authenticated.

OpenTripPlanner

Interface: BUA

What it is: A Personal/Consumer multimodal trip planner using transit and map data.

What the agent can do: Use the visible app to plan walking, cycling, driving, or transit trips and save, review, or delete trips.

Example task: Plan an accessible transit trip that arrives before the requested time.

Login: None.

Important: OpenTripPlanner is a Personal/Consumer exception. The browser UI is the interaction surface.

Optro

Interface: MCP

What it is: An enterprise governance, risk, compliance, and internal audit system.

What the agent can do: Manage frameworks, risks, controls, assessments, audits, issues, vendors, screenings, workpapers, reports, remediation, and workflows.

Example task: Trace a high-risk finding to its control, owner, evidence, and remediation status.

Login: None. The MCP is pre-authenticated.

QuickBooks

Interface: MCP

What it is: An accounting system for customers, vendors, invoices, bills, payments, banking, and reports.

What the agent can do: Read and manage accounting transactions and financial records.

Example task: Find overdue invoices and calculate the customer's open balance.

Login: None. The MCP is pre-authenticated.

SAP SuccessFactors Onboarding

Interface: MCP

What it is: An employee onboarding system.

What the agent can do: Create and review onboardees, processes, status, new-hire activities, meetings, equipment tasks, and goals.

Example task: Review a new hire's onboarding process and identify incomplete activities.

Login: None. The MCP uses a preconfigured role.

SAP SuccessFactors Recruiting

Interface: MCP

What it is: A recruiting system for requisitions, candidates, applications, interviews, and offers.

What the agent can do: Read and manage recruiting records and hiring workflows.
Example task: Review applicants for a requisition and report who is ready for the next stage.
Login: None. The MCP is pre-authenticated.

Wiki.js

Interface: MCP

What it is: A team knowledge base.

What the agent can do: Search, read, create, and update pages and organize knowledge.

Example task: Find the current operating procedure and update it with an approved change.

Login: None. The MCP is pre-authenticated.

WooCommerce

Interface: BUA

What it is: A Personal/Consumer online store built on WordPress and WooCommerce.

What the agent can do: Use the visible app to browse products and work with the cart, checkout, orders, and account features required by the task.

Example task: Find the requested products, complete the cart, and verify the order details before checkout.

Login: Use the credentials in App Configuration when the task requires an account.

Important: WooCommerce is a Personal/Consumer exception. The browser UI is the interaction surface.

Workday

Interface: MCP

What it is: A human capital management system.

What the agent can do: Read and manage workers, organizations, jobs, compensation, time, recruiting, and related HR records.

Example task: Review a worker's current role and compensation history for an approved HR analysis.

Login: None. The MCP is pre-authenticated.

Workiva

Interface: MCP

What it is: A connected financial reporting and technical accounting system.

What the agent can do: Work with documents, presentations, spreadsheets, links, collaboration, revisions, SEC filing, XBRL, Wdata, management reporting, dashboards, KPIs, and audit logs.

Example task: Trace a reported number through linked source data and prepare the disclosure for review.

Login: None. The MCP is pre-authenticated.

Zotero

Interface: MCP

What it is: A research library for sources, metadata, notes, tags, collections, and citations.

What the agent can do: Search and organize research items, inspect metadata and notes, and build citation-ready source sets.

Example task: Collect the sources for a literature review and organize them by topic.

Login: None. The MCP is pre-authenticated.



Key Restrictions

Project Julius
Instruction Guide

Key Restrictions

Legal Restrictions

Do Not Upload Proprietary or Prior Work

You may not upload resources that you created for your job or for another project. You may use these documents as inspiration, but do not directly upload any work produced for another purpose. No proprietary information may be uploaded or used within any world or task in this project.

Artifacts Found Online

You will likely be doing research on the internet to populate the world. Download any relevant resources and upload them to the Within World folder. The bullets below cover the categories you must check before using anything you find.

- **Paywalled content:** you can only use files that are publicly available on the internet; anything behind a paywall or login is unacceptable.
- **Copyrighted content:** carefully review all materials to determine whether they are subject to copyright; anything under copyright is unacceptable.
- **Licensed content:** do not include any content that requires a paid license, special permission, or restricted use. Only use content that is clearly free for public use.
- **Attribution requirements:** if content is licensed for reuse with attribution (*e.g.*, Creative Commons), ensure proper attribution is included and documented.
- **Questions or uncertainty:** if you are unsure whether content is trademarked, licensed, or otherwise restricted, escalate the question to your team lead before uploading.

Stick to Public or Government Sources

Default to resources from a government entity or content that is explicitly in the public domain. When unsure whether something is trademarked, licensed, or otherwise restricted, escalate to your team lead before uploading.

AI Usage in Task Writing

AI tools are allowed for ideation and drafting (i.e., as “thought partners”). They are very helpful at scaffolding tasks, giving feedback, and understanding trajectory failures.

However, our goal is to push **beyond the frontier of models’ capabilities**. Using AI models is not a substitute for your domain expertise in doing the tasks.

- You are fully responsible for everything you submit. If it’s wrong, that’s on you—not the AI.
 - Verify every value, calculation, and piece of logic manually.
 - All values must be grounded. No made-up facts, no hallucinated details.

 In particular, your prompt, Show Your Work files, completion steps, and golden output must be **your own output**. Prompts and golden outputs must be realistic. We do not want everything to be repetitive, generic, or “AI-sounding”. Rewrite everything in your own words.

Ultimately, we trust you to do your work and use your expert judgement. We believe AI tools can help you to be more efficient; but we do not want quality to fall.

AI and External Tool Usage Policy

Default Mercor policy: you may use an LLM to fix grammar and diction and to lightly refine phrasing, provided that the fundamental ideas and structure remain your own. Do not ask an LLM to rewrite your responses wholesale. You may not use LLMs to evaluate or grade model responses, to write your justifications or explanations, or to predict code behavior or runtime results. Submissions that are evidently produced by artificial intelligence will be identified and flagged. See <https://talent.docs.mercor.com/policies/llm-usage>

Project-specific exceptions: on Project Julius, running frontier models inside RL Studio trajectories (for example the Gemini, Claude, and GPT model families) is a sanctioned and expected part of the work, since the goal is to prove where these models fail. Personal Pro subscriptions used for the project are reimbursable through the LLM Reimbursement Form, as described in the LLM Purchase Guide. Synthetic, AI-generated world-building materials are expected for fictional companies and places, subject to the rules in the AI-Generated Content Policy below.

AI-Generated Content Policy

World builders may use AI tools (Claude) to synthetically generate materials for some phases of the project. **Core tasks and golden outputs must be created by yourself.**

Some world materials may include interview transcripts, survey results, financials for a fake company, and similar artifacts.

- All AI-generated material should be **thoroughly reviewed** to ensure that it is realistic, appropriate for the scenario, and does not violate any rules of the project.
- Any mention of the AI used within the document produced should be eliminated (no references to ChatGPT-generated docs or similar).
- You must only AI-generate materials for fictional companies or places. Financials or other company-level data must be for a fictitious company.
- **You must hand-author every seed entity (core company, key people, and named counterparties) from scratch, and never use AI-generated names verbatim** — recurring defaults like "Meridian," "Acme Corp," "TechNova Solutions," "Globex Corporation," "Apex Dynamics," "John Smith," or "Jane Doe," or "Vera" create realism issues, obvious AI fingerprints, and repeated overlap across worlds. Banned AI-default names are a hard send-back; the full canonical list (single source of truth, synced with the AutoQC name check) is here: [Banned AI-Default Names List](#)
- Separately, 555-01xx phone numbers are the correct fiction convention (keep them); the realism concern is uniform or round data (identical figures across periods), not the 555 prefix. **Use AI for inspiration, then spend a minute making every name feel natural, unique, and domain-appropriate before you use it.**

Fictional Entities Only

AI-generated financials or other company-level data only for fictitious companies and places. Never AI-generate materials representing a real company or real person.

Don't Use AI Names Verbatim

AI-generated company, stakeholder, and personnel names should never be used exactly as suggested. Spend a minute making each one feel natural, unique, and appropriate for the domain before it goes into a world.

Banned AI-Default Names

Mirrors the [canonical Banned AI-Default Names doc](#) (the single source of truth, also referenced by the AutoQC name-presence check `wow_banned_ai_default_names`). Edit the canonical doc to change the list everywhere; this section is a copy. Last updated 2026-08-15.

How to use

Hand-author seed names from scratch (core company, key people, named counterparties); never ship a name an AI proposed first. Match case-insensitively on whole-word boundaries. Tier A = banned: any occurrence is a defect, remove or rename before submit. Tier B = review: frequent AI defaults that also collide with real words, so flag and sanity-check rather than hard-fail.

Tier A — banned (hard-fail)

Companies: Meridian, Acme, Globex, TechNova, Contoso, Cyberdyne, Initech, Vandelay, Umbrella Corporation, Stark Industries, Wayne Enterprises, Hooli, Soylent, Wonka, Pied Piper, Northwind Traders, Oscorp.

Placeholder people: John Smith, Jane Doe, John Doe.

Meridian also covers compound forms (Meridian Corp / Inc / Group / Partners / Labs / Systems / Bank / Capital) and meridian.com-style email domains.

Tier B — review (flag, not fail)

Nexus, Apex, Zenith, Vertex, Vantage, Summit, Horizon, Pinnacle, Catalyst, Synergy, Vanguard, Paramount, Keystone, Cornerstone, Beacon, Nimbus, Lumen, Orion, Atlas, Cascade, Sterling, Evergreen, Emerald, Vera, Northwind.

555 phone numbers

555-01xx numbers are the correct fictional convention; keep them. The realism problem is uniform or round data (identical figures across periods) and unfilled placeholder blanks such as 555-555-5555, not the 555 prefix itself.

Phase 1: World Creation Process

World Building Primer

This guide covers what a world is, how to build one, and how to document its structure in knowledge transfer documents. Work through each section in order.

▲Note: All experts unless notified in specific internally-based cases in Accounting and Law will be World Builders and will build worlds.

What is a World?

A world is a realistic, self-consistent corporate, personal, or organizational digital environment populated with documents, data, applications, and personas that an AI agent navigates to complete tasks. A world is not a single task; it is the environment that supports multiple tasks.

When you build a world, you create a fully realized organization with the kind of messy, interconnected documentation that exists inside real companies. The agent must search through these artifacts, cross-reference information, handle inconsistencies, and synthesize conclusions, just as a human knowledge worker would.

What makes a great world?

- Represents a specific organization (company, hospital, law firm, government agency) with a realistic operational context. Define the industry, size, geography, and other key characteristics.
- Contains **~50 artifacts** across multiple modalities
- Organized in folder hierarchies that mirror real organizational structures. A compliance team files differently than a sales team; your world should reflect this.

- Information is deliberately scattered. No single document contains everything an agent needs; the agent must discover, navigate, and cross-reference across multiple sources.
- Contains both File System data as well as Application data.
- Supports 2-6 tasks that will fail the model
- Contains defendable "traps" that are intended to make the world taskable and challenging for the agent.

For context, our Auto QC checks for the following dimensions, and more:

-
- | | |
|---|--|
| • Realistic Document Texture And Markings | • Personnel Names, Titles, And Roles Are Consistent |
| • File Volume And Format Diversity | • No Broken Document References |
| • Supports Independent Tasks | • Document Chronology Is Causally Consistent |
| • No Blind Filesystem Searches Needed | • Consistent Naming |
| • No Synthetic Markers Or Placeholder Data | • All Numerical Values And Constraints Are Consistent Across The World |
| • Coherence Flag Ratio | • Voice, Tone, And Formatting Vary By Author And Document Type |
| • Anomaly Detection Catch-All | • Traceability And Cohesion |
| • No Synthetic Substitutes For Real-World Facts | • Realism Flag Ratio |
| • External Citations Resolve To Real Content | • File Integrity Defect Rate |
| • In-World vs Out-Of-World Separation | |
| • Timeline And Dates Are Coherent | |
-

Real vs fictional entities

- The core entity of your world must be fictional. The central organization your scenario is about (and its people) is invented, and all of its data is synthetic.
- Real companies may appear as supplementary material, not as the core entity or a central party. When you include a real company, you must use its real, publicly available data: for example, a 10-K, an earnings release, a public statement, or other documents downloaded from the open web. Never fabricate, alter, or synthesize data for a real company.
- Match the data to the entity. Fictional entities get synthetic data; real entities get their real public data. Never mix the two: no synthetic numbers on a real company, and no real filings repurposed as your fictional core entity's own records.
- Do not let a "fictional" name accidentally match a real one.

How to pick and check a name:

- Invent a clearly made-up name
- Search it on the web in quotation marks with the industry word (e.g. "Northwind Components" distributor).
- If any real company or person with a similar name could be confused for yours, change it. When in doubt, change it.
- Real public bodies (the IRS, a court, U.S. GAAP) are fine to reference. A real private company is fine only as supplementary material with its real public data, and must never be the core entity or a named central party.

How We Measure World Quality

We measure quality against three north stars: **depth**, **realism**, and **factual integrity**. Each one is the difference between a world that supports a task and a world that supports a real evaluation of agent reasoning.

Depth Enough material to support 5 complex tasks that current AI models cannot complete correctly. Driven by complexity, interconnectedness, and the number of core files.	Realism Matches the texture, friction, and irregularity of real workplaces. Signals are buried in noise, just like real-life production. See the Traps and Noise table on the next page.	Factual Integrity Every shared fact, figure, and entity is consistent across every artifact in the world unless it is a planned trap. No contradictions across files.
--	--	---

Depth

A good world has enough material to support ~5 complex tasks. Complexity, interconnectedness, traps, and the number of core files are the most important drivers. We recommend ~50 file system artifacts, depending on your domain's complexity, plus app data. Use your discretion—remember that the world is fixed at the end of the build process and no other files can be added during the task-writing phase. Make sure there's enough substantive content to support lots of varied tasks.

▲Note: A world is *FIXED* after it is completed and no revisions will be allowed once task writing begins!

Artifact types

Different artifact types serve different purposes. Use the mix below to build depth.

Type	Format	When to Use
------	--------	-------------

File System	Word documents	Internal planning docs, call notes, etc.
	Spreadsheets	Financials, forecasts, data history, reports, and schedules. Must align with real-world professional practices.
	Presentations	Executive decks, meetings, presentations, etc.
	PDFs	Documents, regulatory or publicly available docs, any static content no longer being updated internally.
App	Application Data	Mock applications that may mimic popular ERPs, CRMs, knowledge bases, or other business tools. Used to test an agent's ability to retrieve and parse data, transfer information, update fields, change record states, and execute end-to-end software workflows.

Why World Depth Matters

The richness of the world directly determines the quality and difficulty of the tasks it can support. A shallow world with a handful of documents produces shallow tasks.

A deep world with layered dependencies and cross-document references produces the long-horizon, multi-step tasks that genuinely test agentic capabilities.

Realism

A world is only as useful as its similarity to the real-world context. If it doesn't mimic the texture, friction, and irregularity of how things actually work in the real world, the tasks built on top of it won't reflect real challenges either. Realism is what turns a setting into a stress test: signals buried in noise, etc., are exactly what models and workers must learn to navigate in production.

This fidelity is critical as we train models to perform real-world tasks. In the QC stage, we ensure world realism by identifying:

- Stylistic consistency

- Any markers of synthetic generation
- Document texture and more.

Traps and Noise

In-world noise and/or traps are encouraged when they create realistic reasoning challenges. They are prohibited when they make tasks unsolvable.

✓ Allowed/Defendable	✗ Not allowed
• Embedded rules—only if stated clearly in a findable place (e.g., an email) • Sequential, version, or recency-based canonicity • Canonicity by seniority of the source • Canonicity by explicit statement • Canonicity by official doc over email • Volume noise: many tables, many records, many files • Company-specific policy or guidance; must be sensible, findable, clearly presented	For example: • Unsolvable tasks, i.e., tasks that rely on information/assumptions not given in the world or prompt • Highly subjective matters of taste with no in-world reasoning path • Unrealistic, non-findable embedded rules (e.g., a screenshot of a doc in a random folder with an alphanumeric title containing the only missing value)

Factual Integrity

Factual integrity is the connective tissue of a world. It is the shared facts, figures, and entities that anchor every document to a single underlying reality. When a name, number, or date (referred to as “thresholds”) appears in one artifact, it must hold true everywhere else it surfaces; otherwise, the world stops being one world and becomes a collection of contradictory drafts. This consistency is what makes a world reasoning-worthy: tasks can only test real synthesis and judgment when the underlying facts actually line up across files.

To hold these key factoids consistent across the artifact of the worlds, you will be designated thresholds in your World Specs [\(see more details here\)](#).

Regional Requirements

The scope of all worlds, tasks, scenarios, and evaluations must be based on **United States jurisdiction, norms, standards, and professional practices.**

Domain-specific content should reflect U.S. frameworks and expectations.

Examples include, but are not limited to:

- **Accounting:** U.S. GAAP, U.S. financial reporting standards, and U.S.-based business practices.
- **Law:** U.S. federal and state legal systems, regulations, case law, and legal procedures.
- **HR, Finance, and other regulated domains:** U.S. laws, regulations, terminology, and operational practices.
- **Engineering:** Two standards tracks. Engineering is the exception to the U.S.-only rule above. Every engineering world is built to exactly one standards track, U.S. or International, declared at the Spec stage and marked in the world name as [U.S.] or [INTL]. Cite only the codes and use only the unit conventions of that world's track. Do not mix tracks within a world.
 - U.S. track (U.S. customary units where conventional):
 - Civil / Structural: IBC, ASCE 7, ACI 318, AISC 360, AASHTO.
 - Electrical: National Electrical Code (NFPA 70), IEEE and NESC standards, UL listing conventions.
 - Mechanical: ASME codes and ASHRAE standards.
 - International track (SI units):
 - Civil / Structural: Eurocodes (EN 1990, 1991, 1992, 1993, 1997, 1998) and ISO structural standards (e.g., ISO 2394, ISO 13822).

- Electrical: IEC standards (e.g., IEC 60364, 60947, 61439, 61850) and IEC-aligned national wiring rules.
- Mechanical: ISO standards (e.g., ISO 286, ISO 2768, ISO 1101 GPS, ISO 4126) and ISO/EN building-services standards (e.g., EN 16798, ISO 52000 series) in place of ASHRAE.

Content, references, or examples originating from outside the United States are **not prohibited** and may be incorporated when relevant. However, for all domains except engineering, the center of gravity of each world should remain within the U.S. context. Core tasks, expected solutions, evaluation criteria, assumptions, and domain expertise should be grounded in U.S. requirements and conventions unless the task explicitly calls for international or comparative analysis.

Law, Accounting, and Engineering Experts: During onboarding, you must complete the required Google form specifying your specialization(s) and any regional restrictions.

- [Law Form](#)
- [Accounting Form](#)
- [Engineering Form](#)

Most experts who are not well-versed in U.S. jurisdictions, codes, and standards are not eligible to world-build.

HOWEVER, engineering experts may world-build on either track; International-track engineers are not required to be versed in U.S. codes, but must build only in the track(s) they selected in the Welcome Poll.

If this applies to you, you will instead be placed into an already-built world to [Task Write](#).

Please also notify your Domain Lead of this restriction upon onboarding.

Critical Tips Before You Begin

Tip 1. Build Your World in Stages

Set up a formal structure that tracks every document you create. Start with foundational operational docs, then let the world grow as tasks are written.

Tip 2. The World Spec as the Source of Truth

Use the World Spec as your **central registry**. Your World Spec is the critical document for grounding both you and the AI systems on what the world should contain. Everything in the world, including entities, documents, thresholds, known issues, domain data, and contributor rules, is registered there. Keep your spec as the single source of truth throughout the build.

Your World Spec contains dedicated tabs for the World & Scenario (a snapshot of the world), Canonical Values (numbers that must align across the entire world), Tasks (task concepts for your future world), and Artifacts (all file registry and app data).

Tip 3. Don't Expect Perfection

Synth generation of world content will inevitably have errors. The synthetically generated initial world is just that: initial. It is a starting point to kick off the process.

You need to iterate, clean up, and take full ownership over the final delivery of the world's content.

👉 Link to template: [Julius World Spec Template](#).

World Building Expectations



Project Julius | World Building

Instruction Guide

Expectations for World Building

Before you start, here is what to expect from a world build: how much time it takes, and the quality bar every world has to clear.

Time Expectations

A first time full world build should take 15 hours for file system only or 23 hours for app-enabled worlds end to end.

Phase	Estimated time
Onboarding and Spec creation & revision	7 hours (includes 3 hours onboarding allowance)
World creation / Revision - File System	8 hours
World creation / Revision - App Data	8 hours
<u>Total</u> File System only worlds App-enabled worlds	15 hours 23 hours

Quality Expectations

Hours are only half the bar. The other half is quality!

Build to the Highest Quality

Every world is built to the highest standard so it holds up on three dimensions: depth, realism, and taskability. A build that is fast but shallow, unrealistic, or hard to write tasks against does not meet the bar.

Reminder: the end goal here is to build a world with a corpus of files that can support 5 tasks that the model FAILS to complete.

Expectations for Reviewers

Thank you for becoming a reviewer for world building in Julius! Here are the expectations for your crucial role in this process.

Time Expectations

Keeping reviewer AHT to these expectations is key to project success. Quick reviews help unblock writers, which reduces the time to build a world. They also mean that your experience and domain expertise gets transferred to the most experts, which makes the entire world-building team stronger.

Phase	Estimated time
Spec Review	15 min
World Reviews	30 min

Quality Expectations

Time is only half the bar. The other half is quality!

Accept Only the Worlds that are TASKABLE

Your review helps to keep every world grounded on our three core dimensions: depth, realism, and taskability. A build that is fast but shallow, unrealistic, or hard to write tasks against does not meet the bar. We are relying on your domain expertise to augment AutoQC, so that you can highlight specific deficiencies of a spec in your domain!

Reminder: the end goal here is to build a world with a corpus of files that can support 5 tasks that the model FAILS to complete.

Communication Expectations

As a key part of the team, communication is key to our success!

✓ RLS: Keep all comments here!

Please put all comments and feedback in RLS. This keeps all information about a world build in one location. That way, the team can quickly understand what issues AutoQC missed. Also, another reviewer/EPM can assist with reviews if you are unavailable, as they have full context on the task.

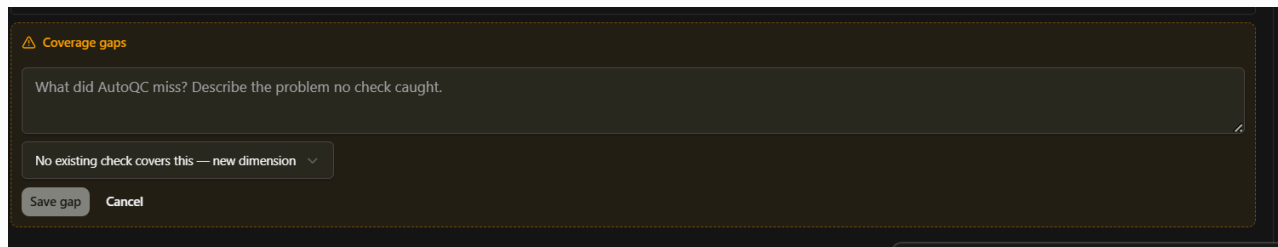
✓ RLS: Claim every review!

Please make sure to claim every review in RLS and send back if revisions are needed, or approve if the builder is good to go to the next stage.

These transitions, and the time spent in them, help us understand and improve the world-building process.

✓ RLS: Note any "Coverage Gaps" in AutoQC

If you find mistakes that AutoQC missed, please annotate them in the orange "Coverage Gaps" section under AutoQC feedback.



The screenshot shows a dark-themed form titled "Coverage gaps" with a yellow warning icon. It contains a text input field with the placeholder "What did AutoQC miss? Describe the problem no check caught.", a dropdown menu currently showing "No existing check covers this — new dimension", and two buttons at the bottom: "Save gap" and "Cancel".

World Building Process Overview

Studio Workflow

RLS (Studio) Workflow

How to technically claim a world and progress with building a world.

1. GETTING STARTED: Claim an Available World

1. Go to your domain link
[Knowledge Work \(HR and S&M\)](#)
[Financial Services \(Accounting and FP&A\)](#)
[Law](#)
[Engineering \(CivE, EE, and MechE\)](#)
[Health Care](#)
[POC](#)
2. **NOTE:** You might have to "Accept" the world assignment first.
 - a. Ignore all numbered worlds and immediately look for your domains world of worlds, such as "Financial Services - world of worlds"

- Use the filter in the top left to filter to “Available for Claim” and “stale/ready for pickup”

Tasks
View and manage tasks across worlds in your campaign · 12 tasks in Engineering - World of Worlds

Engineering - World of Worlds

Tasks Query

Filter Columns Group Actions

Page 1 of 1 12 tasks · Showing 1-12 of 12 100 rows

Task Name	Task Status	Created By	Updated By	Needs Attention From	Criteria	Latest Score	Latest Model	OT Grade	AutoQC	Error Categories
☐ Mechanical Engineering 05	Available for Claim	Spencer Sheff	Spencer Sheff	Spencer Sheff	0	—	—	—	•	—
☐ Mechanical Engineering 01	Available for Claim	Spencer Sheff	Spencer Sheff	—	0	—	—	—	•	—
☐ Mechanical Engineering 02	Available for Claim	Spencer Sheff	Spencer Sheff	—	0	—	—	—	•	—
☐ Civil Engineering 02	Available for Claim	Spencer Sheff	Spencer Sheff	—	0	—	—	—	•	—
☐ Electrical Engineering 01	Available for Claim	Spencer Sheff	Spencer Sheff	—	0	—	—	—	•	—
☐ Electrical Engineering 04	Available for Claim	Spencer Sheff	Spencer Sheff	—	0	—	—	—	•	—
☒ Mechanical Engineering 04	Available for Claim	Spencer Sheff	Spencer Sheff	Spencer Sheff	0	—	—	—	•	—
☐ Electrical Engineering 03	Available for Claim	Spencer Sheff	Spencer Sheff	—	0	—	—	—	•	—
☐ Mechanical Engineering 03	Available for Claim	Spencer Sheff	Spencer Sheff	Spencer Sheff	0	—	—	—	•	—
☐ Civil Engineering 03	Available for Claim	Spencer Sheff	Spencer Sheff	—	0	—	—	—	•	—
☐ Electrical Engineering 02	Available for Claim	Spencer Sheff	Spencer Sheff	—	0	—	—	—	•	—
☐ Civil Engineering 01	Available for Claim	Spencer Sheff	Spencer Sheff	—	0	—	—	—	•	—

- Click on a task to open it (any one of your subdomain, the number is arbitrary)
- Check the **METADATA**. Your world **MUST** match:
 - Industry
 - Entity Size
 - Apps

World Information

World Metadata

World Name* Corporate Law 36 Domain* Corporate Law Industry Technology / SaaS

Entity Size Mid-market World Build Type Select...

World Building Personnel

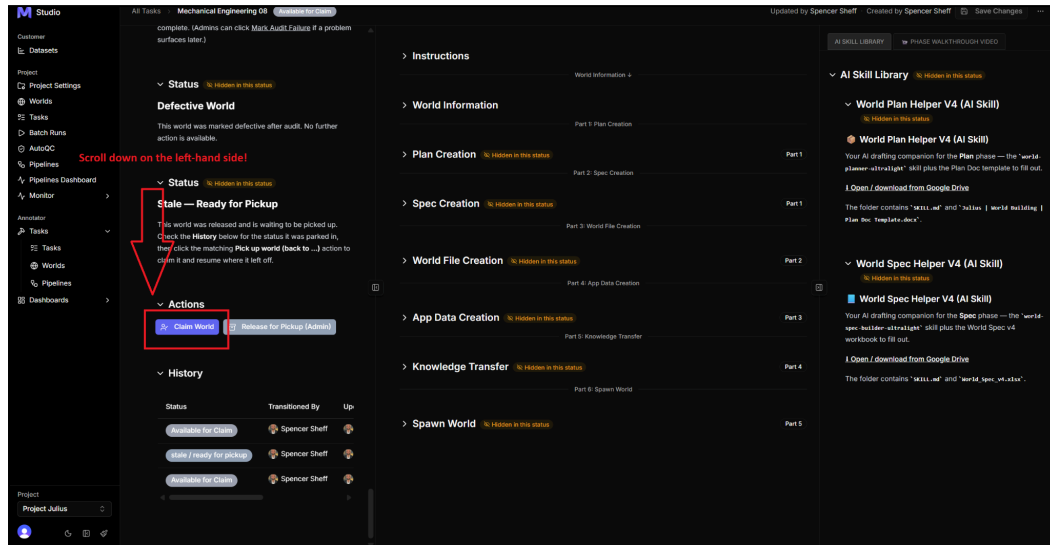
World Builder (First Last)* Kiara Graves

Building Team Members* 1

App Data Target Apps

1 selected Docuseal

- Click “Claim World”



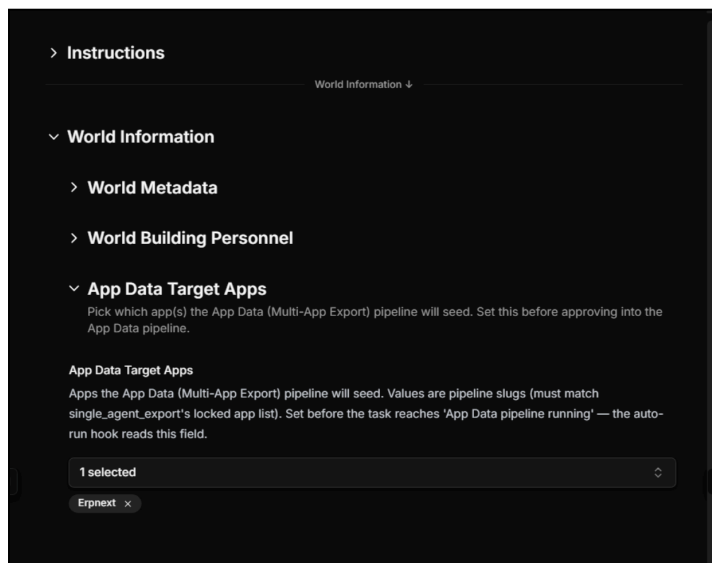
⚠️ Don't see a world to claim?!

Please post in [#julius-wb-questions](#). Please state your domain and when you received a contract offer.

APPS

Check the Metadata!: 99% of worlds include apps: ERPNext, Docuseal, Wiki.js, etc.

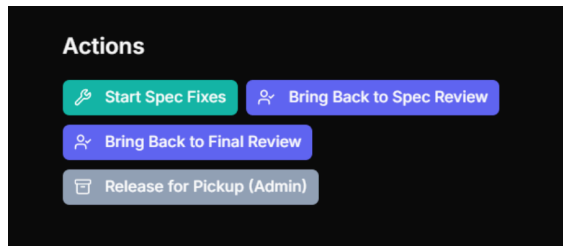
Worlds WILL have APPs listed under "World Information."



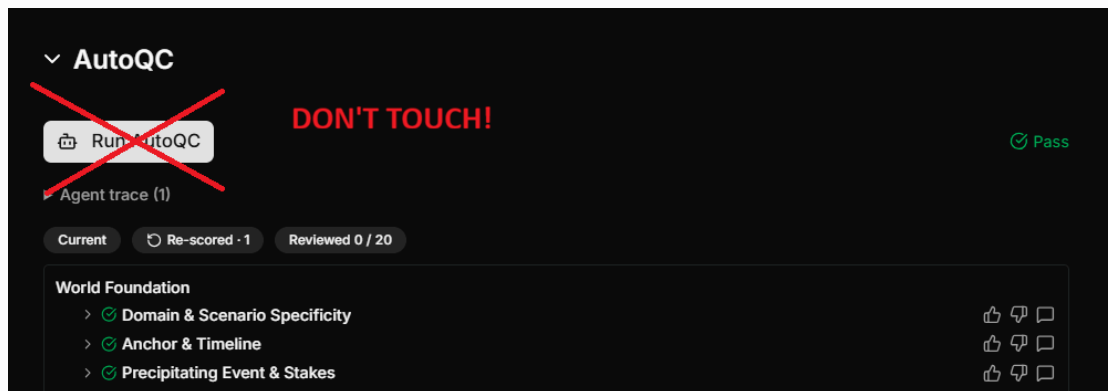
ACTION BUTTONS AND YOU

From the start to end of world building, please use the colorful “Actions” buttons in the left hand pane to complete any actions in your world.

- Autoqc is done? → “Start spec autoqc review”
- Fixes to do? → “Start spec fixes”
- Passed Autoqc and ready for a human reviewer? → “Submit spec for review”

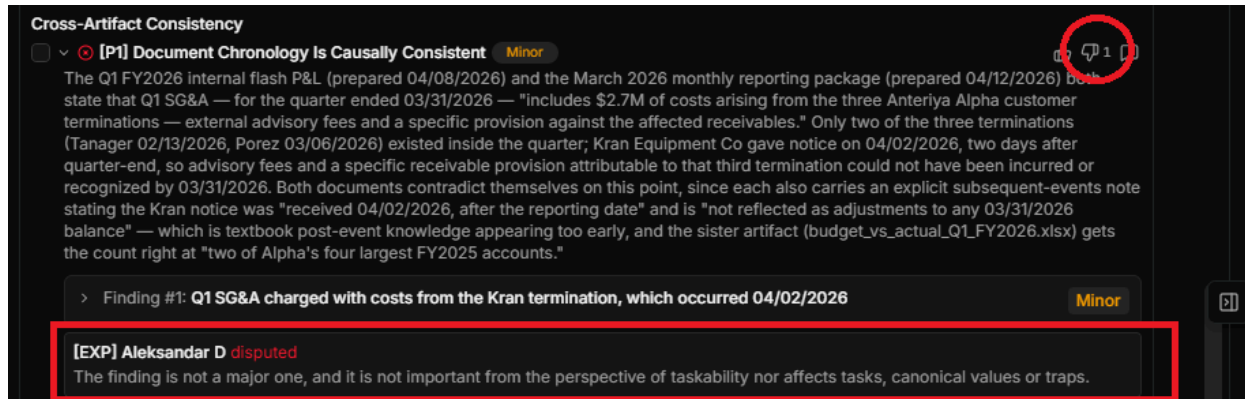


⚠️ ALWAYS run AutoQC using the “Action” buttons.
Never use the plain “Run AutoQC” about the AutoQC results. That button is a lie.



AUTOQC THUMBS DOWN

AutoQC is NOT always correct. Please compare its results to your knowledge of your world's tasks and traps. If an issue would not affect your ability to create tasks, and is **Minor**, you can click the “thumbs down” and comment briefly.

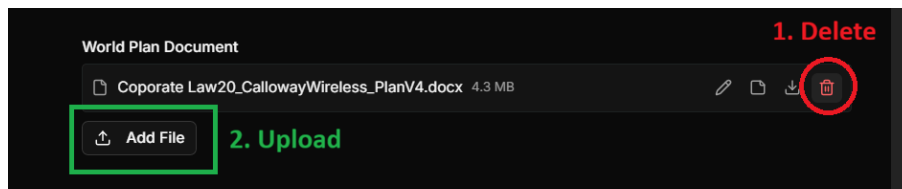


If the finding is **Major**, but you disagree, please post in your world build thread in #julius-world-threads and tag your world building domain lead for confirmation.

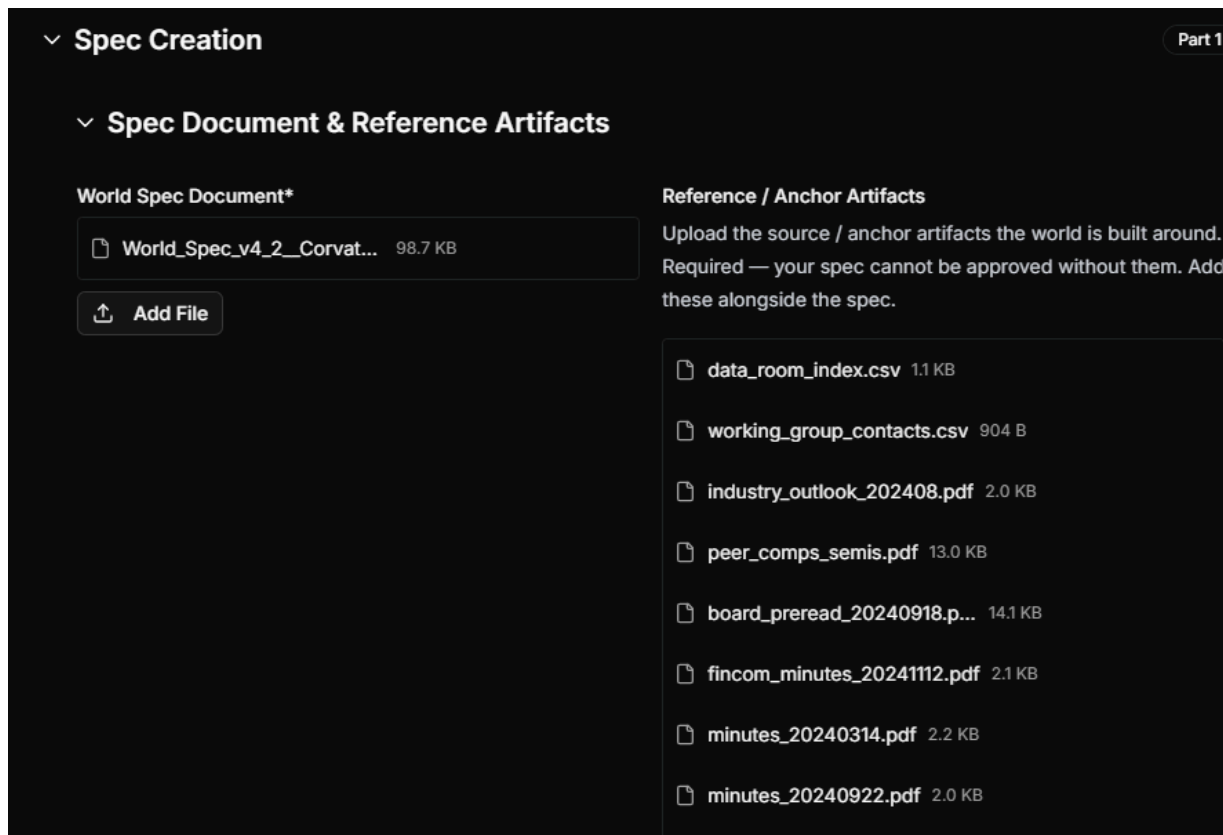
2. FIRST STEPS: UPLOAD A SPEC and ANCHOR FILES

When you have finished using the [claude skill to create an .xlxs spec](#), you should upload in the appropriate section.

Make sure to **delete the old version** before uploading the latest.



Upload both the SPEC and the ANCHOR FILES (see [here for guidance on Anchor Files](#))



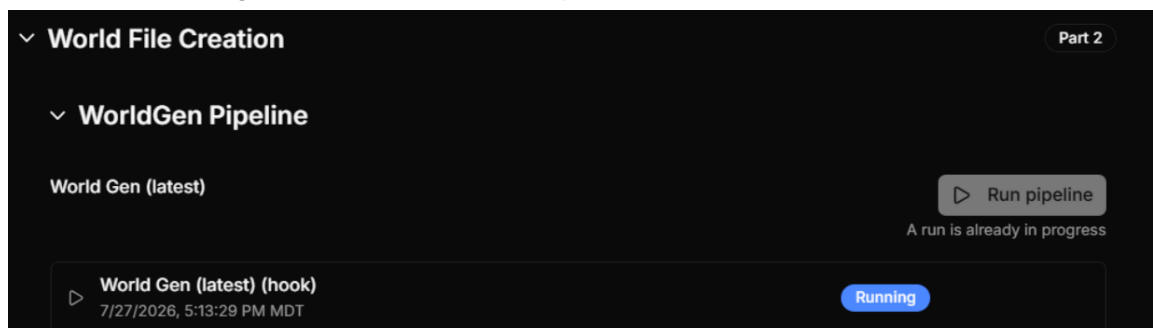
3. THE ACTUAL FILES: FILE GENERATION

When your spec is approved by a reviewer, the system will automatically start creating your files.

Your files are SYNTHETICALLY generated, there is no need for you to manually create files.

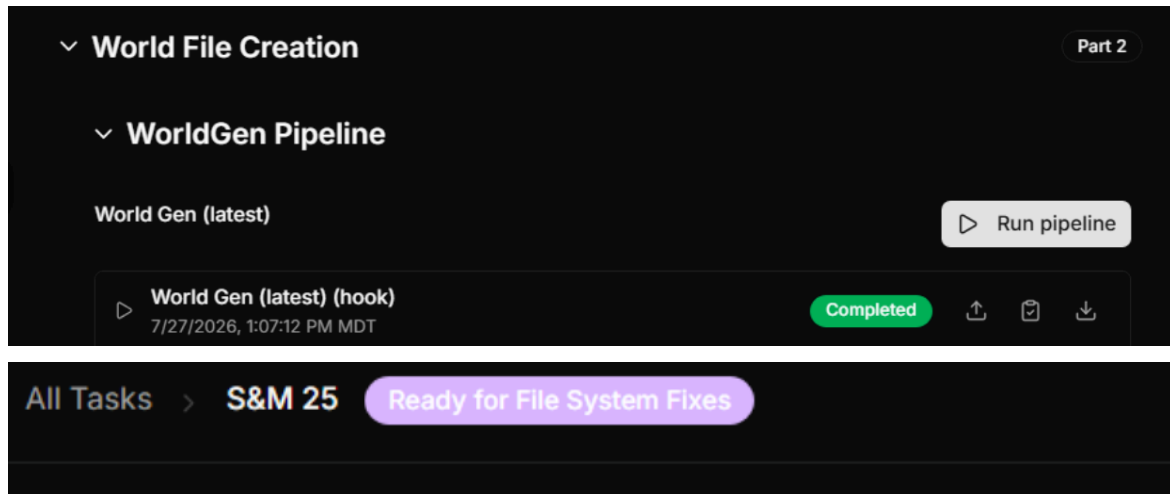
This process can take from 4–10 hours.

The blue “running” means it is not done yet.



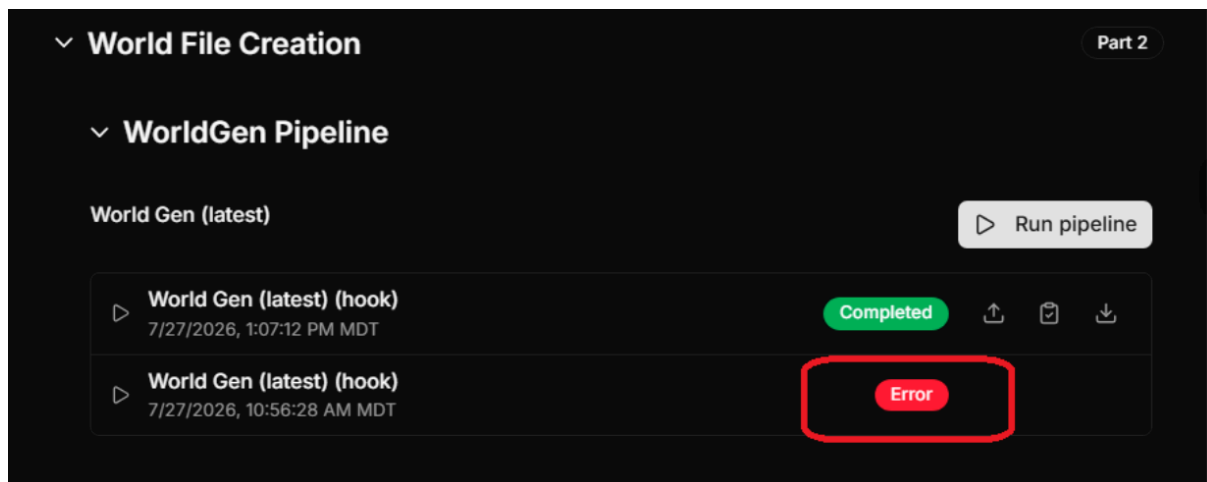
Pipeline Complete

When your files are generated and the first AutoQC has run, your world will change to “Ready for File System Fixes”



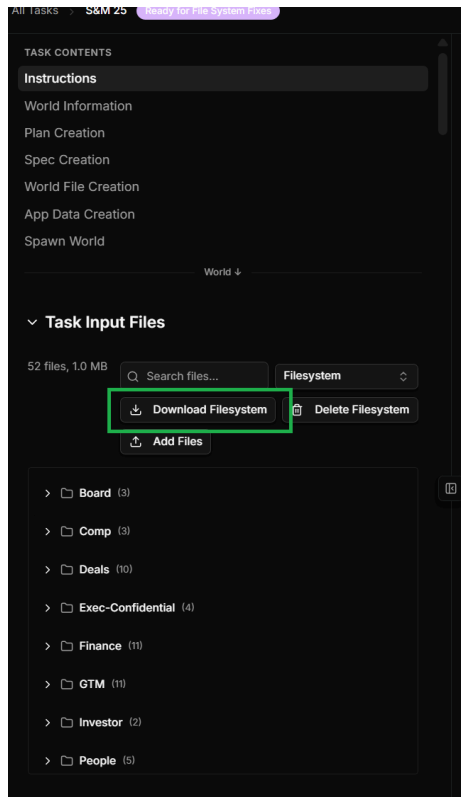
Pipeline Error

If there is a big red "Error" button rather than "Completed," please tag the world building team in [#julius-wb-question](#). Please do not rerun right away as it might be a system wide technical bug and we don't want to waste tokens on something broken!



4. FILE REVISION

To start revising your files, download the filesystem [here](#).

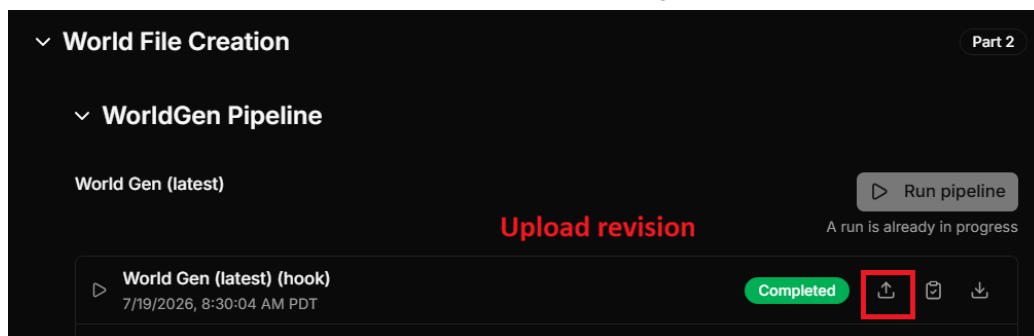


Upload Revised Files

Once you have made corrections to files, upload **ALL** of the files in the file system here (Not just the ones you fixed, the entire complete file system).

It will compare the new version to the old and tell you what files changed.

Write a brief comment on what you had to change.



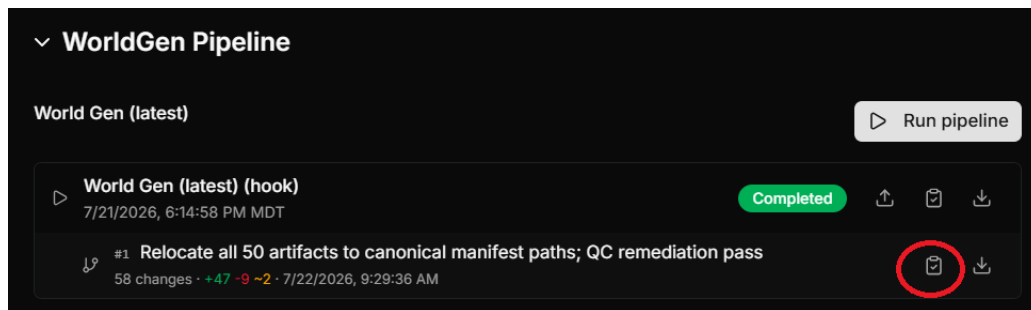
Apply Revised Files to Task

BEFORE you run AutoQC again, click the clipboard to "Apply files to task."

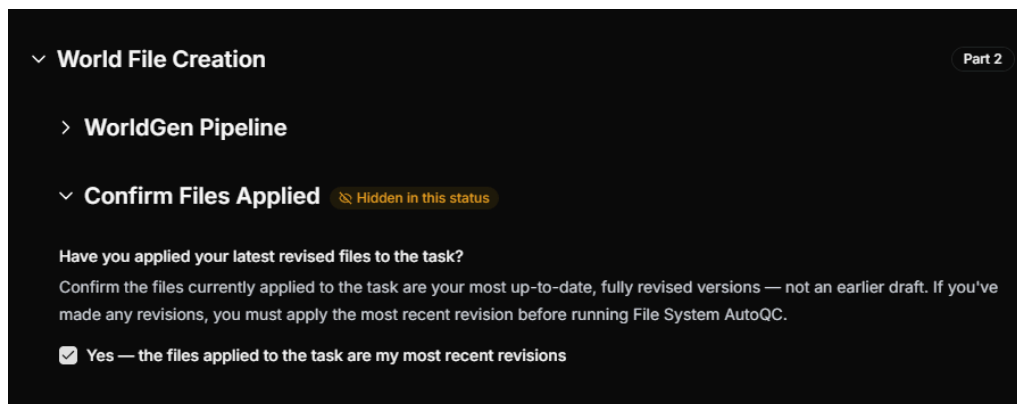
⚠️WAIT at least one minute between "Apply files to task" and clicking the colorful left hand button to "Rerun filesystem AutoQC." Look at ONE file in the left hand pane that was revised,

to make sure that the left hand side has the LATEST files. This prevents you from running AutoQC on the old files!

This updates the files in the filesystem in the left-hand pane, and makes sure AutoQC runs on the revised files!



Finally, make sure you have confirmed that you applied the latest files by clicking this button



FILE REVISION TIPS

Use Claude Code for the revisions. Do not attempt to manually revise your file corpus.

Batching: Try to fix as many issues before reuploading to AutoQC. AutoQC is very slow at this stage, so we want to make sure our revisions are substantial before rerunning it

Understand AutoQC: One round of AutoQC is NOT the whole story.

AUTOQC IS ITERATIVE: One round of AutoQC does not find all the mistakes.

A good goal is for around 6-10 rounds of AutoQC, which is our average for efficient revisions

AUTOQC IS NOT ALWAYS COMPREHENSIVE: Check for the same mistake autoqc flagged in other files

GUIDE THE REVISIONS: Claude makes mistakes. Make sure YOU understand the mistakes that have to be fixed, and guide Claude efficiently

One by One fixes: Once the skill gives you a list of core issues, go through each BY ITSELF. DO NOT TRY TO ONE SHOT the fixes, telling Claude to "Fix all" or any similar prompting. That will put you in Autoqc hell.

CLAUDE CODE SETUP

Claude code is the best for file revisions. The prompt is still natural language.

1. Setup claude code: It needs Git (like gdrive but for code) and possibly some other applications depending on your own setup
2. You download and extract your files to a folder on your desktop. Add your spec as well, that way, Claude knows what things you have labeled as a trap!
3. You click on the folder above Claude code and select the folder with your files
4. Select Sonnet 5 (these are mechanical fixes, not thought heavy, so no need for the higher models!)
5. Select "Auto approve" so it can revise freely
6. Prompt it to revise the files to fix all mechanical issues in AutoQC
7. NOTE: If you are on round 5+ of AutoQC and have the tokens, you can start a new chat, and move to Opus to help remediate the rest of the fixes.

Please see this loom video that outlines how to setup claude code:

<https://www.loom.com/share/197f0eebd0ec40d48b3cc97e20f3d90a>

And this is the skill:

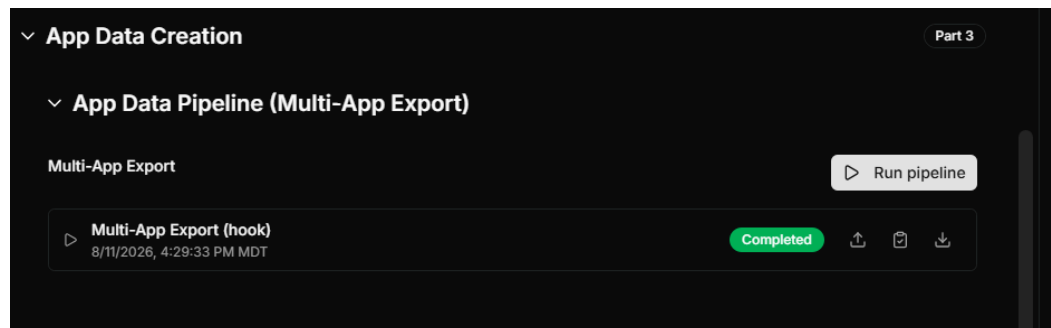
https://drive.google.com/drive/folders/1qcx8sAePCcPbYfclZ43lKeDtsFVQNS8C?usp=drive_link

5. APP DATA REVISION

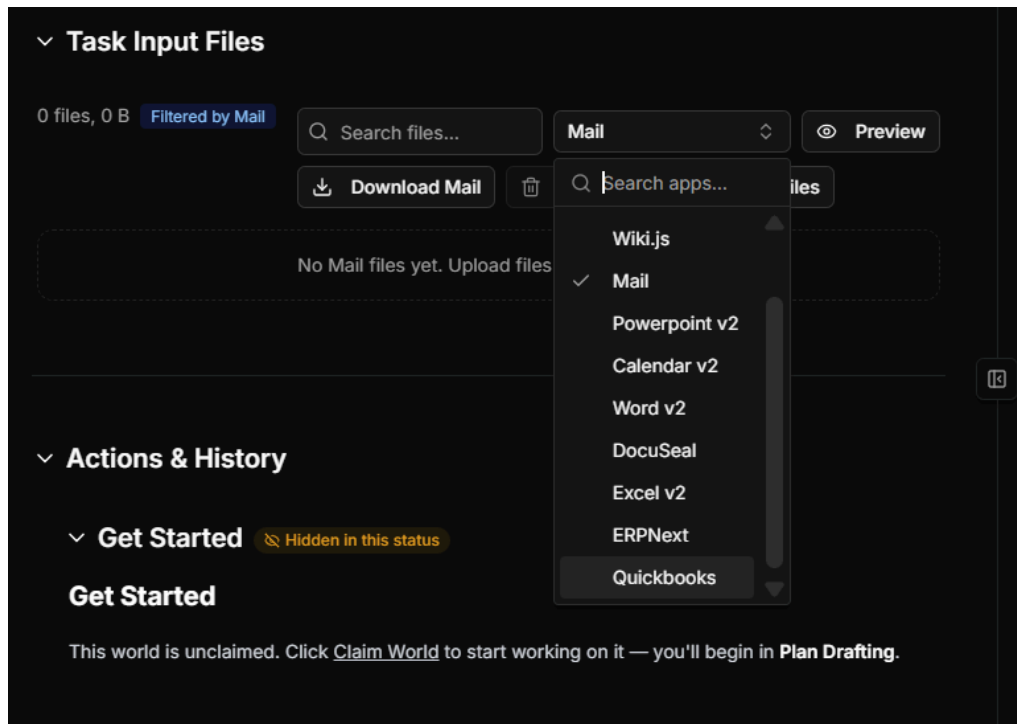
Your app files are generated AFTER your files have passed Autoqc.

This process is shorter and takes about 1 to 3 hours.

Just like the file system, there is a completed pipeline, and you can upload revisions and run AutoQC.



Your app files will be listed under the appropriate name in the left-hand pane



▲Upload BOTH your filesystem files AND app data files! ▲

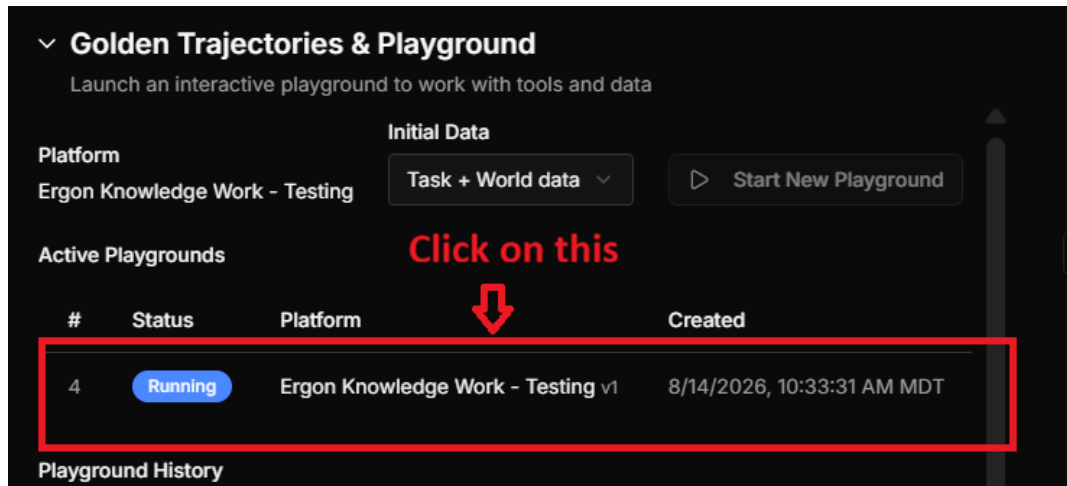
✓Once you have passed AppDataQC, you then move to app data validation and playground!

6. PLAYGROUND

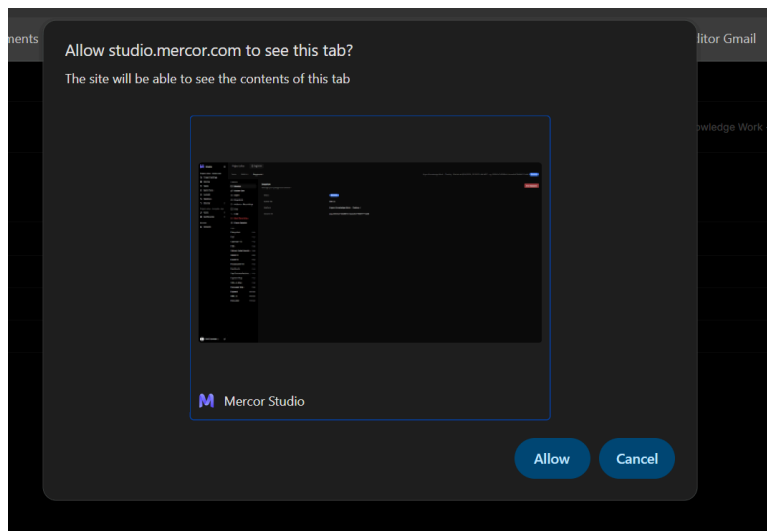
Once your app data is consistent with your files, and has passed AutoQC, we do a final manual check to check that the files ACTUALLY load in our pretend RLS environment!

First, run the "Schema Validator to make sure your app files are formatted correctly. If some have issues, use the heal button to fix them. When it is complete, apply that revision to task and run schema validator again.

▲"Running" in this case does not mean it is creating anything. It means the playground is ready for you to check if your apps load correctly in RLS▲



Then, click "Allow"



You will now see a recreation of your world, a virtual desktop of all you have created!

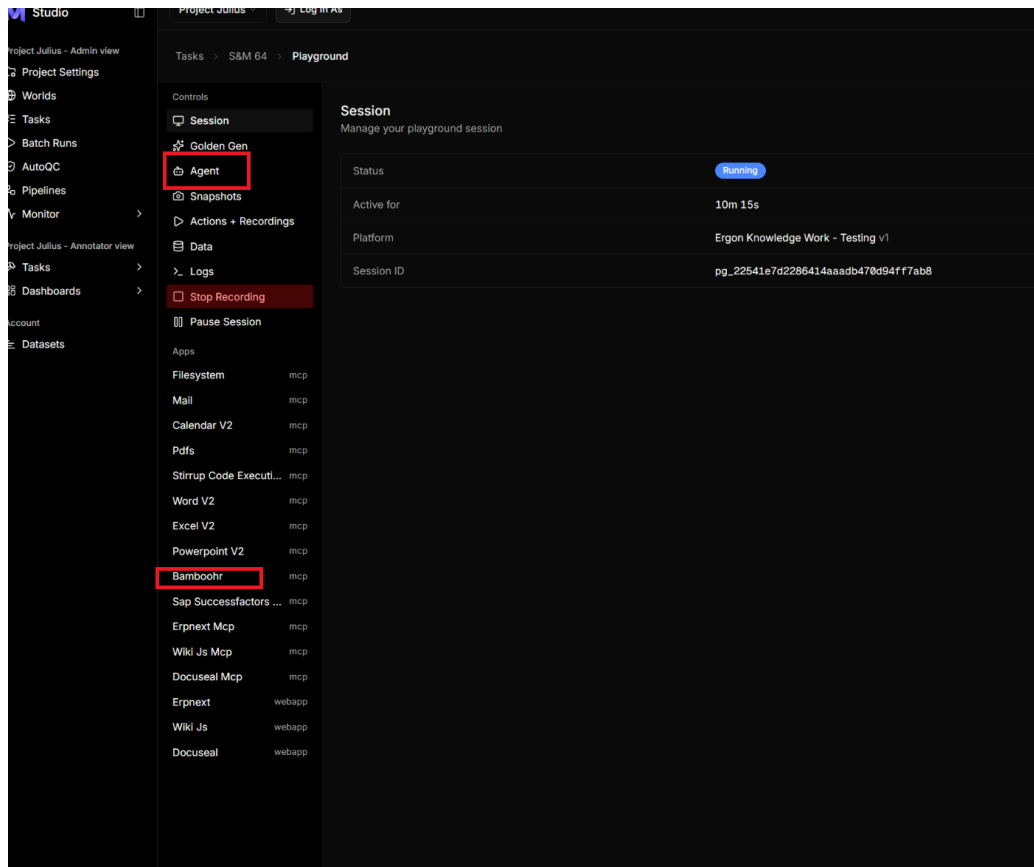
Use the AGENT to:

- Check that the files are all loaded correctly. Use this prompt as a guide for what you are checking:

None

Validate <APP> in this RL Studio Playground using only its MCP tools. First identify the available read, list, search, and get-detail tools. Then list the records and report the count, open <NAMED RECORD> in full, verify <FIELD OR RELATIONSHIP> against the world spec, and report the exact values you found. Do not create, edit, or delete anything.

Use the APP (such as BambooHR in this example) to open the application and check that entries you require for tasking have loaded correctly. Please see the [Playground Validation](#) instructions for login to apps that require this



Examples of what to check and how are in [THIS tab](#)

IF APPS WON'T LOAD/HAVE ISSUES in Playground:

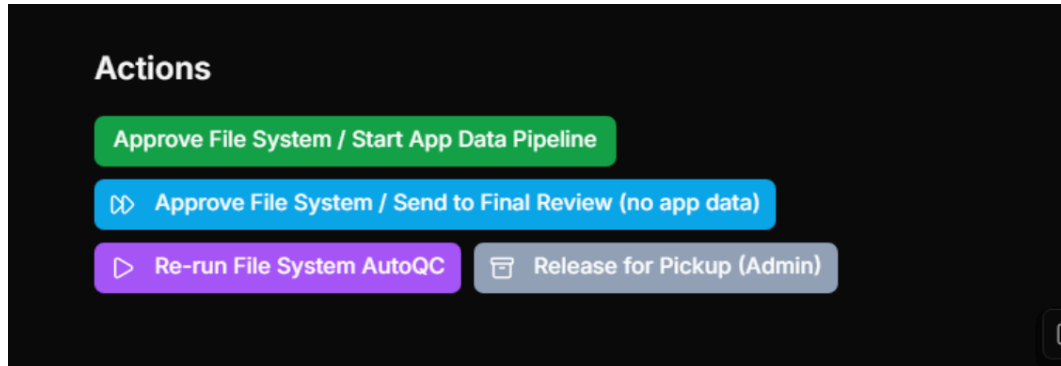
Please fill out this form 🙌 [World App Issues Report Form](#)

7. FINAL REVIEW

When you score a Pass on AutoQC, there are TWO different workflows.

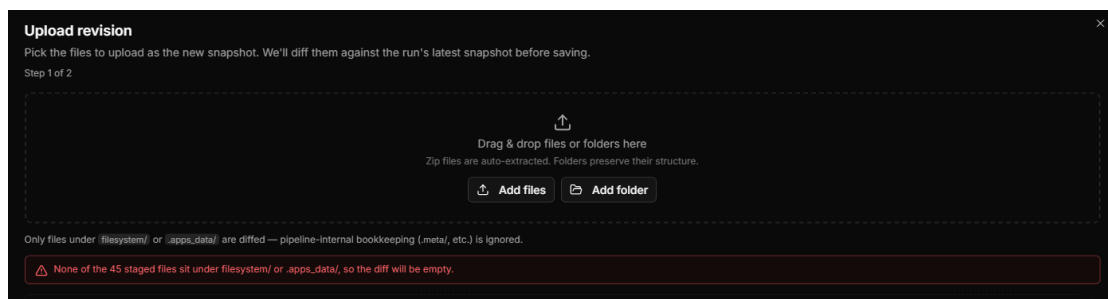
Has Apps (99% of worlds): Select the green option

NO Apps: Select the blue option



TROUBLESHOOTING COMMON PROBLEMS

Error: I upload a file revision, and it says "no filesystem or .apps_data"



SOLUTION

Your uploads must:

- Be in a folder named "filesystem" for files
- Be in a folder named ".apps_data" for app data

DO NOT change the folder names and hierarchy from when you download them!

▲ MAC USERS

Mac hides the ".apps_data" automatically.

To upload the correctly named file from a MAC:

When viewing the downloaded files in Finder, press:

⌘ + Shift + . (period)

This toggles hidden files and should reveal the .apps_data folder so you can keep the correct name and upload it successfully.

World Building Tips

World Building Tips

As you embark on building a world, begin by thinking about the issues you have encountered in your work life and in your use of artificial intelligence to assist with your everyday workflow.

WORK IN REVERSE

The greatest asset to your world is pre-thought fails. Come up with **10 ideas** that would require multiple files in your work life to get a response. These should be difficult questions that you, as a domain expert, could answer, but **areas where you know a model struggles to reason.**

For example, a frontier LLM may understand finance but not the inner workings of a financial planning and analysis business dealing with government regulation. Think about every time you used an LLM and it failed. Then create ideas for your world files based on that knowledge.

LOST IN THE MIDDLE

You may not have complete control over a file that comes out of a synthetic generation pipeline. You do have control over **editing the file**. Take advantage of this. Bury a critical fact somewhere in the middle of a longer document. This forces a model to read the entire document, and it may not be able to retrieve the information.

CONFLICTING AUTHORITY

Many businesses have policy documents that may conflict, not because of the timestamp, but because of the business rules or hierarchy. Perhaps the CEO issued a policy that supersedes one within your corpus. Can the model understand?

TRAPS

Traps represent **“human messiness.”** In our everyday lives, this occurs when our interns or junior associates make a small mistake. It can occur within email conversations when drafts are exchanged.

For example, you edit a document and label it “Final.” Your boss reads the document and sends it back a day later, renamed “Final Final.” Which does the model use?

Failure pattern	One-shot mechanism
WORK IN REVERSE	When creating the world, begin with thoughts of what fails the models (in particular, Gemini 3.8 Flash) in your everyday life. Include 30–50 reasoning ideas across multiple files/apps that would likely work. Build these ideas first, and others around them.
LOST IN THE MIDDLE	Place critical information needed for a reasoning task in the middle of a dense file. This will test retrieval.
CONTEXT ROT/TOKEN DEGRADATION	Models suffer from “token degradation.” Force the model to read long documents, and it will lose retention.
CONFLICTING AUTHORITY	Two grounded sources disagree; precedence is decided by a rule, not by recency.
TRAPS	The traps represent “human messiness,” and when compounded, the model struggles to understand. Build more traps.

Common Failure Modes by Domain

Common Failure Modes by Domain

The Main Idea

Designing "Hard" Tasks for AI

A task isn't "hard" just because it has many steps; modern models handle long, multi-step chains easily. A truly challenging task forces the model to make a difficult judgment call.

To create these "judgment calls," follow these design principles:

- **Hide the Rules:** Place key rules or policies in your "World" files rather than the prompt. If the prompt gives the answer away, the model doesn't need to do any real work.
- **Create Ambiguity, Then Resolve It:** Include multiple paths to an answer, but ensure only one is correct according to professional standards. A smart generalist might choose the wrong path, but a domain expert will know the right one.

Tactics to Test Model Reasoning:

- **Use Conflicting Sources:** Provide two sources but define one as the authority. The model must discern which file to trust.
- **Bury Amendments:** Hide an update in an obscure document that overrides an earlier rule.

- **Require Integration:** Force the model to pull information from three or more files to find the answer. NOTE: Tasks cannot call on 10 or more files!
- **Withhold Proprietary Rules:** Company-specific logic (e.g., specific thresholds or tables) should never be in the prompt; it must exist only within the data provided.
- **Provide Facts, Not Answers:** For professional standards (e.g., accounting or legal rules), provide the raw facts and framework. Do not write the outcome; let the model apply the standard to reach the correct conclusion.

The Gold Standard:

- **Build to one clear answer:** The goal is for all experts to agree on the outcome.
- **Avoid ambiguity:** If two experts could reasonably disagree, the task is likely too ambiguous, not "hard."
- **Reconcile your data:** Unless you have explicitly designed a "trap" (an error for the model to catch), all numbers in your "World" must match perfectly.

Applies to every world

- The world must support a task that uses **Applications plus at least two World Files.**
- Every key number should have **one clear answer:** no ranges, no loose ends.
- A number that appears in more than one place must **match everywhere.**
- The right answer should show up in **outputs that can be checked.**

FP&A

Put this in your world:

- A decision the prompt leaves open but a domain expert would settle only one way.

- A figure that differs between the app (ERPNext) and a file, so the agent has to pick the right source.
- A chain of forecasts where one wrong number throws off everything downstream.
- Don't include a summary file that states the answer—provide only the raw data so the agent has to compute it.

Don't do this: don't add extra steps to make a task "harder." Extra steps make it easier for the model.

Example of a legitimate model failure: the agent excluded \$18.72M of payroll and used gross capex (\$1.8M) instead of net (\$1.566M), overstating book cash by about \$18.9M.

Accounting

Put this in your world:

- A transaction near a period boundary, where a world policy decides which period it belongs to.
- A rule in the world (not the prompt) that fixes which account something goes in.
- Two sources where one is the authority; for example, a Credit Agreement that defines a term.
- Place a tempting-but-wrong figure that's legitimate in context (e.g., a contractual rate the governing definition overrides). You can surround the key file with extra real, in-character documents so the agent must pick the right source among legitimate files

Don't do this: don't leave a key number unfixed (no single right answer); don't let values fail to reconcile; and don't build a task where a partial catch still gets the right answer.

Example of a legitimate model failure: the model used the FY2024 effective tax rate instead of the \$406K of actual cash taxes that the Credit Agreement defined.

What not to build: One task had no answer because working capital in/out was never fixed. That's an ambiguous task, not a failure.

Sales & Marketing

Put this in your world:

- A rule made of stacked parts that must be combined (e.g., discounts that stack).
- An amendment buried in a less-obvious source that overrides an earlier rule.
- A term defined in an SOP that controls what the agent should do.

Don't do this: don't make a task answerable from one input file. It must use the app plus at least two world files. Avoid app values that can't be checked against the rest of the world, fake "errors" the model can just handle, and bait that claims "X isn't possible."

Example of a legitimate model failure: the pricing master says a 3% pre-pay discount stacks on the 18% base, so the right cap is 21%. The model approved 18%.

Law

Put this in your world:

- A correct, current authority in the world, with an old or default one as bait, so a wrong figure can be verified.
- Sign-off thresholds, so that when an amount crosses a threshold, a different person has to sign.
- Linked figures, so one correction has to carry through the whole memo.
- Include a document that needs only a small, specific fix. The model tends to over-rewrite and add.

Don't do this: don't build forks on judgment calls two lawyers could reasonably disagree on; instead, pin the answer to an authority.

Example of a legitimate model failure: the model used per-scan BIPA damages instead of distinct applicants (Cothron/Clay), giving the wrong reserve.

HR

Put this in your world:

- A policy the prompt won't repeat; a cutoff, an accrual cap, or proration.
- Answers that require combining several files.

Build note: keep the canonical HR data in the ERPNext HR modules, not BambooHR.

Don't do this: don't build forks on judgment with no rule behind them (e.g., "who to lay off").

Example of a legitimate model failure: the model counted ineligible staff toward a referral incentive, which caused errors downstream.

Engineering

Put this in your world:

Build note:

Don't do this:

Example of a legitimate model failure:



Step-by-Step Deep Dive



Project Julius | World Building

Instruction Guide

Claim your world in your domain's World of Worlds

[Law - World of Worlds](#)

[Financial Services - World of Worlds](#)

[Knowledge Work - World of Worlds](#)

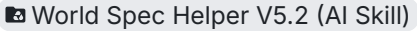
[Engineering - World of Worlds](#)

1 Spec

Writing a Spec That Passes (Start Here)

You write the spec. The pipeline (the automated system that builds your world) reads it and generates the whole world from it. It only has what you wrote and cannot ask you questions, so the spec must be complete and exact. Anything you leave vague, it has to guess.

We have a Claude skill to use to help you fill out the spec, pass Autoqc, and one-shot your spec submission. To use it:

1. Download the skill and template from here:

2. Put both in a new Claude chat. Cowork with Opus 5 works great.
3. Tell it you need to fill out the spec template attached, and it will walk you through all the steps!

Work the [template's](#) tabs from left to right.

Use THIS tool to help you include "Apps" in your 10 task ideas!

<https://ergon-world-app-guide-mercor1.vercel.app/>

 **WATCH THIS!**

**Spec Creation
Walkthrough**

~11 mins

 [Spec Creation Walkthrough](#)

**Skill Training - How to Build a
World Spec**

~5 mins

 [Skill Training → World Spec](#)

A few words you will see throughout

Word	Plain meaning
The pipeline	The automated system that builds the world from your spec. Not a person.
Worldbuilder	You. You write the spec and own the world through to the finished files.
Canonical value	The one official version of a number, name, or date. When two places disagree, this is the one they must match.
Threshold	Another word for a canonical value: a value that must read the same everywhere it appears.
Grounding fact	A fact that is simply true and must hold everywhere.
Trap	An inconsistency you put in the world on purpose, registered with a Key.
Reconcile	The numbers agree and add up.
AutoQC	The automated reviewer that scores your spec before a person reviews it.

The Golden Rule

App data is the source of truth. Pick each real value once, register it as a Fact with a Key, and make every file and app row carry that same value. If two things differ on purpose, label it a Trap.

In plain steps: pick the number once → write it on the **Canonical Values** tab → make every file and app row match it → if two things differ on purpose, that is a Trap and you must label it both as a trap in **Canonical Values** and under Trap Content in the ④ **Artifacts** tab. An unlabeled difference reads as a mistake.

A worked example: meet Dana

Dana is a CFO building an Accounting world about a year-end financial review. Watch how Dana fills the World Spec tabs in order.

World & Scenario. Dana ticks **ERPNext** (the accounting records live there), which auto-fills the App registry. Then the company snapshot:

- **Bad:** "A mid-size company that sells products."
- **Good:** "Northwind Components Inc., a U.S. industrial parts distributor, 220 employees, three U.S. warehouses, \$43.2M FY2024 revenue.
External entities: top 8 customers, two banks, one audit firm."

Dana writes the revenue as exactly **\$43.2M** here because that is the value the fact will carry everywhere. Then Dana names every person the scenario needs (CFO, controller, auditor) in the Key Parties and Roles cell so no one shows up later in a file without being introduced here.

Canonical Values. Dana registers the headline number and a planned trap:

Value Name	Type	Value
FY2024 revenue (audited)	Grounding fact	\$43.2M (sum of ERPNext Sales Invoices)
Q4 FY2024 revenue	Grounding fact	\$12.1M (part of the \$43.2M annual total)
Q3 deck vs invoices	Trap	Q3 deck shows \$11.0M but invoices sum to \$10.4M; intentional; app data wins

The quarter figure (Q4 FY2024 revenue) must fit inside the annual figure (FY2024 revenue (audited)): the four quarters total the year. The trap names both numbers, says it is intentional, and says which side wins.

Tasks. Dana sketches the tasks the world should support, and what registered files, data, and app data are required for the tasks. She lays out 10 distinct and unique tasks to prove the world can carry 5 hard tasks when complete.

Artifacts. Dana writes each file row so the pipeline can build it without any questions. This does NOT include app data. The model row:

Column	Dana's entry
Type	File
Name	FY2024 Board Pre-Read
Format/App	PPTX
Location	/Finance/Board/FY2024_preread.pptx
Label	Key
Prepared by/Source	Finance team (CFO office)
Description / What to generate	8-slide board deck summarizing FY2024; slide 2 cites a \$43.8M flash revenue estimate.
Relates to (IDs)	A07, A12
Trap Content	Stale \$43.8M flash figure never refreshed after the audited close — an authentic copy-forward.
Tasks	T1
Reference Doc (title only)	Board Deck - Coca Cola

The rules that matter most

Specific files

Every file row must let the pipeline build the file without guessing. The reviewer reads your "Description / What to generate" and other columns; if they do not name the actual sections, figures, or other data, the file cannot be built reliably.

Fill in this sentence for each file:

This is a [file type] written by [author].
It contains [the specific sections, slides, metrics, or figures],
and its numbers tie to [which Facts].

- **Bad:** a 12-slide board deck whose purpose reads only "Prior-period board pack for comparatives."
- **Good:** "12-slide year-end board deck; one slide per quarter; charts reconcile to FY2024 revenue (audited); opens with the headline number."

Consistent values (Thresholds)

Every value that appears more than once must read identically everywhere, and every calculated value must equal its inputs. Two ways this breaks:

- **The same number, written two ways.** Bad: prepaid insurance is \$90,000 on one tab and \$45,000 in the canonical fact. Good: register it once (say \$45,000) and match it everywhere via its Key.
- **A calculated number that does not equal its inputs.** Bad: a row states a bundle is "\$936,120 (36 x \$26,040)" but $36 \times \$26,040 = \$937,440$. Good: fix the math, or, if the mismatch is intentional, register it as a Trap and say which source wins.

Before you submit, for every calculated number confirm the inputs produce it, and confirm any deliberate difference is a labeled Trap.

Defendable traps

A good trap has an in-world path the expert can follow to the right answer.

Allowed: a newer or higher-version document wins over an older one; a more senior person's word wins over a junior one; an official document wins over an email; a clearly stated, findable rule settles the conflict; volume noise (many records and files).

Not allowed: anything that makes the task unsolvable; a matter of taste with no in-world reasoning path; a needed value hidden where no one would find it. For every trap, trace it end to end from question to answer using only artifacts in the world. If you cannot, add the artifact that resolves it or drop the trap.

Domain authenticity and depth

- Keep the world's center of gravity in **U.S. jurisdiction** (U.S. GAAP for Accounting, U.S. federal and state law for Law, U.S. regulations for HR and Finance). Use non-U.S. content only where it is genuinely relevant.
- Aim for~ **50 file artifacts plus app data**, enough to support 5 hard tasks. The world is **fixed once finished**; nothing can be added later, so build deep now.

Keep it clean

No author notes, no placeholders, no "TBD" anywhere in the world content. The content should read like what a real U.S. professional in the field would actually produce.

Use the AI drafting partner

The guide gives you an AI assistant as a drafting partner. Attach the template, let it interview you tab by tab, and let your expertise lead: push back on anything generic or wrong before you accept it. Before you submit, ask it to:

- List every value that appears in more than one place and confirm they match.
- Read every file row and flag any row the pipeline could not build from the Context and Other details alone.
- List every company and person named as a party and flag any that might match a real one.
- Trace each trap
- from question to answer using only the artifacts you registered.

Fix what it surfaces, then read its work yourself. It can be wrong; you make the final call.

Reference / Anchor Files

Give the pipeline a real-world example of each primary artifact you register. This should be between 10–15. These are genuine documents from the internet that play the same role as one of your artifacts, such as a pitch deck, an employee roster, or a contract. The synthetic file generation pipeline reads these as inputs and copies their look and feel, so it isn't guessing at what a real version of that artifact should look like. For each reference file, do two things:

1. **Name it in the spec.** Paste the title of the file you downloaded next to the artifact it corresponds to, so the pipeline knows which artifact this example anchors.

L
Reference Doc (title only)
<i>ds · Constraint. 'Relates to' lists artifact IDs this on</i>
Board Deck - Coca Cola
Audited_Financials_Final
-
[Paste title of document here. Note that the actual upload of the document should be in RL Studio under the reference/anchor field.]

2. **Provide the file itself.** Upload the actual document you downloaded alongside your spec, so the pipeline has the real example to read as an input.

Reference / Anchor Artifacts

Upload the source / anchor artifacts the world is built around (optional). Add these alongside the spec.

 Upload File

Not every file-system artifact needs an anchor file—but for your most important, primary artifacts, providing one is strongly encouraged; it is the surest way to keep the world realistic. **Anchor files are for the file system only, and are never required for app data.**

How approval works, and what happens next

When your spec is ready, upload it under the “Spec Creation” tab where it says “World Spec Document Upload.” Then select **Run Spec AutoQC** under “Actions” on your World of Worlds task. It runs, moves to **Spec AutoQC Review**, and you make revisions until you pass AutoQC, or have labeled all AutoQC elements you disagree with. Then, you **Submit Spec for Review**. A reviewer then either sends it back (**Needs Spec Fixes**) or approves it: **Approve Spec / Start File System Pipeline**, which kicks off the build. From there you shepherd the world through these stages on the task:

1. **File System:** the pipeline generates the documents → you fix them → **Run File System AutoQC** until it passes → **Approve File System / Start App Data Pipeline**.
2. **App Data:** the pipeline generates the CSV tables from your fixed files → you fix them → **Run App Data AutoQC** until it passes. Then you communicate to the world building team on Slack that the world needs app data schema and playground check. When they are done → **Approve Verification / Start Knowledge Transfer**.

3. **Knowledge Transfer:** document the finished world so a task writer can pick it up → **Submit for SPL Sign-off.**
4. **SPL Sign-off:** a domain lead does the final review and the world goes live. (They can send it back to any stage to fix.)

Each later stage has its own checklist on the **Checklists** tab. Remember: the file data and app data are **hand-corrected after they are generated**, not regenerated.

Spec pre-submit checklist

Tick every box before you Run Spec AutoQC. This is the detailed self-check; the Checklists tab holds the official Spec checklist.

Golden rule and facts

- ☐ Every value used in more than one place is on the Canonical Values tab.
- ☐ Every file and app row that uses a fact has that fact listed in the Description column.
- ☐ Every calculated number equals its own inputs.
- ☐ Any quarter or part figures add up to their annual or total figure.
- ☐ Every on-purpose difference is labeled Type = "Trap," naming both values and which side wins.
- ☐ Every trap is solvable: I traced a path from question to answer using only in-world artifacts.

Files

- ☐ Every File row could be built by the pipeline with zero questions asked.
- ☐ "Description / What to generate" names actual sections, metrics, and figures.
- ☐ Every File row's Description matches the facts the file really cites.

- ☐ File types, sizes, and lengths are believable.
- ☐ The world has ~50 artifacts total, enough for ~5 hard tasks.

Apps

- ☐ Only the apps actually used are ticked on the World Context tab.
- ☐ Each app row's "What to generate" has real volumes, specifics, and constraints.
- ☐ App values match the grounding facts exactly.
- ☐ Every app row that carries a trap has it listed in Trap Content.
- ☐ Make sure that only apps included in the world are referenced

Real vs fictional / synthetic data

- ☐ The core entity is fictional and its data is fully synthetic; no real company or person is the core entity or a central party.
- ☐ Any real company appears only as supplementary material and uses its real, publicly available data (for example, 10-Ks, filings, or public statements); no synthetic data is attached to a real company.
- ☐ Every fictional company and person name has been web-checked and does not match a real one.

People, dates, and cleanliness

- ☐ Every person the scenario needs is listed on the World Context tab.
- ☐ No dates or time periods contradict each other.
- ☐ No author notes, placeholders, or "TBD" anywhere in the content.
- ☐ Content reads like what a real U.S. professional in this field would produce.
- ☐ Every tab is completed; no grey REF tab or blue computed cell was edited.
- ☐ No apps were referenced or included for a filesystem-only world

- ☐ For a world with >10 employees, a roster has been created.

Final pass with the AI drafting partner

- ☐ I asked it to confirm that the cross-artifact values match.
- ☐ I asked it to flag vague file rows and fixed them.
- ☐ I asked it to flag any real company used as the core entity, or real-company data that was synthesized rather than sourced, plus mismatched Facts, and fixed them.
- ☐ I asked it to trace each trap to its answer and fixed any it could not solve.

2 World Creation

Validating Your World: Files and App Data (Start Here)

👉 **WATCH THIS!**

World Creation Walkthrough

~11 mins

 [World Creation Walkthrough](#)

The big idea: the AI reviewer holds your Canonical Values tab as the answer key

The pipeline (the automated system that builds your world) has finished. It handed you a stack of documents and a set of data tables. Your job is to check what it made and fix what it got wrong, so the world passes QC.

One idea runs through this whole guide. When the pipeline finishes, an automated checker (we call it "the AI reviewer") opens your World Spec Document and reads the "**(3) Facts & Traps**" tab as the answer key. For every row it records four things:

- The **Key** (FT001, FT002, and so on)
- The **Type** (Grounding fact, or Trap)
- The **Value** (the exact fact or number the world must carry)
- Where it should **Appear**

That tab is your **Facts and Traps ledger**. The reviewer checks the world the pipeline built against it. Everything else in this guide makes the world match the ledger.

Here is how the reviewer judges each row:

- **Grounding fact** (Type = Grounding fact): the world **MUST** carry this exact Value, the same everywhere it appears. If two files disagree, or any value contradicts the registered Value, that is a REAL DEFECT and the world FAILS. The report names the FT### and the Value it expected.
- **Trap** (Type = Trap): the inconsistency is on purpose. The reviewer leaves it alone **ONLY** if it maps to an FT### Trap row you wrote down. "It might be intentional" is not enough. If you did not register it as a Trap with a Key, the reviewer treats the mismatch as a defect.

This grounding is additive. All the consistency checks below still run. Each one is now judged against your ledger.

What the AI reviewer hands you back

When the AI reviewer finishes, it gives you a **fix-list**. This is its report. Read it top to bottom. Each line names the file, the problem it found, and (when the problem maps to a fact you registered) the FT### Key and the Value it expected. A line looks like this:

```
[FAIL] FT002 Numerical consistency file: q3_financials.xlsx  
Expected annual revenue $140M (registered grounding fact), found $135M. Fix  
the file to match.
```

Not every line carries an FT### Key. Some flags are general (for example, "this file looks empty" or "this date is in the future"). For those, the line still names the file and the problem, and you check it against your ledger yourself. You will see exactly where this fix-list appears in the next section.

The Golden Rule

App data is authoritative over files unless the fact is a Trap.

The app data is the source of truth. Every file must reconcile to (agree with) the app data. When a file and the app data disagree, fix the file, not the app data, unless you registered that exact mismatch as a Trap.

This rule has a direction, and the direction matters. When a file and the app data fight, you almost always change the FILE. You leave the app data alone. The one exception is a registered Trap. You will see this play out in the worked example below (see "When a file and the app data disagree").

Your two jobs

You are the worldbuilder. You have two jobs at this stage.

1. **Make every grounding fact match.** Make every grounding fact appear with its registered Value, the same everywhere, across both the files and the app data. Fix anything the pipeline got wrong, let drift, or made up.
2. **Make sure every intended mismatch is registered as a Trap.** A Trap is an inconsistency you put in the world on purpose. The reviewer ignores it only if it maps to an FT### Trap row you wrote down. A clever mismatch that is not in the ledger does not read as clever. It reads as a defect. If you find an intended trap you never logged, go back to the spec's Facts and Traps tab and add a row with all four fields: the Key (FT###), Type = Trap, the Value, and where it appears.

A Trap row only works if it is specific. The reviewer has to MATCH the exact mismatch in the world to the exact Trap row you wrote. The Trap's Value and its

"where it appears" must be specific enough to bind to one real mismatch. A vague Trap row does not bind, and the mismatch still fails.

- **Bad Trap row:** Value = "some numbers may differ." (Too vague. The reviewer cannot tell which mismatch this covers, so it does not suppress anything.)
- **Good Trap row:** Value = "Q3 deck shows \$11.0M while Q3 invoices sum to \$10.4M." Where it appears = "Q3 board deck vs Q3 invoice set." (Specific. The reviewer can bind the exact in-world mismatch to it and leave it alone.)

Same review, two file sets, in order

You run the same review twice, on two sets of files.

- **Stage 1: the FILE SYSTEM.** These are the documents the pipeline wrote: Word files, spreadsheets, decks, PDFs.
- **Stage 2: the APP DATA.** These are the CSV tables the pipeline builds FROM your finished files. (A CSV is a simple table saved as plain text, one row per line.)

Both stages are grounded in the same Facts and Traps ledger.

Because the app data is built from the files, **fix the files first**. Once the app data has been generated, you do NOT regenerate it when you patch a file. You hand-correct the app data directly so it stays consistent. (The pipeline can be re-run as a separate, deliberate action, but that is not part of normal fixing.)

A worked example: one worldbuilder, one ledger, side by side

Picture a worldbuilder named Dana. The pipeline has finished a media-licensing world. Dana opens the generated world on one side of the screen and the spec's "**Canonical Values**" tab on the other. Dana works down the ledger, one

row at a time, and treats it as the answer key, exactly the way the AI reviewer will.

Grounding fact FT004: the world's "now" is May 18, 2026. (The "now" anchor is the single date the world is frozen at. Nothing in the world can be dated after it.)

Dana opens the export files. Two of them carry a comment, "Rebuilt 2026-05-26." That date is 8 days after the world's "now." A file cannot have been rebuilt on a date that has not happened yet. One of those files even carries two stamps that fight each other: "Export Date: 2026-05-18" and "Rebuilt 2026-05-26." Dana checks the ledger. There is no Trap row for a future rebuild date. So this is a real defect. Dana fixes both files so the stamps read May 18, 2026, and removes the contradiction.

Grounding fact FT002: annual revenue is \$140M.

Dana looks for the revenue figure everywhere it appears: the financial statement, the model, and the CSV. A clean world keeps a figure like \$140M identical in all three. Dana confirms they match. (If even one file had said \$120M or \$160M, and no Trap row covered it, this would be a numerical-consistency defect, which is a blocking failure.)

When a file and the app data disagree (the Golden Rule in action).

Dana finds something trickier. A contract file says annual revenue is \$140M, but the accounting app's data table (the source the app loads from) says \$135M. The two disagree. Dana checks the ledger: FT002 is registered as a grounding fact with Value \$140M, and there is no Trap row for this difference.

Here the Golden Rule decides the fix. App data is authoritative. So Dana does NOT edit the app data table to say \$140M. Dana fixes the FILE so it reconciles to (agrees with) the app data. Then Dana checks the registered Value: the ledger says \$140M, but the authoritative app data says \$135M. That means the ledger

or the app data is itself wrong, so Dana resolves the conflict at the source (correct the registered Value or the app data table) and then sets every file to match. Remember: when a file and the app data fight, you change the file, not the app data, unless the mismatch is a registered Trap.

Grounding fact FT007: the department is named "Information Technology."

The official master list (the canonical list, meaning the single authoritative version everything else should follow) says "IT." Later files say "Information Technology." Another pair reads "Admin & Ops Support" in one place and "Administration & Operations" in another. No rename memo anywhere in the world shows a name changed on purpose. So the same department wears two names. Dana picks the registered Value and makes every file use it.

- **Bad:** one file says "IT," another says "Information Technology," with nothing explaining the switch.
- **Good:** every file says "Information Technology," because that is the registered Value. OR a dated in-world memo shows the rename and which name is used after it.

Document chronology: an invoice dated before its contract.

Dana checks the order of events. An invoice is dated March 3, but the contract it bills against is dated March 10. The bill comes before the thing it bills for. That is impossible: the effect lands before its cause. This is not a future-date problem. It is an out-of-order problem. Dana checks the ledger for any Trap covering it, finds none, and fixes the dates so the contract comes first and the invoice comes after.

Grounding fact FT011: the royalty rates live in Schedule B.

A master license cites "Schedule B [on file]" for royalty rates in 5 or more documents. Dana searches the world for that file. There is no Schedule A or Schedule B file anywhere in the 121-file world. The documents point at

something that does not exist. Dana either creates the Schedule B file the documents promise, or removes the references. The fix has to leave the world whole.

A streaming-usage CSV that looks too clean.

Dana opens a subscriber CSV. The subscriber counts are exactly 11,000,000 and 7,000,000, flat across all 12 quarters. Revenue is identical every quarter. Stream counts climb by exactly +8,000,000 each quarter. Real businesses do not move in perfectly round, perfectly even steps. The phone numbers are all "555" numbers. This is machine-clean data. Dana edits the numbers so they vary the way real figures do, and replaces the placeholder phone numbers.

A contradiction that should have been a Trap.

Dana finds a Q3 deck showing \$11.0M while the invoices for the same quarter sum to \$10.4M. That is a real mismatch. Dana checks the ledger. There is no Trap row for it. Dana planned this contradiction as a deliberate test, but never wrote it down.

Here is the key lesson. As it stands, the reviewer will flag it as a numerical-consistency defect, because an unregistered mismatch is a defect. Dana has a choice:

- If the contradiction is meant to be a trap, go to the Facts and Traps tab and add a Trap row: FT021, Type = Trap, Value = "Q3 deck shows \$11.0M while Q3 invoices sum to \$10.4M," where it appears = "Q3 board deck vs Q3 invoice set." That Value is specific enough for the reviewer to bind the exact mismatch to it, so now the reviewer leaves it alone.
- If it was an accident, fix the deck or the invoices so they agree.

Dana decides it is a real trap, so Dana adds FT021 to the ledger.

Stage 1: File System Checks

Below are the checks the reviewer runs on the file system, with the ones that matter most first. "Critical" marks a blocking check: it must pass for the world to pass. Every check is judged against your Facts and Traps ledger.

For every flag: read the AI reviewer's fix-list, then open the file yourself and check it against your ledger.

1. Timeline and Dates Are Coherent

Every date must agree with the world's "now" anchor (the single date the world is frozen at) and with the other dates. Nothing can be dated in the future.

- **Bad:** the world's "now" is May 18, 2026, but two export files carry "Rebuilt 2026-05-26," 8 days in the future. One file even has two stamps that fight each other: "Export Date: 2026-05-18" and "Rebuilt 2026-05-26."
- **Good:** every stamp sits on or before May 18, 2026, and one file carries one consistent date.
- **Fix:** find every date, compare it to the "now" anchor in your ledger, move any future date back, and remove the contradicting stamp.

2. Document Chronology Is Causally Consistent

Events must run in cause-then-effect order. The cause comes first. The effect comes after. This is not only about future dates. It is about the order of events relative to each other.

- **Bad:** an invoice dated March 3 bills against a contract dated March 10, so the bill exists before the contract it bills for. Or a payment is recorded before the invoice that asks for it. Or an amendment is dated before the original agreement it changes.
- **Good:** the contract is dated March 10, the invoice that bills against it is dated after that, and the payment is recorded after the invoice.

- **Fix:** walk the chain of events and make sure each cause is dated before its effect. Reorder or re-date any event that lands out of order.

3. All Numerical Values and Constraints Are Consistent Across the World (critical)

Every grounding-fact number must read the same in every file and every app table. One number that disagrees fails the check, unless you registered that difference as a Trap.

- **Bad:** the annual revenue figure reads \$120M in the statement, \$140M in the model, and \$160M in the CSV. Three files, three numbers, no Trap registered.
- **Good:** the registered grounding fact (say FT002, Value = \$140M) reads \$140M identically in the statement, the model, and the CSV.
- **Fix:** pull the number's registered Value from your ledger, set every copy to match. If a difference is meant to be there, register it as a Trap with an FT### Key that is specific enough to bind to the exact mismatch.

4. Consistent Naming

One person, one department, one entity keeps one name everywhere. A name changes only if the world shows the change happening.

- **Bad:** the canonical list (the single authoritative version) says "IT," later files say "Information Technology," and another pair reads "Admin & Ops Support" against "Administration & Operations," with no memo bridging the change.
- **Good:** "IT" everywhere. Or a dated rename memo in the world that shows IT became Information Technology, after which the new name is used.
- **Fix:** pick the one registered name, replace every other version. If a rename is intended, add the in-world document that shows it.

5. Coherence Flag Ratio (critical) and Realism Flag Ratio (critical)

These two are catch-alls. Even when no single problem is fatal, too many small coherence problems, or too many things that just do not look real, will add up and fail the world. Small mismatches are not free. Each one you leave in raises the count toward a fail.

- **Fix:** clear the individual defects below. Fewer small problems means a lower flag ratio.

6. No Synthetic Markers or Placeholder Data

The world must not look machine-made. No data that is too clean. No placeholders. No "TBD." ("Synthetic markers" means data that looks generated, not real.)

- **Bad:** a streaming-usage CSV has subscriber counts of exactly 11,000,000 and 7,000,000, flat across all 12 quarters, identical revenue every quarter, stream counts rising exactly +8,000,000 every quarter, and "555" phone numbers everywhere.
- **Good:** subscriber counts wobble quarter to quarter, revenue moves with them, and phone numbers vary.
- **Fix:** add realistic variation; replace placeholder phone numbers, names, and "TBD" entries with believable values that still match your ledger.

7. In-World vs Out-of-World Separation

World content holds only world content. No author notes. No meta-text. No pointers to anything outside the world.

- **Bad:** a memo carries a leftover note to the author instead of just stating its content.
- **Good:** the memo states its content plainly, with no note attached.
- **Fix:** delete every author note, comment, and outside reference from the world content.

8. No Broken Document References

Every document, schedule, exhibit, or appendix that a file cites must actually exist IN THE WORLD. This check is about references INSIDE the world pointing to other things inside the world.

- **Bad:** a master license cites "Schedule B [on file]" for royalty rates in 5 or more documents, but no Schedule A or Schedule B file exists anywhere in the 121-file world.
- **Good:** Schedule B exists as a file, and the royalty rates in it match what the license quotes.
- **Fix:** list every cited document, confirm each one exists, create the missing file or remove the citation.

9. External Citations Resolve To Real Content (also runs)

This is a different check from No Broken Document References. Number 8 is about references that point INSIDE the world. This one is about citations that point OUTSIDE the world, to real things in the real world (a law, a public standard, a named regulation, a real company filing). When a file cites something external, that thing has to be real and the file has to describe it correctly.

Supplementary material from a real company (a downloaded 10-K, a public filing, a press release) falls under this rule: it must be the genuine document, described and used accurately, never edited to fit your fictional world.

- **Bad:** a contract cites "Section 12 of the Imaginary Data Act of 2024," a law that does not exist; or it cites a real statute but states a rule that statute does not contain.
- **Good:** every external citation names something real and describes it correctly.
- **Fix:** check each outside-world citation; correct any made-up reference, or fix the description so it matches the real source.

10. Personnel Names, Titles and Roles Are Consistent (critical)

One person keeps one name, one title, and one role across the whole world, unless the world shows a promotion or transfer.

- **Fix:** set every person's name, title, and role to the one registered value. Add the in-world record if a change is real.

11. File Volume and Format Diversity (critical)

The world needs enough files, and enough kinds of files. Mix Word documents, spreadsheets, decks, and PDFs.

- **Fix:** add the missing file types and bring the file count up to a realistic range for the domain.

12. File Integrity Defect Rate (critical)

Every file must open, and nothing inside it can be empty or cut off.

- **Fix:** open every flagged file and rebuild any that are empty, truncated (cut off), or unopenable.

A few more checks the reviewer runs (all judged against your ledger)

- **Realistic Document Texture:** a real document has the messiness of a real document (headers, footers, signatures, varied phrasing), not the flat sameness of generated text. Fix: vary phrasing and add the normal furniture of a real document.
- **Anomaly Detection Catch-All:** a sweep for anything odd that the named checks did not catch. Fix: when it flags something, open the file, decide if it is a real defect or a registered Trap, and act accordingly.
- **Traceability and Cohesion:** the files should hang together as one coherent world, where related documents connect and tell a consistent story. Fix: make sure related files reference and agree with each other, so the world reads as one place, not a pile of unrelated documents.

Stage 2: the app data. Same review of files and revisions

The app data is checked against your ledger the same way the files are.

PRESERVE THE FORMAT OF THE CSVs EXACTLY. Keep the exact files and tables, the exact columns in the exact order, and the exact field formats. Any change breaks ingestion. This is the single most important app-data rule.

The fix loop (the same five steps for both stages)

You run the AI reviewer from your World of Worlds task using the named action for each stage: **Run File System AutoQC** (Stage 1) and **Run App Data AutoQC** (Stage 2). The action runs the check and then moves the task to a review step, where the fix-list described above is posted.

The five steps are the same for both stages.

- ☐ **Review.** Read the AI reviewer's fix-list. Then open the files yourself and check each one against your Facts and Traps ledger. Use the template below for each flag.
- ☐ **Remediate.** Remediate means: fix it. Fix every real defect.
- ☐ **Re-upload.** Put the corrected files back as a Zip revision (a single compressed Zip file holding the whole corrected file set; a revision is a new saved version of that set). Leave a short comment on each file you changed.
- ☐ **Re-run.** Run the AutoQC again for this stage (**Run File System AutoQC** in Stage 1, **Run App Data AutoQC** in Stage 2). It moves back to the review step with an updated fix-list.
- ☐ **Apply to Task Files.** The **Apply to Task Files** button is on the same task screen. Clicking it saves your corrected files as the official version the rest of the process uses (the official Task Input Files). You can run it again

later: if you find another defect after applying, fix it and click Apply to Task Files again to replace the official version with the newer one. Click it once the reviewer passes, OR once each false positive is documented in the task's comment thread on the platform (the RLS task comments, where RLS is the platform the task lives on), naming the file and the Trap or source that resolves it.

Step 1 fill-in template (Stage 1, for each flag)

```
Flagged item: _____
Is it a grounding fact in my spec? Name: ___ Value: _____
-> Does the world match that Value everywhere? [ ] yes [ ] no, fix it
Is it an intended trap? [ ] yes -> is it registered with a Canonical Value?
Name____
Is the Trap row specific enough to bind to this exact mismatch? [ ] yes [ ]
no, make it specific
Neither? [ ] -> treat as a real defect and fix it
```

Step 1 fill-in template (Stage 2, for each table)

```
Table: _____ CSV file: _____

Every foreign-key value points to a real row in its target table? [ ] yes [
] no
Field formats correct (e.g. plain date, not datetime)? [ ] yes [ ] no
No auto-filled columns included? [ ] yes [ ] no
Right app and file set identified? [ ] yes [ ] no
Grounding-fact numbers match the ledger Value everywhere? [ ] yes [ ] no
```

False-positive write-up (use when you pass a flag the reviewer raised)

```
File: _____ FT### Trap or source that resolves it: _____
Logged in the task comment thread (RLS)? [ ] yes
```

Closing checklists

World Creation - File System

- ☐ Every grounding fact appears with its registered Value, the same in every file (checked against the Facts and Traps tab).
- ☐ Every intentional mismatch is registered as a Trap, specific enough to bind to the exact in-world mismatch.
- ☐ Dates do not contradict the world's "now" anchor or each other; no future "rebuilt" stamps.
- ☐ Events run in cause-then-effect order (no invoice before its contract, no payment before its invoice).
- ☐ Every grounding-fact number reads the same in every file.
- ☐ Every person, department, and entity keeps one name, unless an in-world rename is shown.
- ☐ No machine-clean or placeholder data (no perfectly round, perfectly even numbers; no "555" phone numbers; no "TBD").
- ☐ No author notes or meta-text inside world content.
- ☐ Every cited document, schedule, exhibit, or appendix actually exists in the world.
- ☐ Every external citation names a real outside-world source and describes it correctly.
- ☐ Every person's name, title, and role is consistent everywhere.
- ☐ Enough files and enough formats (Word, spreadsheets, decks, PDFs).
- ☐ No empty, truncated, or unopenable files.
- ☐ Corrected files re-uploaded as a Zip revision, with a comment per changed file.
- ☐ File System AutoQC re-run and passing (or false positives documented with a thumbs down and comment AND in the task comment thread).
- ☐ Applied to Task Files.

World Creation - App Data and Verification

- ☐ Files were fixed and approved before the app data was generated; after generation, the app data is hand-corrected, never regenerated.
- ☐ Grounding-fact numbers in the CSVs match the ledger Value everywhere.
- ☐ Each CSV's columns remain unedited: none deleted, renamed, reordered, or extra.
- ☐ No auto-filled columns included.
- ☐ Every file agrees with the app data under the Golden Rule (app data wins unless the fact is a Trap; when they disagree, the file is what you fix).
- ☐ Corrected app data re-uploaded as a Zip revision, with a comment per changed table.
- ☐ App Data AutoQC re-run and passing (or false positives documented with a thumbs down and comment AND in the task comment thread).
- ☐ Applied to Task Files.

When AutoQC passes and you have checked the above, proceed to playground verification (next tab).

Playground Validation

Validating a World in the Playground

Ergon is MCP-first. Except for Personal Consumer (POC), apps are meant to be used and verified through MCP tools. A web page is not required. For an MCP app with no user interface, the standard and preferred validation path is the Agent inside an RL Studio Playground.

The one rule: read the world's records back

A green schema check, a running service, or a successful tool call does not prove that the world data loaded. You must read the records from the live app and confirm that they are the world's records.

MCP-first rule

1. Standard Ergon app (all non-POC subdomains): Use the app's MCP tools through the Agent in an RL Studio Playground.
2. Paired MCP + UI app: Use the Agent for MCP readback. You may also use the app's UI in the same Playground when visual confirmation is useful.
3. POC consumer app: POC is the only Ergon subdomain that uses BUA as the primary interaction path. Validate it through the UI in the Playground.

Important: The Agent and the UI are validation surfaces, not seed methods. Seeded data must come through the deployed app's supported CSV loader. Do not create, repair, or replace missing seed records through the Agent, the UI, an API, JSON, SQL, or the filesystem. If the records are wrong or missing, return the world to App Data Fixes.

Choose the correct Playground surface

For every assigned app, first check World Information and the Playground connection:

- 1. MCP tools are present: Open the Agent and validate the app through those tools. This is the required path for non-POC apps, including apps with no screen.
- 2. Both MCP tools and a UI are present: Validate through the Agent. You may also open the UI for a second readback, attachments, rendered documents, dashboards, or other visual details.
- 3. Only a UI or BUA surface is present: Continue only when the world is in POC. Outside POC, stop and report a wrong-interface or platform configuration issue.

An app marked running is alive, not proven populated. A tool that returns success is connected, not proven populated. The output must show the world's actual records.

Login reference for paired UI and POC apps

Use these Playground credentials only when the app appears in World Information. If a credential does not work, do not guess and do not create a new account. Report the app, platform, and failed login to World Building Ops. The current Playground image takes priority over an older login note.

App	Playground surface	Login
ERPNext	Paired MCP + UI	Administrator / admin
Wiki.js	Paired MCP + UI	admin@wikijs.local / Admin123!
Nextcloud	Paired MCP + UI	admin / admin

DocuSeal	Paired MCP + UI	admin@docuseal.localhost / password
Docassemble	Paired MCP + UI	admin@docassemble.local / Admin123! (older image: password)
Fineract	Paired MCP + UI	mifos / password
FASLR	Paired MCP + desktop	No login (noVNC)
OpenProject	Paired MCP + UI	admin / Admin123!
Odoo	Paired MCP + UI	admin / admin
NocoDB	Paired MCP + UI	admin@nocodb.com / password
Chatwoot	Paired MCP + UI	admin@chatwoot.local / Admin123!
JupyterHub	Paired MCP + UI	admin / any password (empty is also accepted)
Metabase	Paired MCP + UI	admin@example.com / Admin123!
Grafana	Paired MCP + UI	admin / Admin123!
Looker	Paired MCP + UI	No login
Gitea	Paired MCP + UI	admin / Admin123!
CKAN	Paired MCP + UI	admin / admin1234
Canvas	Paired MCP + UI	admin@canvas.local / password
OpenEduCat	Paired MCP + UI	admin@openeducat.local / Admin123! (older image: admin / admin)
KiCad	Paired MCP + desktop	No login
FreeCAD	MCP + noVNC UI	No login
BambooHR	MCP login tool	hr_admin / hr_admin_secret
SAP SuccessFactors	MCP login tool	admin / admin_secret
QuickBooks	MCP	No login; pre-authenticated
WooCommerce	POC BUA	admin / Admin123!
Easy!Appointments	POC BUA	admin / Admin123!

OpenTripPlanner	POC BUA	No login
Firefly III	POC BUA	admin@firefly.local / password12345678
Spree	POC BUA	spree@example.com / spree123
Any other paired app	Paired MCP + UI	Do not guess. Report the missing login row to World Building Ops.

Standard MCP validation: use the Agent

For every non-POC app, use this workflow in RL Studio:

1. Spawn the Playground from the world-building task and wait until it reaches running.
2. Open the Agent inside that Playground. Do not leave RL Studio to look for a separate app page.
3. Ask the Agent to identify the app's available read, list, search, and get-detail tools.
4. Use the Agent to list the app's records and report the count.
5. Use the Agent to open at least one named record in full. Check a meaningful field, amount, date, status, attachment, or relationship against the world spec.
6. Repeat for every assigned app. A multi-app world is not validated until each app has its own readback.
7. Log the exact records and values that you checked. Do not write only "looks good."

Suggested Agent prompt

Validate <APP> in this RL Studio Playground using only its MCP tools. First identify the available read, list, search, and get-detail tools. Then list the records and report the count, open <NAMED RECORD> in full, verify <FIELD OR RELATIONSHIP> against the world spec, and report the exact values you found. Do not create, edit, or delete anything.

A successful response is not enough by itself. The response must show the world's names, counts, and facts. Empty results, demo records, and built-in samples do not pass.

Paired MCP + UI apps

A paired app has both MCP tools and a UI. Outside POC, MCP remains the task interaction path. The UI is an additional validation surface; it does not change the Ergon spec to BUA.

Use the Agent first for structured readback. Then use the UI when it gives useful visual proof, such as a rendered PDF, dashboard, attachment, page layout, storefront, or desktop model. Use the login table above.

Do not create missing records in the UI. If the Agent and UI show different data, mark the app NEEDS FIXES and report both results.

POC exception: BUA is allowed only here

Personal Consumer (POC) is the only Ergon subdomain that uses BUA as the primary app interaction path. Open the assigned POC app in the Playground, use the POC login above, and validate the customer-facing surface.

A BUA-only app outside POC is not a normal workaround. Stop and report a platform or task-configuration issue. Do not rewrite the task around the UI.

What every readback must prove

- It is the world's data. Names, companies, people, products, accounts, and documents match the world spec, not the app's samples.
- The count is sane. The live record count is close to the expected volume and is not zero or a stray handful.

- A concrete fact matches. Check an amount, date, status, balance, named person, line item, or other exact value.
- Relationships hold. Open enough detail to confirm links such as order-to-line-item, employee-to-department, invoice-to-customer, document-to-submitter, or task-to-owner.
- Cross-app facts agree. The same company, person, figure, and date must match in every app and in the filesystem documents.

When an app fails

- **EMPTY:** The service and MCP tools work, but the Agent returns no world records, only demo data, or the wrong values. This is an app-data issue. Return it to App Data Fixes.
- **BROKEN:** The Playground fails, the app does not start, or the app registers no MCP tools. This is a platform issue. Escalate to World Building Ops.
- **LOGIN FAILURE:** A paired or POC UI opens, but the documented login fails. Do not create an account. Report the app, platform, image, and credential that failed.
- **WRONG INTERFACE:** A non-POC app exposes only BUA or a UI and has no MCP tools. Stop and report the configuration issue.

If one world has more than one problem, log each problem separately. Fixing a platform issue does not prove that the app data is present.

Reach a verdict

- **Per app: POPULATED** when the Agent or allowed POC UI reads the world's records, the count is sane, a concrete fact matches, and key relationships hold.
- **Per app: EMPTY** when the app is reachable but the world data is absent, partial, demo, or wrong.
- **Per app: BROKEN** when the Playground or app does not start or the expected MCP tools do not register.

- **Overall: READY FOR TASKING** only when every assigned app passes through the correct interaction path, the filesystem is present, cross-app facts agree, and no data was created manually during validation.
- **Overall: NEEDS FIXES** when any app is empty, broken, on the wrong interface, inconsistent with another app, or missing a usable login.

Log the verdict and evidence in the RL Studio task comments. When the world is READY FOR TASKING, select Approve Verification / Start Knowledge Transfer.

[World App Issues Report Form](#)

Playground walkthrough checklist

- ☐ The Playground reached running and every assigned service started.
- ☐ For every non-POC app, I opened the Agent in the Playground and used the app's MCP tools to read records.
- ☐ For every paired app, I used the MCP result and, when useful, the UI with the documented login.
- ☐ For every POC app, I used the BUA/UI path in the Playground and checked the customer-facing surface.
- ☐ I confirmed the world's names, a sane count, one exact fact, and at least one key relationship per app.
- ☐ I confirmed that the same people, companies, dates, and figures agree across apps and documents.
- ☐ I did not create or repair seed data through the Agent, UI, API, JSON, SQL, or filesystem.
- ☐ I logged each app as POPULATED, EMPTY, or BROKEN, with exact evidence and any interface or login issue.
- ☐ If every app passed, I selected Approve Verification / Start Knowledge Transfer.



Apps

Apps

Julius worlds contain realistic applications that the model can “use” when you request so in your task-writing prompt!

Examples of Apps	Real-world equivalents
ERPNext	Salesforce, Microsoft Dynamics 365, Accounting / Journal Entries
Wiki.js	Notion, Asana, Trello
DocuSeal	DocuSign
SAP SuccessFactors - Recruiting	SAP SuccessFactors - Recruiting
BambooHR	BambooHR
QuickBooks	QuickBooks
WooCommerce	WooCommerce
Chatwoot	Helpdesk

Apps Requirements

Julius is centered around multi-app worlds. Apps are required task surfaces, not background decoration.

1. **Always make sure that your world adheres to the app selection requirements that is pre-seeded in your world**

This information is located in “World Information” at the top of the task.

Example ↴

▼ App Data Target Apps

Pick which app(s) the App Data (Multi-App Export) pipeline will seed. Set this before approving into the App Data pipeline.

App Data Target Apps

Apps the App Data (Multi-App Export) pipeline will seed. Values are pipeline slugs (must match single_agent_export's locked app list). Set before the task reaches 'App Data pipeline running' — the auto-run hook reads this field.

3 selected

ErpNext × Wiki Js × Docuseal ×

2. Use apps in every task idea

Single-app world:

Every task idea must require the assigned app.

Two-app world:

Every task idea must require both assigned apps.

Three-or-more-app world:

Every task idea must require at least two relevant assigned apps.

Across the complete idea set:

Every assigned app must be required by at least one task idea.

Meaningful Use:

The agent must read information from an app or perform an action in it. Mentioning, opening, or optionally checking an app does not count.

For each task idea, name the apps and the required action in each.

Quick test: If an app can be removed and the task still works, the app is not meaningfully used.

You can use this tool to help you implement apps into your task ideas:

<https://ergon-world-app-guide-mercor1.vercel.app/>

3. No need to list app files in your world spec

When laying out your world, you only need to incorporate the apps into your task ideas. No need to list the app files. The process is:

- **Spec:** Layout the task ideas with apps that must be used. Layout the files that will be available only
- **File Generation/Revision:** the spec is used to synthetically create around 50 files for you. You revise to make the data consistent for tasking, and to make sure the files appear natural
- **App Data Generation:** the files are used to generate the required app files for your world. You do the same as file revision, revising the files for data consistency according to AutoQC

Checklist

<https://docs.google.com/document/d/1XeGXt0L0H6higW4VCICYVqHv2NF9NUrOZFG04YwE1Ml/edit?tab=t.a82y4ug2i59>

World Builder Checklist

World Builder Checklist

Use this checklist before submitting your world for human review. It turns the standards in this guide — *Depth*, *Realism*, and *Factual Integrity* — into concrete, verifiable steps. If every box below is checked, your world has been built in a way that is compliant with this instruction guide and is ready for review and spawning.

Spec

- ☐ **Standards track declared and marked in the world name** | Every cited code and unit belongs to that track."
- ☐ **Filled out the basic World Information** | my name and the exact apps for this world, aligning my app selection with the precise guidance received from the *Mercor Ops* team.
- ☐ **Copied the World Spec Template** | without altering its tab structure, and completed every required section.
- ☐ **Defined one specific organization** | (industry, size, geography, and operational context) and confirmed the world's center of gravity is *U.S.* jurisdiction (e.g., *U.S. GAAP*, *U.S. federal & state law*, or *U.S. HR* and finance regulations); any non-*U.S.* content appears only where genuinely relevant.
- ☐ **Registered every Threshold** | each numeric value cited so the spec is the single source of truth for the facts that must reconcile across artifacts.
- ☐ **Logged my Facts & Traps and Known Issues** | in the spec, with each trap defensible (sequential, recency, seniority-of-source,

official-doc-over-email, or a clearly-stated, findable embedded rule) and none that make any task unsolvable.

- ☐ **Mapped each Fact and Trap** | to the appropriate artifacts in the spec.
- ☐ **Confirmed enough depth for ~5 tasks** | that failed the model — 50+ file system artifacts planned, plus app data — and kept the spec open as the central registry throughout the build.
- ☐ **Ran Spec AutoQC** | after clicking *Start Spec AutoQC*, via task status transition (not the in-pipeline *Run AutoQC* button). The run passed — or I documented every false positive in the *RLS* comments and explained my reasoning to the reviewer.
- ☐ **Passed human review on the spec** | which automatically triggers the *AI* pipeline build of the world.

World Creation Checklist (File System)

- ☐ **Includes 50 file system artifacts** | within the ~50 target, with enough substantive content to support 5 tasks that fail the model.
- ☐ **Used multiple modalities and format diversity** | *Word* documents, spreadsheets, presentations, and *PDFs*.
- ☐ **Mirrored real organizational folder hierarchies** | (a compliance team files differently than a sales team), so information is deliberately scattered and no single document contains everything an agent needs.
- ☐ **Populated all facts and traps in the file system** | every *Threshold* registered in the spec appears in the files and reads identically everywhere it surfaces.
- ☐ **Built defensible traps and human messiness** | (e.g., Final vs. V2.1, conflicting authority resolved by a business rule rather than recency, an amendment buried in a non-obvious source)
- ☐ **Verified factual integrity world-wide** | every shared name, title, role, number, and date is consistent across all files; there are no broken

document references; the chronology is causally coherent; and there are no contradictions.

- ☐ **Removed synthetic tells** | no placeholder data, no “as mentioned above,” no generic boilerplate, and no synthetic substitutes for real-world facts; voice, tone, and formatting vary by author and document type, and external citations resolve to real content.
- ☐ **Remediated every File System AutoQC defect** | after pulling the findings and downloading the world locally — fixing each issue I found and each one *AutoQC* flagged, reuploaded as a new world revision, and commented on what had to be remediated.
- ☐ **Re-uploaded the file system as a Zip revision** | in the pipeline module, leaving a brief comment on each changed file to help improve the *SyntheGen* pipeline.
- ☐ **Re-ran File System AutoQC** | from the *Task Action Button* after clicking *Start File System AutoQC*. My most recent *AutoQC* run passed perfectly. If it did not pass perfectly, I commented exactly what the false defects were — naming the file, the design intent (e.g., an intentional trap), or the source that resolves it — in the *RLS* comment section.
- ☐ **Applied the passing revision to Task Files** | so the *Task Input Files* — the source of truth for all world artifacts — reflect the current file system.

World Creation Checklist (App Data & Verification)

- ☐ **Generated app data from the file system** | using the apps selected at claim time (e.g., *ERPNext*, *BambooHR*, *SAP SuccessFactors*, *DocuSeal*, [Wiki.js](#))
- ☐ **Confirmed app-plus-files taskability** | Every planned task idea requires the assigned app in a single-app world, both assigned apps in a two-app world, or at least two relevant assigned apps in a three-or-more-app world. Across the full idea set, every assigned app is used at least once. Each task also uses at least two World Files.

- ☐ **Preserved every app's CSV schema exactly** | the exact files/tables present, the exact columns, and the exact field formats — making no additions, removals, renames, or format changes.
- ☐ **Ran the App Data AutoQC review loop** | reviewing the data in *Task Input Files*, downloading it, remediating defects, re-uploading a precise revision, clicking *Start App Data AutoQC*, and re-running until it passed or every false positive is documented and defensible.
- ☐ **Ran the realism check across apps and files** | — company name, key people and roles, headline financials, contract values, and key dates read the same everywhere they appear; flagged every contradiction and any AI tell.

Final Check

- ☐ **Confirmed 5+ judgment-fork tasks that break the model** | each built around a real judgment fork (a rule that lives in the world but is withheld from the prompt, or a profession-correct path a smart generalist would miss) — not merely long step-chains.
- ☐ **Built the world to one answer** | domain experts would all land on the same conclusion, the world's own numbers reconcile, and any deliberate mismatch is a denoted trap; if two experts could reasonably disagree, the world is too ambiguous, and I fixed it.
- ☐ **Pinned every key number to one value** | no ranges and no loose ends, any number appearing in more than one place matches everywhere, and the right answer surfaces in outputs that can be checked.
- ☐ **Passed all AutoQC checks** | (Spec, File System, and App Data). My most recent *AutoQC* run passed perfectly. If it did not pass perfectly, I commented exactly what the false defects were in the *RLS* comments.
- ☐ **Applied the latest passing pipeline to Task Files** | the source of truth
- ☐ **Acknowledged the world is FROZEN after submission** | no revisions, edits, or additions once production task writing begins — and confirmed it

clears the highest bar on *Depth*, *Realism*, and *Taskability* before submitting for human review.

World Reviewer Checklist

Reviewer Workflow

For world building reviews, please:

- Claim review in RLS
- Add comments in the Spec or SPL Sign-off Comment
- Send back to builder or approve for the next step
- Bump in #julius-world-thread, can be very brief

Spec Reviewer Checklist

1. World Foundation

- ☐ **METADATA CHECK** | Pass when the world matches the industry, entity size, and selected apps Flag and send back right away if any of these are not matching. We CANNOT change the industry or apps, these are preseeded according to project needs.
- ☐ **Domain & Scenario Specificity** | Pass when the domain is explicitly set and the scenario is specific enough to host FIVE distinct, model-failing tasks and includes TEN unique task ideas. Flag if the domain is unset or inferred, or the premise is generic and cannot host multiple hard tasks.
- ☐ **Job & Task Field Fit** | Pass when the agent's seat fits the world and every task sits in the world's field. Flag a world written from the wrong chair, or a task from another field (a legal world with an accounting task).
- ☐ **Anchor & Timeline** | Pass when a clear anchor date is set and all events sequence into a coherent, assemblable timeline. Flag if there is no anchor,

documents predate the event they describe, or the timeline cannot be assembled.

- ☐ **Precipitating Event & Stakes** | Pass when there is a specific, dated triggering event with concrete stakes (a named dollar amount, deadline, or consequence). Flag if it is a vague background condition with no date, or stakes with no figure, deadline, or consequence.
- ☐ **Taskable (re-verify)** | Pass when the actual files and canonical values can genuinely produce about 5 model-failing tasks. Flag if the tasks only work on paper and the built files or values won't support them. Please use your experience in task writing to guide if the files and task ideas are setup for successful model failing tasks.

2. Cast

- ☐ **Stakeholder Completeness** | Pass when every populated stakeholder has a name, title, organization, role, and a valid email. Flag if any stakeholder is missing a required field or carries a placeholder (blank, dash, initial-only).
- ☐ **Stakeholder Plausibility** | Pass when roles, names, and email domains are realistic and match the entity. Flag if a role is implausible, or an email/domain has a typo or mismatches the entity.
- ☐ **Stakeholder Coverage** | Pass when every stakeholder and external entity appears as an author, audience, or named party in ≥ 1 file. Flag if a stakeholder is an orphan, or an external entity is used in file descriptions but never declared.
- ☐ **Stakeholder Role Conflicts** | Pass when no person holds two incompatible roles. Flag if a person holds conflicting roles (e.g., mediator and responding officer) not declared as a trap.

3. File Inventory

- ☐ **Adequacy** | Pass when the file count is within the campaign range and the set covers the scenario with real depth. Flag if there are too few or too many files, or thin filler documents pad the count.
- ☐ **Specificity** | Pass when every file row has substantive purpose/content, the key facts it must contain, a real size. Flag if descriptions are one-line or label-only, or key figures are missing.
- ☐ **Realism** | Pass when formats and authorship are realistic (editable docs not forced to .pdf; plausible access), and any file that must be computed against is a live model whose dynamic structure (tabs, drivers, formulas) is specified in the spec before generation. Flag if there is a format/type mismatch, a file the agent's team would not realistically possess, or a computable file left as a flat static extract (e.g., CSV) where the domain needs a live multi-tab model or its dynamic structure is undefined
- ☐ **Cleanliness** | Pass when the registry is well-formed: rows aligned, no label-vs-content mismatch, key-vs-supplementary clear. Flag if there is a label-vs-content mismatch, shifted or misaligned rows, or the key-vs-supplementary distinction is missing.
- ☐ **Trap architecture coherence** | Pass when every trap fact maps to ≥ 1 trap-flagged file and vice-versa, and trap links resolve cleanly. Flag if there is an orphan trap (no carrier file), trap leakage onto a trap=No file, or a broken trap link.
- ☐ **Solution leakage** | Pass when no file hands over the finished answer to a task, the agent has to compute it. Flag if a file's content or description is the completed deliverable, or a summary states the answer.
- ☐ **Anchor/source artifacts** | Pass when the source artifacts the world is built on are uploaded with the spec. Flag if they're missing.
- ☐ **App-use requirement** | Pass when every task idea uses the assigned app in a single-app world, both assigned apps in a two-app world, or at least

two relevant assigned apps in a three-or-more-app world. Across the complete idea set, every assigned app must be used at least once. Flag any app that is unused in task ideas.

- ☐ **Domain breadth / coverage** | Pass when file descriptions span the domain's core functions and divisions, giving taskers multiple distinct surfaces to task across (e.g., FP&A: LRP, rolling forecast, AOP support schedules, opex, capex, headcount/comp, cash, working capital, liquidity, scenario planning). Flag if descriptions cover only a narrow slice of the company, or whole divisions or functions a real org in this domain runs on are absent, so tasks could only be cut from one area.
 - ☐ **Law:** file set must carry multiple substantive legal memos and several sets of prior counterparty/internal redlines (layered tracked-change drafts, negotiation correspondence) — enough volume and depth that 5 complicated redline and drafting tasks can be built, not just one or two.

4. Consistency

- ☐ **Canonical values** | Pass when there is one value per fact, every file/app reference matches it, and arithmetic ties. Flag if there is an undeclared value or number conflict, or arithmetic that does not tie (and the mismatch is not trap-labeled).
- ☐ **Temporal integrity** | Pass when dates sequence correctly and a file carries only facts dated on or before its own creation date. Flag if there is a causal-ordering break: a document predates its facts, or two facts give contradictory dates.
- ☐ **Cross-reference resolution** | Pass when all factual references resolve, external entities are declared, and trap-architecture links resolve. Flag if there is any data inconsistency, typo, or undefined reference or an undeclared entity.

5. Realism & Buildability

- ☐ **Domain authenticity** | Pass when values, ratios, titles, and terminology are both credible to a practitioner and accurate to the real world (wage floors, tax rates, regulatory figures). Flag an economically or logically impossible value (USD to USD FX not equal to 1, a subset larger than its total), or a real-world fact that's simply wrong, for example paying 100 full-time California employees below the state minimum wage.
- ☐ **Real-vs-fictional compliance** | Pass when no real company or person is used as a named scenario party without a logged Domain-Lead exception, and invented names are cleared of collisions. Flag if a real entity is used as a named party, or a fictional name collides with a real company on first search.
- ☐ **PII / compliance hygiene** | Pass when the spec is PII-safe (fictional phone/email ranges, shielded domains) with no compliance red flags. Flag if there is real PII, real-looking contact data, or a compliance red flag.

6. Engineering Hygiene

- ☐ **Spec structural validity** | Pass when dates are typed as dates, there is no placeholder text, registries are row-aligned, and required metadata is present. Flag if there is a type error (a date stored as text), a leftover placeholder (TBD, Need to fill in, etc), a misaligned registry, or missing task-seed metadata.
- ☐ **Complexity budget** | Pass when the apps match the declared apps AND are incorporated into task ideas well, and world complexity is within the target range. Flag if the declared depth/tier is not met (volumes below tier), or complexity is outside the target range.
- ☐ **Meta-language hygiene** | Pass when no trap, decoy, persona, or builder-/solver-addressing vocabulary appears in content-flowing fields. Flag if trap-architecture or solver-addressing language ("(intended trap)", "agent persona") is left in content fields.

7. Other

- ☐ **Other failure type** | Pass when there is no blocking issue beyond the criteria above. Flag any blocking issue not covered above — and describe it in the reviewer comment.

Domain Specific Checklists

Project Julius | World Building

Instruction Guide

Domain Specific

Accounting

Building Worlds That Survive Review

A stage-by-stage guide for world builders and QC reviewers — only the things that actually decide whether a world passes or comes back. Everything here comes from real review cycles across 30+ worlds.

⚠ WHAT MAKES THIS A GREAT WORLD. NO GREY AREAS	
Domain	ACCOUNTING ONLY. Not corporate development. Not FP&A. Not banking.
Jurisdiction	US GAAP ONLY. Nothing international. No cross-border content, no IFRS.
Completeness	FULL LEDGER + EVERY FILE. Every document a real accountant would touch to do this work. No gaps.
Per Domain Lead: this is not a grey area. A world that drifts into corp-dev, FP&A, banking, or non-US GAAP territory fails review, no exceptions.	

The Entire Workflow: SPEC → FILE GENERATION → (APP DATA (Sometimes!)) → FINAL REVIEW

TLDR: EVERY CHECK AT A GLANCE

Just the headlines. More below for the bullets, the why, and the example behind each one.

STAGE 1: SPEC

1. One number, one source. Everywhere else must match it.
2. Traps show the wrong path only, and big enough to matter
3. Recompute every figure. Do not eyeball it.
4. Lock the spec before anything gets built
5. Excel counts as a real input file
6. Search real-world facts. Do not recall them from memory.
7. Check that the task is actually hard, not just correct

STAGE 2: FILE GENERATION (FILESYSTEM)

1. Check every file for answers already given away
2. Open the file and check the fact is actually there
3. Hunt down numbers that do not match across files
4. Fix the spec, then regenerate. Do not hand edit the output.
5. Keep the files matching the spec
6. Make sure a task cannot be solved by accident

STAGE 3: APP DATA

1. Once filesystem passes, stop touching it
2. Every reference has to point to something real, both ways
3. App data has to match the paper files

STAGE 4: FINAL WORLD REVIEW

1. Check every automatic flag by hand before you trust it
2. Try to prove your own answer key wrong
3. Grade what the agent actually did, not what it says it did
4. Check for leaks again after every fix
5. Do not fix one thing and break two others

FINAL GATE: ASK THESE 4 QUESTIONS

1. Could a reviewer find the answer written down anywhere in the world?
 2. Could they argue a second, different answer?
 3. Does any number disagree with itself somewhere else in the world?
 4. Does the world support at least 5 clean tasks?
- Any yes, it comes back. All four no, you have built a good world. Send it.**

STAGE 1: SPEC

The spec is where you write down every number and fact before the files get built. A tool reads the spec and generates the actual files from it. Whatever you leave vague in the spec, the tool fills in on its own, and you do not get to pick what it chooses.

1. Names: web check every one. Never use AI's first idea.

- If a name matches a real operating business, review stops right there. No exceptions.
- Also banned: the stock names AI always defaults to, like *Meridian*, *Acme Corp*, *TechNova Solutions*, *Globex*, *Apex Dynamics*, *John Smith*, *Jane Doe*, and their cousins.
- Use AI to get started, but never keep its first suggestion. Spend one extra minute making the name sound real and specific to your industry.

Why it matters: Stock names make every world look the same, and they are an obvious sign the content was AI generated.

REAL EXAMPLE: “The only action items were changing one name that was too close to a real company... I also changed two other names that related to two real-world locations.”

2. Every task needs one answer that actually holds up

- A task idea (we call it a *seed*) needs a wrong path that looks tempting, and a right path that is the only one the accounting rules actually support.
- Decide now what locks in that right answer, like the method, the rate, or the date you are using.
- If you can argue the task two different ways, it is not ready.

Why it matters: A task a reviewer can argue two ways cannot be graded. It gets sent back.

EXAMPLE: A depreciation task where the spec never states which method the company uses. A reviewer could defend straight-line or double-declining balance equally well, so there is no single right answer to grade against.

3. Stay on the accounting concept, start to finish

- Pick one accounting concept for the world, like recognizing revenue or accounting for a merger.
- Make every task test that concept.
- Do not let tasks drift into HR policy, company strategy, FP&A, or anything else that is not accounting.

Why it matters: Tasks have to align with the domain for our delivery to the client.

REAL EXAMPLE: Reviewing a spec filed under Accounting, Shanmukh found it was actually built around employment law and HR topics like WARN notices, severance, and final pay rules, not accounting judgment calls. He sent it back with a suggestion to rebuild it around a real accounting trigger, like severance cost accounting, instead.

4. Keep your own notes out of the files

- Never write words like “trap,” “distractor,” or “key” in your spec.
- Never leave notes to the reviewer or sign off text.
- Anything you write here can end up copied straight into the real files later.

Why it matters: If it ends up in the files, an automatic scan catches it, and worse, it hands the model doing the task a free hint.

REAL EXAMPLE: after his files were generated, he found “dozens of A## and D##” internal reference codes still sitting inside every file, exactly the kind of builder residue this rule exists to prevent. He had to manually scrub every file by hand before it passed review.

5. One number, one source. Everywhere else must match it.

- Every number that matters lives in exactly one source file. Every other place it shows up has to match that number exactly, tagged with an ID so it is easy to check.
- Decide the winning number before any files get built, so the generator pulls one real value instead of inventing three different ones.

Why it matters: We have seen net income show three different values inside a single file, and a task where the purpose field and the expected answer field flatly disagreed with each other.

EXAMPLE: A build where net income reads \$482,000 on the income statement PDF but \$479,500 in the trial balance workbook, and nothing in the spec says which one is correct.

6. Traps show the wrong path only, and big enough to matter

- A trap shows the tempting wrong answer and stops there. Never show the corrected answer, both sides side by side, or a policy note that gives away the right treatment.
- Make sure the trap is big enough to matter. If the dollar difference is smaller than normal rounding noise, or the wrong answer and the right answer land on the same number anyway, the trap proves nothing.

Why it matters: A trap that quietly contains its own solution is not a trap. It is an answer key with extra steps.

EXAMPLE: A trap built around a \$50 rounding difference between two depreciation methods, when the world tolerates \$500 of rounding noise everywhere else. The wrong answer and the right answer land in the same bucket, so the trap proves nothing.

7. Recompute every figure. Do not eyeball it.

- Check that the trial balance actually balances.
- Check that the financial statements tie to each other and to the trial balance.
- Recalculate each task's expected answer yourself, from the real numbers in the file. Do not assume it is right.

Why it matters: If the numbers do not actually tie, the workbook is lying about its own math, and every task built on top of it inherits that lie.

REAL EXAMPLE (Slack, Shanmukh's review notes): a review came back flagging that the reconciliations did not tie: the bank balance did not match the GL cash account, the AR aging did not match the AR control account, and the payroll register did not match payroll expense. The GL, the trial balance, and the financial statements all needed to agree with each other, and they did not.

8. Lock the spec before anything gets built

- Every number that is still up for debate needs a final, locked value and a match ID before anyone signs off on the spec.
- If you sign off with numbers still open, the files get built already contradicting each other.

Why it matters: The spec is the script every file gets generated from. If it is still soft in places, the generator invents the missing value itself, and now you have a fact nobody actually chose.

EXAMPLE: A spec that leaves a contested revenue number unresolved. The generator fills the gap with its own guess, and that guess ends up baked into three different files before anyone notices it was never real.

9. Excel counts as a real input file

- Excel workbooks are allowed as world files. The rule some people repeat, that only CSV or PDF are allowed, is not accurate, and only applies to TASKING.
- Do not fail a spec, or force a rebuild, just because of file format. Use whatever format the real document would actually be.

Why it matters: Rejecting a real Excel file over a format rule, or forcing an artifact into PDF or CSV when a real accountant would use a workbook, makes the world less realistic for no reason.

REAL EXAMPLE: "Around 80% of the flagged issues are valid. A handful relate to Excel formatting/aesthetics, which are fine as-is." The QC process was flagging normal Excel formatting as an error when it was not one.

10. Search real-world facts. Do not recall them from memory.

- Any real number that comes from the outside world, like a statutory rate, a wage base, a filing threshold, a regulatory deadline, or a legal test tied to a specific state, has to be looked up at build time. Do not type it from memory.
- Pin that number as a canonical value, and record where it came from and the date you looked it up, so anyone can check it later.

Why it matters: A number pulled from memory instead of looked up is exactly how a wrong year, a wrong rate, or a wrong deadline ends up sitting quietly inside a world that otherwise looks correct.

REAL EXAMPLE (from past world reviews): a 2024 wage base used inside a 2025 payroll register, a Section 382 rate off by more than a full point, a PDUFA goal date wrong by three and a half months, and a hurricane deductible built on the wrong state's trigger.

11. Check that the task is actually hard, not just correct

- Every other check on this page asks whether a task is correct. Nothing else asks whether it is hard, so a seed can pass every other check and still fail to challenge a model at all.
- Watch for single-rule recall, where solving the task just means remembering one fact.
- Watch for an answer already printed inside a policy's worked example, so there is nothing left to solve.
- Watch for a wrong path so implausible that a model would reject it immediately without doing any real reasoning.
- Watch for back-solvability: a shortcut that reaches the right answer without doing the reasoning the task was supposed to test.

Why it matters: A task can be fully correct and still be worthless if a model can land on the right answer without doing the reasoning the task was built to test.

REAL EXAMPLE (from past world reviews): in one world, two separate tasks collapsed into a single piece of subtraction off a stated fully diluted total. In another, a task was solvable just by finding the only non-zero cell in a 53-row column, with no document leaking anything. The structure of the file gave the answer away by itself.

STAGE 2: FILE GENERATION (FILESYSTEM)

This stage turns your spec into the actual files. A tool reads the spec and creates every file automatically, then you use Claude Code to review and fix them, not by hand. Read each file the way a model seeing it for the first time would, since this is also where a task's answer can accidentally end up sitting in a file.

1. Check every file for answers already given away

- No file should state, hint at, or already calculate a task's answer. The obvious cases are rare. The sneaky ones are everywhere.
 - The right journal entry is already sitting in the trial balance or draft financials, so "solving" the task just means copying a number that is already there.
 - A memo already splits the allocation the agent was supposed to work out itself.
 - A file states a total the agent was supposed to add up on its own.
 - A policy note tells you the treatment straight up, like "apply prospectively," instead of making the agent figure that out.

Why it matters: Leaks come back after every fix round, not just the first one. Check for them every single time you touch the files.

EXAMPLE: A depreciation schedule that already shows the corrected monthly expense in a column labeled "Adjusted," right next to the original wrong number. The agent does not need to calculate anything. It just reads the right column.

2. Open the file and check the fact is actually there

- If a task depends on a fact that is supposed to live in a specific cell or file, open that exact cell or file and check the fact is really there.
- Check that it exists. Do not just check that you meant to put it there.

Why it matters: A missing fact that a task depends on makes the task impossible to solve, and that looks exactly like a model failing the task, when the real problem was a broken file.

REAL EXAMPLE (Slack, Luke B): after his files were generated, "4 of the 5 emails generated by the pipeline generator are corrupt and I can't even open them." Any task depending on those emails would have looked like a model failure, when the real problem was a broken file.

3. Hunt down numbers that do not match across files

- If the same number shows up differently in two different files, or even twice in the same file, that is a real defect, not a small thing.
- Check every important number against every file it appears in before you hand the world off. Do this with a script, not by eye. A person will miss one.

Why it matters: A number that changes depending on which file you open means the world does not agree with itself, and neither will any task built on top of it.

EXAMPLE: A vendor spelled "Nordholt Supply Co." in the vendor master file and "Nordholt Supply Company" in the accounts payable aging report. A script matching records by exact name would treat these as two different vendors.

4. Fix the spec, then regenerate. Do not hand edit the output.

- A contradiction lives in the spec, the source everything gets generated from. Editing the rendered PDF or workbook by hand only hides the problem until the next time the files get regenerated.
- Fix things in the order that actually works: fix the spec, get sign off, regenerate everything in one pass, then re-run QC.

Why it matters: Hand patching a symptom means you pay for the same fix twice, once now and once again at the next regeneration.

EXAMPLE: A builder notices a wrong date on one invoice PDF and edits just that PDF by hand. Three weeks later the world regenerates for an unrelated fix, and the wrong date is back, because the spec itself was never corrected.

5. Keep the files matching the spec

- The files you build should follow the spec exactly. Only deviate when it is genuinely necessary.
- If you do need to deviate, update the spec first, so the spec still reflects the truth. Do not quietly change a file and leave the spec out of date.
- For fixes, use Claude Code, per Danielle's instruction, instead of editing files by hand one at a time.

Why it matters: Silent drift means someone has to rebuild the spec from the files later, backwards, which is slower and more error prone than doing it right the first time.

EXAMPLE: A builder adds an extra bank statement to a world because it seemed useful, without updating the spec's artifact list. Weeks later, nobody reviewing the spec knows that file exists, so it never gets checked against the rest of the world.

6. Make sure a task cannot be solved by accident

- Watch for a contingency that nets to zero no matter which way it is decided, so the decision never actually changes the answer.
- Watch for a task where both the right path and the wrong path land on the same final number.
- Watch for a task that is solvable without ever opening the one file that is supposed to be required.

Why it matters: If the right answer is reachable without doing the real reasoning, the task is not testing anything at all.

EXAMPLE: A task asking whether to capitalize or expense a cost, where rounding makes both treatments land on the exact same net income. Getting the "right" answer does not prove the agent understood the accounting rule.

STAGE 3: APP DATA

The filesystem stage is closed now. Everything in this stage lives in one folder, `.apps_data`, and every reference in it has to actually resolve to something real.

1. Once filesystem passes, stop touching it

- Once the filesystem stage passes its check, it is locked. From here on, fixes only go into the `.apps_data` folder.
- If you find a filesystem problem at this point, log it for a separate fix later. Do not edit the filesystem live.
- Do not add extra files just because they seem helpful. Anything added that was not asked for gets flagged as drift. Being able to defend a change does not mean it was in scope. Flag it and ask first.

Why it matters: Editing a stage that is supposed to be closed means nobody can trust that a passed stage is actually still what it says it is.

REAL EXAMPLE (Slack, Shanmukh): "I've already passed filesystem QC 3 days back. Just because Spencer asked me to remove appdata from filesystem, I had to re-run the filesystem and again the errors are highlighted." Touching a stage that was supposed to be closed cost him a full extra QC cycle.

2. Every reference has to point to something real, both ways

- Every item code used somewhere has to actually exist in the item master list.
- Every foreign key has to point to a record that is really there.
- Every file needs a matching row in the schema registry, the master list of what is supposed to exist.
- Every task and its linked file have to point to each other. Check both directions, not just one.

Why it matters: One broken link in this chain fails the entire app data stage, not just the one file involved.

REAL EXAMPLE (Slack, Shanmukh): he uploaded a `Cost Center.csv` file that "maps to no entity in the world's schema registry, which declares 36 ERPNext entities, none named Cost Center." The file itself was a valid, correctly formatted dataset. It just was not registered anywhere, so the system had no way to know it was supposed to exist. He had to either register it properly or remove it.

3. App data has to match the paper files

- The general ledger export from the app has to match the trial balance and close workbooks sitting in the filesystem.
- Employee rosters have to be the same across every system that lists them.
- A document's stated total has to equal what its line items actually add up to.

Why it matters: The app data and the filesystem are supposed to describe the exact same company. If they disagree, the world is telling two different stories, and that breaks trust in everything in it.

EXAMPLE: An ERPNext general ledger export showing \$1.2M in total revenue for the quarter, while the trial balance workbook in the filesystem shows \$1.15M for the same period. Nothing in the world says which one is correct.

STAGE 4: FINAL WORLD REVIEW

This is the last check before a model ever sees the world. Your job is to break it yourself, before a model or a reviewer does it for you.

1. Check every automatic flag by hand before you trust it

- The automatic scan matches exact text, so it throws false alarms. Trace every flag back to the actual file before you act on it.
- If you decide a flag is a false alarm, prove that with the same level of detail you would use to prove a real defect. "I don't think this is real" is not proof.

Why it matters: A tool that is wrong some of the time is only useful if a person checks every result by hand, in both directions, not just the ones that are inconvenient.

REAL EXAMPLE (Slack, Luke B): "This world is ready for app data generation though so can we at least get that moving? Most recent QC had two false flags." Even a run with real errors in it can also contain false ones. Both kinds have to be checked by hand.

2. Try to prove your own answer key wrong

- Recalculate every task's answer yourself, straight from the built files, without looking at the answer key first.
- Actively hunt for a second answer that could also be defended, like a different accounting basis or a different ceiling on the calculation.

Why it matters: Finding a second defensible answer yourself, before a model finds it, is the cheapest bug you will ever catch.

EXAMPLE: A revenue recognition task where the answer key assumes the performance obligation was satisfied at delivery, but the contract terms actually support recognizing revenue over time instead. Both readings are defensible, so the task has two right answers instead of one.

3. Grade what the agent actually did, not what it says it did

- If an agent says it was blocked, treat that as a claim, not a fact. Check it against the actual files.
- Grade strictly against the specific deliverables the task named. Nothing else counts.
- The verdict is only Accept or Reject. There is no partial credit.
- If every single rollout you grade gets the same verdict, that is a warning sign. Go back and recheck how strict your bar actually is.

Why it matters: A grader who takes an agent's excuse at face value ends up grading the agent's story instead of its actual work.

EXAMPLE: An agent claims a required file "did not exist" and stops there. The file was present the whole time, just one folder deeper than the agent searched. Taking the claim at face value would have marked a passing rollout as blocked.

4. Check for leaks again after every fix

- Do not just check whether the requested changes were made.
- Check whether the fix introduced a brand new leak or problem that was not there before.

Why it matters: A fix that quietly adds a new leak is worse than doing nothing, because everyone now assumes the file is clean.

EXAMPLE: A builder fixes a leaked answer in one file, and while doing so pastes the corrected number into a second file for consistency, accidentally leaking the same answer into a place that used to be safe.

5. Do not fix one thing and break two others

- A fix that solves the flagged problem but quietly breaks two other things has made the world worse, not better.
- After every round of fixes, re-check the entire checklist. Do not just re-check the one item you changed.

Why it matters: Reviewers are checking whether the world holds up as a whole, not whether you fixed the one thing they happened to point at.

EXAMPLE: A builder renames a vendor to fix a name collision with a real company, but the old name is still hardcoded in two other files that reference it, creating a brand new contradiction across the corpus.

BEFORE YOU PASS ANY GATE, ASK

1. Could a reviewer find the answer written down anywhere in the world?
2. Could they argue a second, different answer?
3. Does any number disagree with itself somewhere else in the world?
4. Does the world support at least 5 clean tasks?

If you answer yes to any of these, the world is not ready. If all four are no, you have built a good world. Send it.



Phase 2: Task Creation Process

Making A Task

Before diving into the specifics of task writing, please watch the videos in the section [here](#) — they provide a general overview of the project.

📺 Task Writing Calibration Session Recording:

<https://drive.google.com/file/d/1bYoviMS-NnIII9oHDV37HnFFnIaAj4nk/view?usp=sharing>

★ Your Objective – Be A Test Maker, Investigator, & Teacher

1. Learn about an entirely fiction-based **world** (documents, files, data, people, etc)
2. Design a **task**, or challenge, via a prompt someone in your field might do in that world that is extremely hard
3. Run the AI, determine it *cannot* do that task – document why it failed
4. Design the steps someone would have to follow to complete that task
5. Create a “golden” or perfect example of that task getting done correctly – via making the correct output
6. Write a scoring methodology (rubric criteria) that could score if some output *did it correctly*. The golden answer should score 100%

A Julius task is a complex, multi-step piece of real domain work that today's frontier AI models cannot reliably complete. Your job as a writer is to design that task, prove it is genuinely hard, and capture a perfect example response so we can grade other model attempts against it.

The flow has two halves. The first half is prompt work: write the prompt, run trajectories to prove it is hard, document the failures, and define the approach a competent expert would take. That half is faster to iterate on. The second half is crafting: build the golden output and the rubric, run diagnostics and full QC, and submit for final review. That half is expensive, so do not enter it until your prompt has passed midpoint review.

The End-to-End Flow

Each step below builds on the one before. Steps 1 through 4 are quick to redo if your task is rejected, so iterate freely. Do not invest in steps 5 onward unless your midpoint reviewer confirms the task is ready.

[Please see the Quick Start guide for more information](#)

Before writing any task, know the world well enough to write a realistic, specified prompt grounded in it. Find your world and work through:

**Note: This only applies to legacy worlds and not worlds you build and own end-to-end.*

- **Scouting Report** — orientation doc; read first.
- **World Spec ("Map")** — Entity Profile, Scenario, Consistent Facts, Artifact Registry. The Artifact Registry tells you what files exist — essential for citing inputs in prompts.
- **Generated files in the world Drive** — open a few to see what artifacts actually look like.
- **Confirm you're in the world Slack channel.** If you can't ground a prompt in specific files and facts from the spec, you don't know the world well enough yet.

1

Understand Task Flows

Review and understand the Drafting Prompt + Prompt Review cycle and the Crafting + Final Review cycle.

2

Draft the Task Prompt

Develop a complex task seed grounded in real-world work. Create the task entry and write the prompt with file references, key assumptions, and the exact requested output.

3

Run Trajectories

Run the prompt against both frontier models (Tab 2). Iterate the prompt until both clearly fail in a valid way (reasoning, domain inexperience, or misreading world documents).

4

Document Failures

Identify failures through prompt review stages across both models, conduct in-depth trajectory analysis for one.

5

Define the Completion Approach

Fill in the completion steps (required) plus a human time estimate and the expected outcome type. Add supporting files ("show your work") when calculations or supporting materials clarify the approach. Build out these materials while iterating on the prompt and dialing in on model failures.

6

Review Additional Fields

Expected output, time estimates, show your work, and collapsing sections.

Midpoint Prompt Review Checks

7

Build the Golden Output

Once your prompt is approved, build the perfect example answer in the file format for your domain (Law: .docx, IB: .xlsx, Consulting: .pptx). The golden output should match your rubric perfectly.

8

Build the Rubric

Write 15 to 25 LMAAJ verifiers via Doc Eval (Weighted). Anchor every criterion to a value or claim in the golden output. Weight each by difficulty (see Tab 4) and write each per the six principles in Tab 5.

Final Review Checks**Run Diagnostics and Full QC**

Run the quick diagnostics to fix structural issues. Then run the full QC autograder, which runs the rubric against the data set and gives detailed feedback. Both must be cleared before final submission.

Submit for Final Review and Finalize

Mark diagnostics reviewed and submit. After approval, complete preference ranking, expand your failure analysis to the four-box structure, and record the golden trajectory for one trajectory (browser apps). Keep two or three active tasks at a time.

Common Prompt Mistakes

- Prompts do not require persona statements like "You are the FP&A Manager" or "You are outside counsel."
- Avoid storytelling, scene-setting and/or narrative details that don't materially affect the deliverable being graded.
- If information is not necessary for the agent to complete the task, it shouldn't appear in the prompt.
- Avoid artificial time references such as "Today is March 14, 2026." Instead, reference dates naturally within the task context (for example, "using Q3 2025 revenue data" or "per John's email dated May 17, 2025").
- Numeric verifiers should test exact expected values and may not use acceptable ranges (FP&A and Accounting exception).

Important Checkpoint: Midpoint Review Gate

Only proceed past the midpoint review if your task is realistic, challenging, and diverse, and (based on reviewing the trajectory) has induced real failures. The steps that follow are time-consuming; do not invest in them unless you have actioned the feedback and your midpoint reviewer confirms the task is

ready.

0 Task Quality Requirements

What Does A Good Task Look Like?

This section summarizes the project quality standards that define a high-quality task aligned with project aims. It covers what a good task looks like — not the step-by-step process for making one. Task elements are interdependent: understanding the rubric standards while writing your prompt helps you frame a better request, and knowing the alignment rules before building your golden saves rework.

Take the time to understand each quality standard, or ‘Spec’, and use them with the [Task Review Checklist](#) as a reference while writing and reviewing tasks. Also check out the [Example Tasks](#) for your domain.

For more detail on each task element’s requirements and the tasking process, reference the specific sections for [task prompts](#), [verifiers](#), [trajectory analyses](#) and [golden outputs](#).

Task Quality Requirements

High level summary

A task is a complete package: a prompt, a reference-correct answer (the ‘golden output’), a rubric of true-or-false checks (verifiers), and evidence that frontier LLMs fail genuinely.

Four quality standards determine whether a task meets the requirements. **Use the links in the ‘Spec name’ column below to get more details on each requirement.**

Category	Spec #	Spec name	Short description
1. Clear Prompt	Spec 1A	<u>Realistic and complex</u>	The task reflects realistic and long-horizon professional work.
	Spec 1B	<u>Specified for a unique correct answer</u>	The prompt and input files entail one correct approach.
	Spec 1C	<u>Timeless</u>	Prompt contains realistic date anchors
	Spec 1D	<u>Tightly scoped outputs</u>	Task outputs are testable within project criteria limits
	Spec 1E	<u>Self-sufficient</u>	Prompt is solvable using world files and task inputs only
	Spec 1F	<u>Purposeful</u>	All information in the prompt specifies an input, states an assumption, or requests an output
2. Genuinely Challenging Task	Spec 2A	<u>Gemini 3.8 Flash fails</u>	Gemini 3.8 Flash scores a mean of under 40% on the rubric across five runs
	Spec 2B	<u>Genuine reasoning failures</u>	Models fail due to reasoning limitations, not poor task design or technical errors
3. Complete and Correct Outputs	Spec 3A	<u>Calculations and logic correct</u>	Writer solution to the task is defensible and correct (evidenced via Show Your Work and golden output)
	Spec 3B	<u>Golden scores 100% on rubric</u>	Golden output scores 100% on the task rubric
	Spec 3C	<u>Golden output is client ready</u>	Golden output is professional and complete
	Spec 3D	<u>Task metadata completed</u>	Additional fields such as Expected Output, Time Estimate, and World Files are populated
4. High Quality Rubric	Spec 4A	<u>Verifier-level requirements</u>	Verifiers meet 6 key requirements: (1) Specific and Gradable, (2) Self-Contained,(3) Atomic, (4) Output-Focused, (5) Precise, and (6) Weighted.
	Spec 4B	<u>Rubric covers golden</u>	All important claims and values in the golden output have verifiers.
	Spec 4C	<u>Rubric traces to prompt</u>	Every verifier traces to something asked for or necessarily implied by the prompt.

1 Drafting Prompts

Drafting Prompts

This page covers all the prompt requirements:

1. [Realistic and complex](#)
2. [Specified for a unique correct answer](#)
3. [Timeless](#)
4. [Tightly scoped outputs](#)
5. [Self-sufficient](#)
6. [Purposeful](#)
7. [Example prompt](#)

Develop your task prompt in RL Studio to meet the following specifications for **Prompting (Specs 1A-1F)**.

Before you start!

Name your task with your domain abbreviation, your world shortcode if you have one, and a short descriptor of the request (for example, "IB w003 DCF for Acme" or "Consulting w012 SaaS growth deck"). The exact convention may vary by domain and world; check your world channel if in doubt. The point of the name is that anyone scanning the task queue should know within a few seconds what the task is and which world it sits in.

Prompt Requirements

Spec 1A

Realistic and Complex

Realistic and complex

- Your task should capture work professionals genuinely perform. What would you assign to a capable junior?
- An average expert in your domain with basic knowledge of the world would take **3 to 8 hours to complete your task without LLMs**.
- Tasks should require **deep reasoning** (in which each calculation or logic step depends on the outcome of the previous) rather than multiple independent asks.

Spec 1B

Natural prompting

Variety of task topics

Aim to maintain variety between tasks in your world. Tasks may share broad topics, draw from the same in-world data sources, and could result in similar failures. However, no two tasks should be close duplicates of each other (e.g., the same basic request applied to a different time period or company, or with only minor permutations). You should also not repeatedly index on exactly the same failure point in multiple tasks (e.g., the model confuses metric X with metric Y in file Z).

Natural Prompting Standards

Prompts should sound like a real user typing into Claude, not a spec doc

Key dimensions:

- Process specification: low, avoid spelling out step-by-step instructions
- Formality: natural sounding
- Length: short and tight
- Tone: realistic, not "Clauded"
- Grammar: tolerate some realism, but avoid spelling mistakes
- Formatting: no need for structured formatting, plain / realistic is fine
- Specific *output* file names: keep exact names (needed for verifiers and expected outputs)
- Input/world file references: prefer realistic phrasing like "most recent version of X" over exact versioned names where possible

SOPs and policy documents: move lengthy instructions into environment documents

- Writer drafts guidance as a realistic doc, uploads as context or world file
- Model finds and uses it, keeping the prompt short and natural

Output File Naming:

Your prompt cannot have spaces and must have the filetype in the name. The prompt section 1.4) Expected Output Files and the output dependencies section of the verifiers will not accept filenames with spaces or without filetype!

Ex.:

"Final Legal Memo" is NOT acceptable

"Final_Legal_Memo.docx" IS acceptable

Spec 1C

Specified for a unique correct answer

Specified for a unique correct answer

- The information in the prompt and the attached files, plus the findable information in the world, entail one correct approach. Human experts in your domain would agree on this approach given the same prompt and materials.
- **Describe the *what* and the *assumptions* – but not the *how*.**
- Describe input files as needed in an unambiguous and realistic way (e.g., “using Target’s Q3 2025 investor report”).

✗
"Prioritize the highest-value stalled accounts and recommend the single biggest strategic move." — Highest-value by what?

✓
Rank top 5 stalled Enterprise accounts by recovery-weighted ACV (Win% x ACV). Recommend the highest."

Specify the what, not the how

Define the deliverable, assumptions, and inputs precisely enough that only one correct output exists. Do not tell the model which steps to take. The challenge comes from the reasoning required to reach the answer, not from ambiguity about what the answer should be. If your task’s difficulty depends on the prompt having multiple defensible correct answers, the challenge isn’t valid.

If models take different paths and both produce acceptable correct outputs, the prompt is ambiguous
When tracing a trajectory, ask: is the model’s alternate path defensible given the prompt and files? If yes, you will need to tighten the prompt.

The below examples show typical information you need to specify or not specify in your prompts. *These should be considered a general guide, since task and world files will guide what’s required in each case.*

✓ **Specify these (the what)**

- **Time periods and as-of dates.** ‘Q1 2026,’ ‘as of March 31, 2026.’
- **Decision rules** if the output requires a conclusion or judgement. ‘Flag if the margin is below 15%.’
- **Assumptions** such as. ‘Entry multiple of 15x 2025 EBITDA’
- **Output scope and format.** What the deliverable contains, its file type, and its structure.

✗ **Don’t specify these (the how)**

- **Steps to follow.** ‘First open X, then calculate Y, then compare Z.’ This turns the task into a checklist.
- **Which pages, cells, or rows to look in.** The model should discover what’s relevant.
- **Exact formulae to use.** You may need to request a particular technique or approach but you should not give formulae which are common domain knowledge.
- **Excessive file references.** Name files realistically (‘Target’s Q3 2025 investor report’) and only where needed to ensure the prompt is not ambiguous.

Example of an underspecified prompt

Your prompt: "Prepare a staffing plan that includes the right roles for the new location." The model then produces a plan that is clearly wrong — it misses half the roles.
The problem: Your golden output is just a plan you think is good, but other HR professionals might staff differently. You also can't write specific verifiers because "includes the right roles" is subjective.

Spec 1D

Timeless

How to fix it

Tighten your prompt by giving a more specific framing, such as: "Calculate the headcount and cost for the new location using the staffing ratios in the V3 Expansion Policy". This moves the task towards an output grounded in specific numbers derived from a specific source.

Referencing input files in the prompt

Prompts should describe input files only when they need to be described for the request to be unambiguous (e.g., when you would realistically reference those files in a real-world request). When including file references, you should do so in a realistic style (e.g., "using Target's Q3 2025 investor report").

Timeless

Where required, prompts should reference the date – with day, month, year – to maximize timelessness. Dates are often critical to the task mechanics and how we trick the model.

In these cases, you need to weave the date into the prompt in a very explicit but still natural way:

- "Run this analysis using August 23, 2025 as the current date."
- "Get me the results up to the end of Q1 2026."
- "Use July 14, 2026 as the last date for recording invoices."
- "Prepare a counter-redline as of March 11, 2025 in response to the owner's prime contract redline (v2)."

Do not use contrived time references. The following forms in particular are not permitted: "Today is August 23, 2025" or "As of August 23, 2025".

x

"Ahead of next week's board meeting."

✓

"Based on the December 12, 2025 email from General Counsel."

Date formats

Use U.S. date formatting conventions when referring to dates in all task elements, including prompt, rubric, and golden outputs. *Prefer MM/DD over DD/MM.*

Standard formats are the following (using reference date August 23, 2025):

- "August 23, 2025"
- "08/23/2025" (or "08/23" if the year is referenced separately)
- "2025-08-23" (for digital/database references, where relevant)

Tightly scoped outputs

Scope within the verifier limit

Tasks should have **at least 5 verifiers**. For any tasks resulting in fewer than 5 verifiable outputs, ensure that the output request captures the full set of values and claims which would be important for a real-world output to the task.

Input file scope

Tasks should **generally not require more than 10 files as inputs**, and most will require far fewer. A high input file count may indicate that the task is scoped too widely, or is relying too heavily on data extraction for complexity.

Deep reasoning

Task the models with sequential, interdependent reasoning over many independent requests - models are not likely to fail at extracting parallel pieces of information. A task with one long dependent reasoning chain and tightly-scoped outputs is harder for models to get right and easier for us to evaluate.

Read through your prompt and consider the number of important values and claims that are requested in the output. **If your prompt has several sub-tasks all requesting many output values, your task is likely too wide, and will not meet the verifier limits.**

Prompt output restrictions by domain

Outputs for tasks in each domain are limited to the formats below. See [Domain-Specific Guidance](#) for further details.

Domain	Allowed Output Types	Notes
Accounting Consumer FP&A HR Sales & Marketing Consumer	You can request one of the following output types: 1. Direct output like an Excel or Powerpoint 2. Stateful app update You can also use up to two outputs of only this combination: • Stateful app update + Excel model.	Unique information must be conveyed in the two outputs that can be graded. All applicable output types must be selected in the "Expected output" field.
Corporate Law	You can request one of the following output types: 3. Direct output like a Word doc or an Excel 4. Stateful app update You can also use up to two outputs of only this combination: • Stateful app update + Word doc.	

Engineering	Word docs or Excel sheets.	No CAD files or technical drawings for the time being.
-------------	----------------------------	--

You must explicitly ask for your expected file output in your prompt.

Using Excel style templates

Prompts which require an **Excel workbook as part of their output** can optionally request for the output to be aligned with the world's style template, if the world includes one. In this case, incorporate the following language into your prompt to ensure alignment with the template (Update the placeholder for the actual style template file name in your world):

To create the spreadsheet, adapt the style template in: [\[style_template_sheet.xlsx\]](#).

Make sure to match the sheet's style and formatting, staying consistent with its presentation. You need to produce a professional, high-quality, and clearly-presented spreadsheet:

1. Strictly adhere to the provided style template (fonts, colors, layout, spacing).
2. If you have to design new tabs or tables, make sure they match the existing style.
3. Remove any tabs that are not required.
4. Placeholder numbers are not meaningful: Make sure to update all of the fields with the correct values.

- Reminders:
- If you request for the agent to align its output with the world style template, your golden output must also align with the same template.
 - The Expected Output for the task is still 'Make New Sheet', not 'Edit Existing Sheet' - the model will use the template to create a substantially new sheet.
 - Do not include verifiers for style. We test only for content in the final output.

Spec 1F

Self-sufficient

Self-sufficient

- Your task should be completable from world files and attached task inputs only. The task must not rely on information from web search.
- The task must require at least two world sources (app data counts).

Tasks are flat to the world: you cannot use the output of one task as the input to another, and you cannot add task outputs back to world files for future tasks.

When to use Additional Task Files

If you need a file that doesn't exist in the world, you may attach an **Additional Task File in section 1.3** for that one task and reference it in the prompt alongside the world files.

- Task input files must be realistic, relevant, and non-copyrighted (see the restrictions for [Task Input Files](#) here), and the task must still require at least two — ideally more — files from inside the world.

1. Prompt 🔒 Hidden in this status

1.1) Task Name & Details

Task ID
task_567d382bd1e8476392e954045c05abc6

Task Name
Task nio2416d

1.2) Prompt

User

New task - please update this prompt with task instructions.

📎 📄 📁 Upload to context

1.3) Additional Task Files

0 files, 0 B

🔍 Search files...

Filesystem

No files for this resource.

1.4) Prompt Goal

Prompt Goal*

What is the goal of your prompt?

Spec 1G

Purposeful

Purposeful

- **Every sentence in your prompt specifies an input, states an assumption, or requests an output.** Nothing else.
- No storytelling or scene-setting that does not represent strictly necessary context for completing the task.
- Prompt includes domain boilerplate text (output specification, ERPNext login details, style template request; see [Domain-specific guidance](#)).
 - LAW REDLINES: ***"Return the document with the same filename as the original. Make all edits with tracked changes enabled and return the document with redline edits visible."***

Persona and authorship

Do not begin your prompt with 'You are [name]' openers - instead, imagine you are the professional making the request to the model.

If the output should have a specific author or the model requires knowledge of a person's identity to complete the task, state it naturally as an assumption rather than through roleplay: "I sent an email to Karen flagging several issues I had with the redline edits made by Phil Durkin to the Brookfern MSA."

Write the prompt directly to the agent, the way you would hand off an assignment: "Prepare the subcontract for Halcrest Roofing on the Mereland project."

Example prompt

See an example prompt below, and further full task examples for each domain in the [Golden Examples](#) tabs.

Example Prompt (Law)

Prepare the first draft of a conservatively updated LoneRidge agreement using the available deal materials, including the draft change order, email chain on revised milestones, and the data migration agreement. Also ensure changes do not conflict with the 1977 Rose v. Spa Realty Assoc. Case. Use the MSA template as a starting point.

Return the document with the same filename as the original template. Make all edits with tracked changes enabled and return the document with redline edits visible. You need to produce output reflecting professional legal standards.

Difficulty Calibration

Designing for Failure

As you write your prompt and think through the golden solution and rubric, this section provides practical guidance for increasing task difficulty by understanding model failures, current model strengths and weaknesses, and techniques for turning your expertise into tasks that models reliably fail.

Generally, difficulty comes from two things: your deep domain expertise (which sets what is genuinely hard and what the correct answer is) and deliberate task design (the techniques below).

Fair failure vs. unfair failure

Not only are we looking for model failure, but we need it to fail for the right reason. The ones we want expose how the model **reasons**, which is a real limitation the model owns. A failure worth keeping needs to be fair:

Fair Failure: The model failed because of *its own* ability. It failed because of its own reasoning, knowledge, tool use, or output, not because the prompt was ambiguous, a fact was missing, the tools broke, or the grading was off. A fair failure looks like: the model misreads a clear instruction, invents a value instead of deriving it, defers to an authoritative-but-wrong source, or answers confidently when the honest answer =was "not enough information."

Unfair Failure: The model "fails," but the cause is the task design itself: an ambiguous or contradictory prompt, a rubric that doesn't match the prompt, missing data, a broken tool/harness, or unfair grading... The model didn't fail because of its ability, but because of the setup. An expert will not always be able to get it right either from what was provided.

Below are examples of fair vs. unfair failures a model can make.

✓ Fair failure (keep)

Answer genuinely wrong

The answer is materially wrong even though the prompt was clear and everything it needed was available.

Misreads a clear prompt

The prompt is clear, but the model still misreads it (e.g., ignores an explicit "exclude X").

Model misuses working tools

The tools work, but the model misuses them or assumes a silently-failed action succeeded.

Resolvable conflict

Two sources disagree, but one clearly governs (a stated precedence, or recency/seniority settles it), and the model still picks the wrong one.

✗ Unfair failure (exclude)	Fix to make the task fair (include)
Missing needed data You meant a specific answer to be correct, but did not include the needed data in the files, so the task is unsolvable as built.	Give the needed facts Trace the golden answer backward; confirm every fact it needs is in the files (or make "cannot be determined" the correct answer).
Ambiguous prompt The prompt can be interpreted in multiple ways and the model may correctly pick a reasonable one that wasn't intended.	Make it unambiguous Rewrite the prompt so there's only one reasonable reading. If you want the model to take a specific way, say so explicitly in the prompt. If more than one reading is genuinely valid, accept them all in the rubric.
Rubric-prompt mismatch The rubric rewards something the prompt never asked for, or demands one specific answer a strong expert couldn't have guessed.	Align rubric to prompt Map every rubric line to the prompt; credit any well-justified answer.
Broken tools / harness A tool errors out, a file won't open, or the grader is buggy. This is an infrastructure failure, not a thinking failure.	Test the environment Run the whole task yourself first; confirm every tool and file works.
Over-sensitive grading A correct answer is marked wrong over formatting or rounding (\$50,203 vs \$50,202.60; ½ vs 0.5).	Calibrated Grading Grade for what actually matters but stay strict where precision is the point. Match strictness to what the task is really testing.
Conflicting Instructions The prompt and another source (an email, a DB field) give directly contradictory instructions with no way to tell which wins. No single answer can satisfy both.	Remove the conflict or set precedence Get rid of conflicts in your prompt, or state a clear precedence rule (e.g., "the prompt governs over the email") so exactly one reading is correct.

What models are good and bad at today

Aim your task at what models are weak at, not what they are strong at. This reflects what we know now; the frontier moves and techniques saturate over time, so re-check strong tasks periodically.

Weak today (design toward these)	Strong today (do not build your task around these)
• Cross-document reasoning • Long-horizon, multistep reasoning • Applying professional convention and domain knowledge • Handling exceptions and edge cases	• Recalling common facts and well-known rules • Running standard, textbook methods and familiar problem types (pattern-matching a known solution path) • Following explicit, step-by-step instructions stated up front. • Forward calculations (inputs → result)

- Precise instruction following
- Discernment and independent judgment
- Tailoring output to audience and purpose

- Extracting stated text values and summarizing

Where to start

The best experts use curiosity and grit to figure out how to induce great failures. We have gathered some of their expertise and presented them here. If you are new, this is a great place to start: learn what the best of the best do. Please keep in mind that these are some best practices, but definitely not hard rules!

1. Start by thinking of a real, most specialized task you've done in real life

Pick a genuine piece of work you understand deeply and have actually produced, not a blank-slate "what would be a hard task" (that turns into aimless Googling). Then push past the surface: skip the routine, obvious version and reach for the specialized judgment only a practitioner in your domain would apply. For example, in education, almost anyone can state a learning objective; the specialized work is diagnosing a learning problem, proposing an action plan, and choosing the right pedagogical approach. That deeper layer, an uncommon case, or a call only a specialist would know how to make, is where the model is least likely to have seen the answer.

— EXP Kunal (Physics): "I tried a hard graduate problem my advisor used to stump students. The model solved it in 5 minutes. Then I tried to use research problems, and it got those too. Finally I used my actual PhD research. That worked. 11 of my 12 tasks are from my own research papers."

2. Pinpoint what makes it hard, and predict where the model breaks

Before you build, decide the one thing that should trip the model up, then picture the exact wrong answer it will give. Difficulty almost always comes from some kind of tension in the task: two sources that conflict, an inconsistency you have to catch, a special condition that overrides the normal rule, two things that look alike but aren't, or a requirement that's easy to miss. Point the task straight at that tension. You won't always predict the failure exactly, and that's fine, even a rough guess of where it'll break gives you something to design toward and test. Aim it just past the model's edge: hard enough to fail, but a strong expert still gets it right.

— EXP Dana (Generalist): "It can extract, but it cannot discern. Knowing that helped me frame my task better at the beginning."

3. Study examples, and learn from the best

You don't have to figure this out alone or from a blank page. Look at approved example tasks in your domain to see what a real stump looks like, and compare your idea against them. When you get stuck, ask early: bring your idea (or your near-miss) to your domain lead or reviewer, post in your Slack channel, or join office hours. The strongest writers treat this as a collaborative loop rather than trying to curate the perfect task alone.

A quick question often saves hours.

— EXP Matt (Finance): "Top writers 'iterate and keep going — I tried X, tried Y, any tips?'"

— EXP Nenad (Finance): "Even a 90–95% result is a starting point, not a failure. Dig into the trajectory, find the missed points, and spin the task around them."

Model stumping techniques

Below are some techniques gathered from our strongest experts and our research team. A technique does not guarantee a stump, but it boosts your chances, and you will often combine more than one.

Technique	What to do in your prompt	How the model fails (example)
Chain of dependent calculations [multi-step chaining]	Ask for a result that needs a sequence of dependent steps, where each step's output is the next step's input.	Step 1 needs "headcount excluding contractors"; the model uses the total roster, so every cost downstream is wrong.
Familiar rule that doesn't apply [make the wrong method look right]	Set a problem that looks standard but where the usual method fails. State the special condition plainly; never signal it is non-standard.	Reads like a standard supply-and-demand question, but a stated price cap voids the usual method, and the model runs it anyway.
Resolve a conditional rule [conditional statements]	Set an "if A, do B; if X, do Y" rule; the model must first work out which condition holds.	"Escalate projects >90 days late or <80% complete"; the case sits near both thresholds and the model picks the wrong branch.
Negation or exclusion	State a key requirement as a negation, using "not," "excluding," or "except."	"Active vendors except those flagged for review"; the model reads past the negation and includes the flagged ones.
Buried instruction [late constraint]	Place one requirement mid-prompt or inside a longer instruction; keep the other requirements prominent.	Asks for a summary, a risk rating, and a sensitivity table; the mid-paragraph table is never produced.
Look-alike terms	Use two terms that look alike but mean different things in your domain; require the correct one.	Asks for "gross profit" next to "gross margin"; the model reports the margin.
Unstated domain requirement [hidden rules]	Request a deliverable that carries a professional convention you never spell out (live formulas in a	Asks for a financial model; formulas aren't stated as required, so the model hardcodes values and the

	model, exact citations in a legal memo).	workbook is unauditabile.
Reason backward from a result	Give a finished result and ask which inputs or assumptions produce it. Make the inputs uniquely recoverable, so one answer is correct.	Asks which assumptions a half-finished spreadsheet relies on; the model can't recover them going backward.
Write for a specific audience	Name the intended reader, but do not say what to prioritize, emphasize, or how to format.	"Project update for the board"; the model writes a generic update instead of framing cost, risk, and decisions.

Inducing Failures

Inducing Failures

What Does it Mean to Induce a Failure?

In Project Julius, your aim is to create tasks that elicit genuine reasoning failures from frontier models (i.e., tasks that make models fail).

In the context of this project, inducing a failure means creating a task that causes a frontier model to fail the way a *careful-but-imperfect junior associate* would fail on real professional work. A failure can come from one of three sources.

Three Sources Of Failure

Requirements complexity. Many interdependent decisions, deep reasoning chains, or two authoritative sources disagreeing. Difficulty is in the structure of the work, not a planted obstacle or artificial constraint.

Designed failure points (a). Specific items embedded in the world such as an outdated opinion, a buried covenant breach, a defined-term divergence, or a cascade-inducing assumption.

Realistic messiness. Distributed context, mid-task requirement changes, or tool and environment friction. This mirrors real professional work.

Failure-Inducing Tactics That Actually Work

✓ TACTICS THAT WORK

Cross-document synthesis

Join data across three or more files; no single file has it all.

Multi-hop dependencies

Carry intermediate results across sequential steps.

Conflicting authorities

Two sources disagree; expertise is required to pick.

✗ LOW-SIGNAL, AVOID

Pure extraction

Reading one value out of one document.

Laundry lists

Many unrelated requests stacked together.

Jargon-only difficulty

Hard words over hard work.

Under-specifying

Prompt which have multiple defensible outputs are unfair.

The tactics below draw from all three sources of failure. Mix freely.

When running trajectories, review and categorize model failures into three tiers.

- **True reasoning failure (*signal*)**. The agent missed a fact, failed to cross-reference, produced an unsupported claim, or missed a designed failure point. These are genuine model failures.
- **Task-design bug (*your fault*)**. Ambiguous prompt, missing file, or answer not grounded in the world. These failures indicate the task structure or constraints need iteration.
- **Style or format diff (*noise*)**. Differences in voice, headings, or ordering that do not reflect a substantive error.

Tactic	How It Induces Failure	Example
Cross-document synthesis (requirements complexity)	The agent is required to read three or more files and join or match data across the sources to reach the answer. No single file contains everything.	Consulting: A market-sizing answer requires pulling the addressable segment from a market-research PDF, pricing from a sales dataset, and churn assumptions from a client-interview memo. Using any single source would give the wrong number.
Conflicting authoritative recommendations (requirements complexity)	The agent must evaluate two experts or rules that pull in opposite directions, where neither references the other. This forces a genuine judgment.	Consulting: Market-research firm A puts TAM at \$2.1B; firm B's methodology gives \$3.8B. Neither addresses the other's assumptions. The task requires picking one and defending the choice.
Multi-hop dependencies (requirements complexity)	The agent must carry intermediate results across multiple reasoning steps, where each step depends on the correctness of the previous one (sequential reasoning).	IB: Valuation requires forecasting revenue growth from a macro report, translating that into EBITDA using margin benchmarks, then applying a sector multiple. Any error in an intermediate step leads to an incorrect valuation.
Temporal and timeline reasoning (requirements complexity)	The agent must reconstruct a multi-step sequence across files, where the critical fact sits in an easy-to-miss location.	IB: Reconstruct how purchase price moved across an initial LOI (\$50M), a later email confirming \$45M, and a side letter reducing earnout by \$2.5M. A failing agent reports \$50M or \$45M instead of the \$42.5M net.
Interacting constraints (requirements complexity)	The agent must satisfy multiple constraints that interact or include conditional logic. It must avoid violating one constraint while trying to satisfy another.	Corporate Law: Draft a contract clause that limits liability to €5M unless the breach involves data privacy, in which case liability is uncapped, while also ensuring compliance with GDPR language requirements. A failing agent misses the conditional override, leading to an invalid clause.
Cascade failure (requirements complexity)	One wrong upstream assumption by the agent in a long calculation chain (multistep or circular numeric reasoning) breaks five or more downstream values. Tests whether the agent verifies the root before trusting derivatives.	IB: Using basic share count instead of diluted in an accretion/dilution model breaks EPS, pro-forma EPS, breakeven synergies, and the sensitivity table. One root error, five wrong outputs.
Action, not just reading (requirements complexity)	The agent is required to produce a calculation, reconciliation, or formula-driven deliverable. Extrapolation or recalculation is required, with realistic non-round numbers, specified rounding, and outputs that flow	IB: Build a DCF with stub-period fraction 0.4627 (Jul 15 to Dec 31, divided by 365.25), not 0.5 or 5.5/12. Hardcoded downstream cells should fail the grader.

	from inputs (no hardcoded results).	
Hidden domain rules (designed failure points)	The agent must know and apply industry standards, best practices that 80%+ of experts would agree on, or niche professional norms that a domain expert would know but the agent is not told.	Management consulting: A pricing strategy recommendation ignores the standard industry practice of quoting enterprise SaaS contracts annually upfront, resulting in unrealistic monthly pricing assumptions.
Outdated authoritative doc (designed failure points)	The agent is provided with a well-formatted summary or opinion that predates a later change. Tests whether the agent trusts authoritative-looking files over dates.	Corporate law: Outside counsel's clean IP opinion dated Jan 15; Amendment No. 1 dated Apr 3 materially changes the IP warranty. The agent cites the opinion without checking the amendment.
Defined-term divergence (designed failure points)	The agent must assess the same term with different thresholds or scope across two or more files. Tests whether the agent applies the wrong one.	Corporate law: "Material Contract" = \$250K threshold in the merger agreement but \$500K in the disclosure schedule. Agent applies the wrong threshold and misses two contracts.
Buried critical finding (designed failure points)	The agent must reference a decisive instruction, a hold, override, or exception, placed in a secondary file where casual readers would miss it.	IB: A covenant-breach disclosure in a footnote of the target's latest 10-Q. Easy to miss when the agent anchors on the press release and investor deck.
Apparent resolution that does not resolve (designed failure points)	The agent is provided with a document that looks like it settles the question but covers the wrong period, scope, or is overridden elsewhere.	Corporate law: A compliance certificate covers Q1 to Q3 2024, but the task is about a Q4 obligation. The agent treats the certificate as resolving the question.
Multiple interactions with a file (realistic messiness)	The agent must iteratively retrieve, filter, and update information across multiple tool calls. A missing step leads to incomplete results.	Investment banking: Building a comps table requires pulling financials from several filings, normalizing metrics, and updating the dataset with calculated ratios across multiple queries. Skipping a retrieval or misaligning data produces an incorrect comparison set.
Red herrings and version conflicts (realistic messiness)	The agent is provided with Draft vs. Final, V1 vs. V_Final, or superseded documents. Tests whether the agent correctly filters for what is current, not what is well-formatted.	Consulting: A polished client-ready deck dated two weeks ago; a follow-up email from the engagement partner notes revised numbers that never made it into the deck. The agent uses the deck's numbers.

100 100+ Step Requirement

Step Plan + Model Checkpoints

Production tasks must be designed so that at least one of five Gemini 3.8 trajectories contains 100 or more steps, while preserving every existing difficulty and quality requirement.

High-level requirement

At least **1 of the 5 Gemini 3.8 trajectories** must reach **100+ steps**. This gate applies to production tasks before Prompt Review. **Work Trial tasks are exempt**. Existing task-difficulty requirements remain unchanged.

Steps vs. Tool Calls

A **step** is the transition from one reasoning trace to the next, including the transition from the final reasoning trace to the agent output.

- **Step count and tool-call count are different metrics.** App reads and writes often create additional reasoning steps, but 100 steps does not mean 100 tool calls.
- Do not count prompt instructions, files, rows, or requested outputs as steps.
- Do not estimate the result from prompt length or tool-call volume. Verify the count from the completed Gemini 3.8 trajectory's reasoning sequence.

Designing for 100+ Meaningful Steps

- **Start with a realistic long-horizon workflow.** Choose work that would take a human professional at least 20 hours.
- **Create true sequential dependencies.** Later decisions should require outputs from earlier research, calculations, reconciliations, or state changes. The agent should not be able to complete step 5 without completing steps 1–4.
- **Distribute evidence across sources.** Place required information across files and app records so the agent must retrieve, reconcile, and carry forward the relevant state.
- **Use multiple meaningful app reads and writes.** Require real database lookups and updates where the task naturally calls for them, and grade required state changes with App DB verifiers.

- **Prefer one connected workflow over parallel fetch work.** Independent items that can be processed simultaneously do not create the intended sequential reasoning burden.
- **Avoid artificial inflation.** Do not add filler, repetitive actions, or arbitrary tool calls merely to increase either metric.
- **Validate before submission.** Run all five Gemini 3.8 trajectories and confirm that at least one reaches 100 steps. The task must still satisfy every existing quality and difficulty gate.

You must complete section 1.3) of the prompt stage to validate that your task will reach 100+ steps. Examples of checkpoint steps below:

Accounting - Vantage Consumer Group

AT1 - Q4 2025 Commission Close and Off-Cycle Payroll

Checkpoints

#	Checkpoint - what the model must do
1	Pull Q4 2025 (Oct-Dec) sales for the 125 in-scope commissioned reps from the SQL warehouse into q4_commission_sales (flagging split/disputed attribution) and build the hr_master_sales org table from BambooHR, resolving everyone to their commission employee_id - these two tables seed every calculation that follows.
2	Using q4_commission_sales and hr_master_sales, validate the commissioned roster against ADP and BambooHR (catching unpropagated promotion/termination memos), keep mid-quarter terminations in-scope, and write the confirmed commissioned_roster_validated that steps 3-6 price against.
3	For the validated roster, roll the comp_plan_reference tabs forward from the prior period and reconcile each plan to the signed comp-agreement PDFs, capturing Q4 changes and flagging unrepresented provisions - the resulting rate structures drive attainment and commission math.
4	Against those rolled-forward plans and the disputed-attribution flags, log the six-category judgment calls into comp_plan_judgment_log with Vantage's standing treatments, so the attribution and treatment decisions feed the attainment and commission calculations.

#	Checkpoint - what the model must do
5	Applying the judgment-log treatments, compute each rep's Q4 attainment in q4_attainment_workings using pro-rated quotas and FY2025 annual cumulative sales for tier and accelerator - the tier each rep lands in sets their commission rate next.
6	Using attainment and the rolled-forward plans, build commission_calc_workings for all 125 reps (base_commission from the BU tier accelerator; Executive pay from the executive schedule) - this per-rep gross is the base for overrides, the top-20 recheck and payroll.
7	From hr_master_sales derive the sales_hierarchy, then roll the commission_calc_workings rep gross up each Sales Director's team to compute overrides (0.5% + 0.25% overlay) and the SVP's 0.5%, zeroing sub-threshold managers, producing manager_override_workings.
8	Before payroll, validate the hierarchy (every rep to one Director, each Director to the SVP) and tie the sum of Director team aggregates to the underlying rep gross within \$0.01, logging results - this gate must clear before amounts go to ADP.
9	Independently re-derive the top-20 BU earners via SQL re-aggregation (netting credit memos posted after the 2025-12-22 cutoff) and flag deltas over \$1 against commission_calc_workings, confirming the calculated gross before it is paid.
10	In parallel, pull the 100 flagged Q4 timesheets from ADP, classify each, and apply the correctable fixes - the corrected hours feed the gross-to-net for any rep who is both commissioned and timesheet-corrected.
11	With commissions, overrides and timesheet fixes settled, set up the ADP off-cycle run for the combined 130-person population, pulling amounts from commission_calc_workings and manager_override_workings and confirming headcount and totals match within \$0.01.
12	On that staged run, clear any invalid payroll configuration and run the gross-to-net preview (no zero-pay or error employees), then execute the off-cycle commission run.
13	From the executed run, reconcile the ADP register to the calculated amounts per employee (flagging >\$1 variances) and confirm employer tax accruals - this reconciled register is what the journal entry books from.
14	Map each ADP account on that register to the correct entity-specific QuickBooks account and post the balanced JE once per business-unit entity (JE-Q4-CMSN-FINAL, no undo), writing the per-entity posting summary.

#	Checkpoint - what the model must do
15	Finally, pull the posted QB ledger and build the calc-to-ADP-to-QB three-way reconciliation by cost center, then roll every upstream open/flagged item into the consolidated q4_discrepancy_log and the audit-readiness email to the CFO and HR head.

AT3 - Q4 2025 Multi-State Sales Tax Reconciliation

Checkpoints

#	Checkpoint - what the model must do
1	Initialize the sales-tax close checklist from the prior quarter and aggregate the full Q4 US-shipped e-commerce universe (BU2 plus BU3, tagged by source) into q4_us_ecommerce_transactions - the transaction base for every reconciliation.
2	Pull TaxJar's per-state collected summaries into q4_taxjar_state_summary for each nexus state, giving the collected side of the reconciliation.
3	Three-way reconcile each state - transactions to TaxJar to the QuickBooks accrued liability (2120, BU2) - and validate exemption certificates, establishing the baseline the disputed-product and holiday work adjusts.
4	Identify the 4 disputed product lines from the launch log and, for each, retrieve TaxJar's per-state rate explanation - the default treatment the DOR research will test.
5	Pull the state DOR rulings for each disputed product and build the cross-product shared-ruling map, marking rulings that govern two or more products.
6	Pull the company's prior treatment of similar SKUs to anchor each product's ruling in precedent alongside the DOR research.
7	Using the shared-ruling map and prior treatment, form coherent cross-product rulings per the resolution policy (citing DOR where it overrides TaxJar, flagging open_default cases) and write a memo per product - these rulings drive the corrections.
8	Verify each ruling is applied consistently across every state the product sold in, documenting state-specific exceptions.

#	Checkpoint - what the model must do
9	Apply the finalized rulings to all Q4 transactions per product per state, computing the per-state correction amounts into the pending-corrections table.
10	Before posting, run the aggregate materiality gate per product against the policy threshold to set each product's classification and posting approach (local vs restatement-pending).
11	Post the corrections to account 2120 per the gate - TaxJar refunds plus matching QuickBooks JEs for under-threshold, the TN rate-change remit, and bulk entries routed to the tax director for over-threshold.
12	Separately, audit Q4 transactions against each state's holiday windows and run the cross-state systemic-config gate, setting a bulk-reclass or per-state approach for multi-state overcollection patterns.
13	For states crossing the \$100K nexus threshold mid-Q4 but not yet registered, compute the uncollected-period accrual, create first-time returns, and post the accrual as a discrete JE to 2120.
14	After all corrections, holiday reclass and nexus accruals post, re-reconcile per-state TaxJar-to-QuickBooks and finalize the four per-product memos with the consistency, materiality and holiday findings.
15	Roll everything into the cross-product strategy memo for the tax director, compile the audit-ready package, save the draft email, and run the final close verification logging each criterion pass/fail.

L2 - Corporate Governance & Control-Threshold Review

Checkpoints

#	Checkpoint - what the model must do
1	Create the Corporate Governance session and map Smith's officer/director positions, governance/veto rights and committee memberships - the foundation every later workstream draws on.

#	Checkpoint - what the model must do
2	From that mapping, identify the charter/bylaw provisions triggered by his retirement with the DGCL remediation basis.
3	Using the governance findings, compile Smith's full equity schedule and confirm each security's DGCL 212 voting rights.
4	For each security, flag transfer restrictions and retirement-triggered repurchase/redemption rights with required action and timeline.
5	Review the ESOP docs, applying ERISA 203 vesting, IRC 409(p) prohibited-allocation and the 1.409A-1(h) separation triggers to Smith's combined position.
6	Inventory the surfaced side agreements and assess authority, ratification, enforceability and promissory-estoppel risk under Delaware law.
7	Identify the legacy comp arrangements (cross-referencing the side agreements), apply IRC 409A and quantify income inclusion and 20% excise exposure.
8	Using the legacy-comp figures, assess IRC 280G: base amount, contingent payments vs the 3x threshold and shareholder-approval feasibility given his control.
9	Flag customer contracts referencing Smith by name or as key-man, determine consent/termination/survival and quantify revenue at risk with a recommendation.
10	Using the customer revenue-at-risk, review credit facilities for key-man covenants, guarantees, MAC and change-of-control, mapping lender rights and timelines.
11	From the lender findings, develop the strategy: guarantee-release feasibility, cross-default and the escalation matrix keyed to the retirement date.
12	Using the governance findings, identify non-voting post-retirement influence mechanisms, apply DGCL 141(a) and rate High/Medium/Low with remediation.
13	Using the voting rights mapped in step 3, assess Smith's effective voting control, quantify the percentage and produce the transition plan.

#	Checkpoint - what the model must do
14	Produce every Excel deliverable exactly to its template (tab names and row-1 headers case-sensitive; the row-2 hint replaced by the first real data row using the specified types and enums).
15	Write each deliverable to its filesystem/<filename> path with the headers as the first row and no title or description rows above them.

HR3 - Coastline Pilot Integration into GlobalAir ALPA

Checkpoints

#	Checkpoint - what the model must do
1	Identify the 612-pilot Coastline workforce from the UKG acquisition-close export and populate the Summary and Coastline Roster in coastline_integration.xlsx.
2	Reconcile the roster against the parallel payroll snapshot (authoritative), holding the 4 payroll-only add-backs, so 608 pilots proceed as fully integrated (frozen at 612).
3	Pull the live GlobalAir ALPA seniority list from Workday into its tab, kept separate from the Coastline roster until the merge.
4	Read the merger LOA and arbitration award together to extract the seniority-credit penalty, tiebreaker and overflow rule into the read-only Integration Rules tab - the rules the next steps apply.
5	Apply the credit penalty to each Coastline pilot's hire date to compute an adjusted seniority date into the Adjusted Date Register.
6	Sort and validate the adjusted-date set: apply the tiebreaker to shared dates, confirm completeness against the frozen roster, and flag out-of-range dates - only a passing set is final.
7	Merge the validated Coastline dates into the GlobalAir list sequentially from earliest upward (each insertion shifts natives below it), assigning Integrated Slot Numbers.

#	Checkpoint - what the model must do
8	Map each equipment qualification to a GlobalAir profile, tracking fill against establishment limits and routing overflow to the adjacent profile in seniority order.
9	For each assigned profile, compute the GlobalAir-equivalent salary from the bands, keyed on the length-of-service tier from the adjusted date.
10	Cross-validate the seniority, role and compensation tabs (every pilot present with matching profile) before pay protection, clearing all exceptions.
11	Apply pay protection, setting Final Protected Salary to the higher of Coastline current and GA-equivalent and flagging protected pilots.
12	Sum post-protection payroll across the 608 and validate against the merger ceiling; if breached, halt all writes and escalate to the board.
13	Once within budget, file each pilot's Workday hire, ADP pay profile (salary = Final Protected Salary) and DocuSeal integration offer.
14	Update BambooHR, open a compliance case per pilot, and run the BW1 preview reconciling actuals to Final Protected Salary with a cross-population anomaly check.
15	Produce the ALPA ratification export over the full integrated list (~2000 pilots) and the closure package reconciling the integrated population to the merger-plan target.

LAW EXCEPTION

The Law domain is excepted from this general guidance wherever it conflicts with current Law-specific guidance.

Continue to enforce the 50-word prompt limit. Do not lengthen Law prompts or add artificial instructions solely to chase 100+ steps. Keep Law prompts concise and create sequential depth through evidence review and meaningful app actions. Until updated Law-specific direction is issued, existing guidance takes precedence.



Task Files

Task Files

What are Task Input Files?

Input files are documents attached directly to a specific task. They belong to one task only, and the agent sees these files at task runtime alongside world files.

This page covers:

1. [When to use task input files vs world files](#)
2. [Using input files to make tasks more challenging](#)
3. [What not to upload](#)
4. [Copyright and legal restrictions](#)
5. [How to upload](#)
6. [FAQs](#)

When to Use Task Input Files vs World Files

World files are the default. They are uploaded at the world-building stage and represent the canonical artifacts of the simulated company, including master ledgers, employee directories, standard contract templates, and regulatory filings the whole world references. They are reusable across many tasks and serve as the shared source of truth.

Task input files are an option for adding documents that belong to one task only. They represent an edge case, a one-off scenario, or a complexifying document.

- A redlined draft for a single negotiation
- A quarterly variance memo for one specific analysis
- A customer complaint relevant to one workflow

Decision Rule

If more than two or three tasks in the same world would reuse the same document, it should be a world file. If it is specific to one task's scenario, it is a task file.

Using Task Files to Make Tasks More Challenging

By using task input files rather than promoting everything to world files, we can introduce per-task complexity that would be confusing if it lived globally. Good uses include:

- **Distractor documents.** A near-relevant memo or report that does not actually answer the prompt forces the agent to filter rather than pattern-match.
- **Version conflicts.** A marked-up draft and a clean final require the agent to reason about which is authoritative.
- **Format variation.** A document with non-standard header structure, OCR-quality noise, mixed languages, or unusual layout tests robustness.
- **Implicit-knowledge edge cases.** A document containing information the prompt does not explicitly request, but a competent domain expert would know to check.
- **Noisy supplementary material.** Partially redundant documents that compete with the canonical source for the agent's attention.

What to avoid

- Padding tasks with files just to add bulk. Every task file should change something specific about the model's task, whether by providing necessary information, introducing a controlled distractor, or stressing a particular capability.
- Files the agent simply ignores, which add no signal.

What Not To Upload

- Real company financials, legal documents, HR records, or operational data, even from companies you have legitimate access to. Use the synthetic equivalents from your world.
- Personal information of any kind, including names, emails, phone numbers, account numbers, addresses, and government IDs.
- Files containing your real signature, employer logo, or identifying metadata.
- Files from paid services you have licensed access to, such as Bloomberg PDFs, paid research, or subscription databases. The license restricts redistribution even when you have permission to view.
- Files that would identify the real underlying company in a synthetic world.
- AI-generated files that have not been hand-reviewed for residual prompt instructions, AI-tool references, or implausible content.

Copyright and Legal Restrictions

All the same legal restrictions that apply to world files also apply to task input files.

Standard Project Julius Restrictions

- **No copyrighted content.** Anything under copyright is unacceptable.
- **No licensed content.** No paid licenses, special permissions, or restricted-use material.
- **Attribution:** if licensed for reuse with attribution (e.g., Creative Commons), include proper attribution.
- **AI-generated content is acceptable for fictional scenarios when hand-reviewed.** References to the AI tool used must be stripped before the file is uploaded. Treat generated files as starting points, not finished products. See [this tab](#).
- **If uncertain,** escalate to your Domain Lead before uploading. It's better to catch a problematic input file early, before it leads to a late-stage rejection.

How To Upload

1. In Studio, open the task in edit mode and find the Additional Task Files in section 1.3
2. Click Add File, select the document from your computer, and confirm the upload. The file appears in the task's input pane and is available to the agent at trajectory time.
3. Verify after upload that the file is listed under the task's input files in the review view, and reference it in your prompt in a realistic way..

Note: Refer explicitly to task input files as an “attached file” in your prompt.

FAQ

Can task input files conflict with world files?

They sit in different snapshot paths, so technically no. If a task input file and a world file have similar names or contradictory content, the agent may get confused. Be explicit in your prompt about which file you want the agent to use.

Can reviewers see my task input files?

Yes. Reviewers see everything the agent sees. They should also verify your input files meet the copyright and legal restrictions before approving the task.

What if my world does not have an input-files upload section in Studio?

Then per-task file storage is not enabled on your world yet. Talk to your domain lead; they may need to enable task-level file storage on the world's configuration.

2 Verifying Difficulty

Verifying Task Difficulty Against The Models

This page covers:

1. [How to verify task difficulty](#)
2. [Task difficulty requirements](#)
3. [Identifying genuine reasoning failures](#)
4. [What counts as a valid failure?](#)
5. [Valid vs. invalid failure examples](#)

How to Verify Difficulty

Tasks are tested against two frontier models: Gemini 3.8 Flash and GPT Sol 5.6. You run five trajectories for Gemini 3.8 Flash and three for GPT Sol 5.6 — eight in total. Each run is a trajectory — an agent's attempt to complete the task.

Once you have constructed your prompt, run five trajectories for Gemini 3.8 Flash and three for GPT Sol 5.6 to see if they can solve it when given the same files and environment. You will need to:

1. [Run trajectories for both models — five for Gemini 3.8 Flash and three for GPT Sol 5.6 — and review the outputs.](#)
2. Write a short [failure analysis for Gemini 3.8 Flash at the prompt-review stage](#), then an in-depth analysis on Gemini 3.8 Flash's lowest-scoring run.

See the tabs linked above for step-by-step instructions.

Task Difficulty Requirements

Verify that your task meets the below requirements for Difficulty (Specs 2A & 2B).

Spec 2A

Models produce incorrect outputs

Models produce incorrect output

Run five trajectories for Gemini 3.8 Flash and three for GPT Sol 5.6. Gemini 3.8 Flash must produce incorrect output — getting at least half of the key outputs wrong (output figures or claims incorrect or missing).

Note: Before submitting for Prompt & Trajectories review, you do not need to have completed all five Gemini 3.8 Flash runs. If you identify failure in at least one run per model, you can run the rest while working on your Rubric in the next stage.

Spec 2B

Genuine reasoning failures

Calibration gate (Mean Rubric Score): Gemini has to have a mean score of under 40% across 5 runs. If your runs are borderline against these thresholds, flag to the SPLs or Quality Team — it may be possible to move the task forward.

Gemini 3.8 Flash must fail in a serious, valid way with significant parts of the output must be incorrect or missing, reflecting a genuine capability limit (see Spec 2B).

Genuine reasoning failures

The model produces an incorrect and unacceptable output resulting from a real reasoning limitation — not simply problems with prompt ambiguity, system errors, file search, or formatting.

✖

"The agent wasted 12 steps searching for the file." — Process, not output.

✔

"Applied flat 10% instead of graduated 8%/12%/15%, collapsing \$340K of tier-specific revenue."

● Not valid failures

• Style or formatting

• File search inefficiency

• Console vs file output

• Model choosing a defensible alternative to an ambiguous prompt

• System errors

What counts as a valid failure?

✔ Valid Failure	✖ Invalid Failure
The model should fail to produce correct outputs to your task. This could be the result of shortcomings in: Reasoning — broken or abandoned multi-step logic, downstream work built on a wrong step. Calculations — numeric output wrong by a margin unacceptable in real work (wrong formula, unit, period, or dropped adjustment). Domain knowledge — misses a standard-of-practice step any practitioner in the domain would include.	Ambiguous prompts. Tasks must be solvable given the information provided. Misaligned rubrics. Criteria must measure the most salient aspects of a good output. Missing or erroneous world files. Highly subjective questions of taste or style; style and formatting issues in the output. Technical issues not directly related to task reasoning, such as corrupted output files. System or model error and timeouts.

Inefficient workflow which does not affect final output.
File search issues (e.g., relating to “WebDAV” or “Nextcloud”).

Failure examples

Domain	✓ Valid failure	✗ Invalid failure
Corporate Law	Gemini 3.8 Flash redlines an MSA against an outdated template pulled from an "old templates" subfolder instead of the current one the prompt named. Entire redline anchored to wrong baseline. <i>Reasoning failure.</i>	Gemini 3.8 Flash redlines the correct template and catches every material issue but phrases two clauses more verbosely than the golden. <i>Style difference, not a valid failure.</i>
Sales & Marketing	Gemini 3.8 Flash accepted that a 40% discount would bridge the \$2.1M Q3 gap, despite noticing that a "CRO no 40% match" constraint existed. The invalid 40% figure then propagated into the ERPNext update and Q4 renewal risk calculation. <i>Reasoning failure.</i>	Gemini 3.8 Flash calculated the Q3 gap and maximum discount without consulting ERPNext, relying solely on the data provided in the brief. However, the discount recommendation and Q4 renewal risk figures were correctly calculated. <i>Process failure that didn't affect the output.</i>
FP&A	Gemini 3.8 Flash builds a capital project NPV analysis correctly but discounts levered project cash flows (after debt service) at the company's unlevered WACC, double-counting the capital structure. Error compounds into an inflated NPV and a wrong investment recommendation. <i>Reasoning + domain knowledge.</i>	Gemini 3.8 Flash completes the revenue build correctly but uses 2.0% annual price escalation instead of the 2.5% that the writer used in their gold. Neither is implied by the prompt or any world file. <i>Prompt is underspecified. Tighten the prompt for a unique correct output.</i>

Running Trajectories

Running Trajectories

★Your Objective – Run Models Against Your Task, Explore Results and Analyze Failures

- 1. Run trajectories each time you change your prompt.
- 2. Review the trajectory logs and outputs using the below guidance to inform any changes you may need to make to your prompt to meet our requirements.

This page covers:

- 1. [The two models you must run your task against](#)
- 2. [How to run trajectories for each model](#)
- 3. [Confirming trajectory status](#)
- 4. [Checking trajectory logs and outputs](#)
- 5. [Adjusting or canceling trajectories](#)

Step 1

Run Agent (AI Models)
Against your Prompt

The Two Models You Must Run Your Task Against

You need to run your task against both of the following frontier models — Gemini 3.8 Flash and GPT Sol 5.6 — running five trajectories for Gemini 3.8 Flash and three for GPT Sol 5.6, while iterating and before submission for reviews. The agent will return a set of outputs and a log of its **trajectory**.

Gemini 3.8 Flash	GPT Sol 5.6
Google's frontier model.	OpenAI's frontier model.

How to Run Trajectories for Each Model

All tasks run on Julius, with PDF, filesystem, code execution, and Nextcloud available to the agent. Trajectories are generated on the agent and orchestrator combinations listed in the table below.

Agent harness*	Orchestrator
React Toolbelt Agent (250) - no BUA	Gemini 3.8 Flash

<

Analyzing trajectories for apps

If your prompt asks the model to use apps like ERP Next, you need to go through the model's trajectory and track its interaction with the app. For ERP Next, the agent will actually enter the application, and you must verify/check this.

- [Playground](#): provides video of the agent's actions. Can watch it go through the app
- [Log](#) (access by clicking on the individual trajectory): Step-by-step agent process moving through the app with reasoning.

Analyzing trajectories for apps

If your prompt asks the model to use apps like ERP Next, you need to go through the model's trajectory and track its interaction with the app. For ERP Next, the agent will actually enter the application, and you must verify/check this.

- [Playground](#): provides video of the agent's actions. Can watch it go through the app
- [Log](#) (access by clicking on the individual trajectory): Step-by-step agent process moving through the app with reasoning.

- ### Analyzing trajectories for apps
- If your prompt asks the model to use apps like ERP Next, you need to go through the model's trajectory and track its interaction with the app. For ERP Next, the agent will actually enter the application, and you must verify/check this.
- [Playground](#): provides video of the agent's actions. Can watch it go through the app
 - [Log](#) (access by clicking on the individual trajectory): Step-by-step agent process moving through the app with reasoning.

Viewing Model Output Files

Use the 'View Files' button underneath the 'DIFF' section of the model trajectory logs in the trajectory view to download the model's output files.

Viewing Model Output Files

Use the 'View Files' button underneath the 'DIFF' section of the model trajectory logs in the trajectory view to download the model's output files.

Advanced Step

Adjust or Cancel Trajectories (Model Runs)

Adjusting or Canceling Trajectories

Rerun a trajectory whenever you hit a rate-limit, system, or model error — these aren't real failures and shouldn't be analyzed as such. Cancel a run that is clearly stuck or no longer needed. If you significantly change your prompt (for example, swapping source files or targeting a different output), delete the trajectories irrelevant to your final submission to simplify the reviewer's work.

Managing Used Trajectories: Keep all trajectories run on the final version of your prompt. Delete any trajectories that are based on outdated versions of your prompt.

Advanced Step

Adjust or Cancel Trajectories (Model Runs)

Adjusting or Canceling Trajectories

Rerun a trajectory whenever you hit a rate-limit, system, or model error — these aren't real failures and shouldn't be analyzed as such. Cancel a run that is clearly stuck or no longer needed. If you significantly change your prompt (for example, swapping source files or targeting a different output), delete the trajectories irrelevant to your final submission to simplify the reviewer's work.

Managing Used Trajectories: Keep all trajectories run on the final version of your prompt. Delete any trajectories that are based on outdated versions of your prompt.

Adjusting or Canceling Trajectories

Rerun a trajectory whenever you hit a rate-limit, system, or model error — these aren't real failures and shouldn't be analyzed as such. Cancel a run that is clearly stuck or no longer needed. If you significantly change your prompt (for example, swapping source files or targeting a different output), delete the trajectories irrelevant to your final submission to simplify the reviewer's work.

Adjusting or Canceling Trajectories

Rerun a trajectory whenever you hit a rate-limit, system, or model error — these aren't real failures and shouldn't be analyzed as such. Cancel a run that is clearly stuck or no longer needed. If you significantly change your prompt (for example, swapping source files or targeting a different output), delete the trajectories irrelevant to your final submission to simplify the reviewer's work.

Adjusting or Canceling Trajectories

Rerun a trajectory whenever you hit a rate-limit, system, or model error — these aren't real failures and shouldn't be analyzed as such. Cancel a run that is clearly stuck or no longer needed. If you significantly change your prompt (for example, swapping source files or targeting a different output), delete the trajectories irrelevant to your final submission to simplify the reviewer's work.

Trajectory	Annotated	Status	Harness	Model
3		Completed	React Toolbelt Response api agent (250) - no BUA	openai / gpt-5.5 {"temperature":1,"reasoning_effort":"high"}
2		Completed	React Toolbelt Agent (250) - no BUA	gemini / gemini-3.1-pro-preview
1		Completed	React Toolbelt Agent (250) - no BUA	anthropic / claude-opus-4.7 {"temperature":1,"reasoning_effort":"high"}

Show Details



Failure Analysis x

Trajectory Failure Analysis

Summary

Your failure analysis is a short description of how and why the model failed your task to help justify to your reviewer that the model failures you have found are valid. You need to write before submitting for Prompt Review:

- 1. **Gemini 3.8 Flash:** One-paragraph failure analysis (one trajectory).

Note: You only need to rewrite when the prompt changes, resulting in a change to requested outputs. If the prompt remains materially the same through Crafting and Final Review, you will not necessarily need to rewrite the analysis.

The rest of this page covers:

- 1. [Documenting trajectory failures for Gemini 3.8 Flash](#)
- 2. [Helpful tips](#)
- 3. [Using the chat function to guide your failure analysis](#)

Step 1

Document Failures for Gemini 3.8 Flash

Documenting Trajectory Failures for Gemini 3.8 Flash

Add a failure analysis in the 'Additional Notes' box inside the trajectory view in RLS for **one Gemini 3.8 Flash trajectory** that was run on the final version of your prompt before submission for Prompt Review.

Use the following structure to write a short **one-paragraph** failure analysis for Gemini 3.8 Flash failure analysis:

- 1. Summary of the incorrect output values/claims (1 sentence). What was wrong with the model's **output?**
- 2. Identify up to three causes (e.g., evidence of poor domain reasoning, or poor interpretation of a file/instructions). For each, write 1-2 sentences describing:
 - a. Exact point where agent diverged from the correct path, including the step number(s).
 - b. What the agent should have done
- 3. State why the failures make the output unacceptable (1 sentence).

Example Failure Analysis

Gemini 3.8 Flash added one clause with incorrect language relating to amortization, and missed a definition for Service Credits entirely. The failures result in a provision that cannot be properly enforced and an overstated refund method:

	• In step 22, the agent chose a 5-year amortization period copied from the Snow MLSA. It did not recognize this is tied to a perpetual license term, and should be adapted to the three-year term of Meridian's agreement. • In step 23, the agent incorrectly decided against adding a definition for Service Credit: "I'm thinking about whether I can add the term 'Service Credit' without needing a separate definition. It might be acceptable to use it as is since it could be clear enough from context."	
Failure Analysis Tips	Helpful Tips 1. When you draft an annotation, use the <i>Cite Step</i> feature liberally. – Citing the specific reasoning step or tool call that went wrong is what turns a vague "the model got confused" into a defensible failure record. 2. Use the Chat feature to help you write your analysis. Check its claims carefully: you are responsible for the accuracy of final output. See below for suggested prompting.	

How to Use the Chat Function to Help Guide your Failure Analysis

1. Paste the prompt below into the Chat agent for the trajectory you want to analyze.
 - a. Insert a short summary of key correct outputs in the highlighted section to help the agent identify key differences to the final trajectory outputs.
2. Verify all failure claims manually against the trajectory steps and edit as needed.
 - a. Check that the failures are valid and result in incorrect outputs.
3. Make sure your final analysis aligns with the structure set out in Step 1 above.
 - a. The Chat agent can help you get started, but its output won't align perfectly with our guidance.

Remember!
You are responsible for your final failure analysis. The Chat agent can hallucinate, misinterpret the trajectory, or misjudge the correct outputs or approach to your task. Always verify all claims made by the agent and edit as necessary.

Chat Box Prompt

****Analyze this agent trajectory for failure.****

Review the task prompt and the attached trajectory (including reasoning traces, tool calls, and outputs). Write a short failure analysis in the format below.

****Focus only on genuine reasoning failures that affect the final output.**** Do not flag or comment on:

- Technical issues, system errors, or timeouts
- Inefficient workflows or unnecessary steps
- Style or formatting issues
- Anything that did not affect the final output

Only flag failures where the agent had the information it needed but reasoned incorrectly, misunderstood a constraint, used a tool wrongly, or drew a wrong conclusion — and where this directly caused an incorrect or incomplete final output.

[OPTIONAL — fill in with a brief summary of key correct outputs]:

The correct output should have been: _____

****Format your response as follows:****

****Failure category**:** [main category + sub-category]

1. Searching + Finding failure — 1a: no attempt; 1b: searched but found nothing; 1c: found and used wrong file
2. Tool use failure — 2a: wrong tool; 2b: correct tool, unresolved errors; 2c: correct tool, executed incorrectly
3. Planning and reflection failure — 3a: doom loop; 3b: gave up
4. Reasoning failure — 4a: numerical error; 4b: non-numerical/logical error; 4c: domain reasoning failure (critical misunderstanding of subject matter leading to a reasoning error — can be numeric or non-numeric)
5. Other — explain

****Analysis:****

- What the final output got wrong (one sentence)
- Proximate cause 1: step number(s), what the agent did wrong, and what specific action or reasoning it should have applied instead
- Proximate cause 2 (if applicable): same structure
- Proximate cause 3 (if applicable): same structure
- Why these failures make the output unacceptable (one sentence)

For each proximate cause, identify the specific alternative action or reasoning the agent should have applied — not "it should have verified" or "it should have checked", but the concrete thing it should have done: e.g., "it should have read clause X as applying only to Y" or "it should have used field Z to calculate W rather than relying on field V."

Be precise and concise. Use exact step numbers and quote reasoning traces where they reveal the mistake directly.

3 Rubric Creation

Rubric Creation

What is a rubric?

A rubric is a set of small, gradable checks called verifiers, each anchored to a value or claim in your golden output.

Note: your rubric + golden creation will likely be a combined, iterative process and not two truly distinct steps

Example rubric

See below an extract from a task rubric in the FP&A domain. *Note: Only three verifiers are included for display purposes. See Example Tasks section for full task examples including rubrics.*

Index	Verifier Type	Criteria	Weight	Grading target
1	Model Final Output	States the FY25 total firm compensation expense is \$8,715,214.	8	Spreadsheets (.xlsx, .xls, .xlsm, .ods)
2	Model Final Output	States the IMA 30% principal share is \$1,342,825.	5	Spreadsheets (.xlsx, .xls, .xlsm, .ods)
3	Model Final Output	Does not state that the IMA 30% principal share is an external counterparty payout.	2	Spreadsheets (.xlsx, .xls, .xlsm, .ods)
...	...	...	...	...

Step 1

General Rubric Guidance

General Rubric Writing Guidance

There is no set cap on criteria per task. Create as many criteria as necessary to evaluate the task, but only add necessary criteria.

Tasks should have at least 5 verifiers. For any tasks resulting in fewer than 5 verifiable outputs, ensure that the output request captures the full set of values and claims which would be important for a real-world output to the task.

Not all fields within the verifier are required. Required field are:

1. **Verifier Type**
2. **Criteria**
3. **Criteria Explanation**
4. **Numerical Weight**
5. **Reference Artifacts**
6. **Grading Target**
7. **Depends on (when applicable)**
8. **Output Dependencies (when the verifier grades a file)**

Output Correctness Rewards

Rubric verifiers evaluate **output correctness**, i.e., they award points for the model putting the correct information or values in the final output. They should cover everything important to a good response:

- **Excel outputs.** Correct values and short supporting claims or conclusions.
- **Document outputs** (Law only). Correct wording and legal requirements.
- **App outputs.** Use App DB Programmatic verifiers to check the records the model creates, updates, or deletes.

✓ What Output Correctness Should Cover

Right values in the final output, right claims in the conclusion, right wording and legal requirements in the contract. Anchor every reward to a claim or value present in the golden output.

✗ What It Should Not Cover

Style, format, or presentation (covered by separate style rewards).
The steps the agent takes to reach the answer.

How to Design Verifiers

- Start from your golden output. Everything in the golden output should be values and claims you consider important in a real-world response.
- Do not add anything you would not put in a real slide deck, spreadsheet, or document, and do not exclude an important output that you would expect to see there.
- You can write a verifier for any value or claim that should appear in the output, including intermediate values you would expect to see presented.

- Each value or claim should be weighted per the Weighting Rewards rules.

Negative Criteria

Negative "Does not" criteria, which penalize the agent for doing something it shouldn't, can be used sparingly. They should focus only on specific incorrect outputs that (1) would make the response very bad if included, and (2) are actually likely to arise (for example, due to a red herring in the prompt). The same rules as for all verifiers apply, and most should take the form "Does not state...".

Step 2

Rubric Requirements

Rubric Requirements

1. Rubric covers golden (Spec 4B)

- All important values and claims in the golden output have verifiers. A model that gets a key figure or claim wrong should not score 100%.

2. Rubric traces to prompt (Spec 4C)

- Every verifier traces to something the prompt asked for — directly or through referenced files. If the golden or verifier includes unrequested content, add the requirement to the prompt or remove it from both.

✗
Golden includes a hiring recommendation; prompt only asked for cost analysis. Verifier tests the recommendation.

✓
Add "include a hiring recommendation" to the prompt, or remove the verifier and the recommendation from the golden.

The Anchor Rule

If a value matters it is: (1) requested in the prompt, (2) present in the golden, (3) tested in the rubric.

Walk the chain both ways. Prompt → Golden → Rubric → Prompt.

Step 3

Verifier-level requirements

Verifier-level requirements (Spec 4A)

All verifiers in your task must meet the following six requirements.

1. Specific and gradeable

- **Binary yes/no.** Any expert, looking only at the output, says 'met' or 'not met.' No middle ground.

- Test for a specific and verifiable claim, not just an easy test for the presence of an output element.
- Avoid subjective language such as “thorough” analysis or “convincing” recommendation.

✗
"Includes an appropriate conclusion in the memo." — What is the topic of the conclusion? What is appropriate?

✓
"States in the conclusion that the reclassification reduces EBITDA by \$1.2M."

2. Self-contained

- Everything needed to grade is inside the criterion. No references to files, other verifiers, or the prompt.
- **Timeless:** no ‘current’ or ‘latest’ without a fixed anchor. Follow the same date conventions as for prompting.

✗
"Accurately cites the Q2 budget."
— How does the grader know what's contained in the Q2 budget?

✓
"States the Q2 budget is \$425,000."

3. Atomic

One verifiable fact per criterion. Each fact once, in one verifier.

- **Not stacked:** bundling hides which part failed and prevents partial credit.
- **Not duplicative:** no two verifiers test the same thing, even partially.

Exception

Low-weight verifiers may group closely related values if the model would very likely get all correct or all wrong at the same time (i.e., they require very similar reasoning steps).

✗
"States \$50K min ACV, 30% max discount, and CMO sign-off at 26–30%."

✓
Three separate verifiers: ACV floor, discount cap, approval threshold.

Law domain atomicity exception

In cases where legal reasoning flows through multiple verifiable facts **and** failing any one of those criteria means all fail, we accept “stacked” criteria.

Verifiers themselves must still have a binary yes/no verdict. Any stacking that leaves room for possible partial credit will not be approved.

4. Output-focused

- **Test values and claims visible in the final output** — not the model's process, methodology, or data sources. We test for content only, and do not test for style or formatting.
- App data verifiers use CUD language ("Creates", "Updates", "Deletes") and legal tasks use Inserts/Deletes/Edits for redlining verifiers.

Use "States" to begin most criteria

This forces you to be very explicit about the required output. Other verbs are permitted in limited cases where "States" does not make sense.

✗

"Uses ERPNext Employee records."
/ "Displays formulas."

✓

"States that the total headcount is 247." / "States that the total reserve is \$22,391.66."

5. Precise

- State the exact value or key claim elements from your golden.
 - For FP&A and Accounting only we allow verifiers with ranges.
- **Tip:** set units and rounding in the prompt. You can include 'Do not round intermediate values' to enforce deterministic answers.
- Language-based verifiers should test for important and meaningful elements of the text while not constraining to inconsequential personal drafting choices (e.g., using "language substantially similar to").

✗

"...approximately \$287,717 (range \$284K–\$294K)."

✓

"...is \$287,717."

✗

"Inserts at the end of Section 3.3(b)(i) of the SPA using redline the following exact text: *"A client shall be included on the Baseline Client List only if such client has an active engagement with the Company as of the Closing Date, as evidenced by an open contract, work order, or master services arrangement"*" —Enforces inconsequential drafting choices by testing for verbatim text

✓

"Inserts at the end of Section 3.3(b)(i) of the SPA using redline **text substantially similar to the following:** *"A client shall be included on the Baseline Client List only if such client has an active engagement with the Company as of the Closing Date, as evidenced by an open contract, work order, or master services arrangement"* **that includes the exact phrase "active engagement with the Company as of the Closing Date".** —Separates text which must appear verbatim from drafting choices.

Acceptable ranges in FP&A and Accounting:

Use ranges for calculation rules, and define the calculation rules in the prompt as tightly and explicitly as you can. The range is a safety margin, not a substitute for a clear rule.

- Ranges shouldn't be so broad that a wrong method still passes — but they should cover cases where more than one answer is reasonably supportable
- If a model answer misses by a trivial amount (say \$1), treat that as a sign the range needs widening, not as a true failure

Bad example: "States that the unreconciled membership-revenue amount flagged for management is \$213,223.12 (accept $\pm$ \$0.10)."

Good example: "States that the unreconciled membership-revenue amount flagged for management is \$213,223.12 (acceptable range is \$213,223.02 to \$213,223.22)"

6. Weighted

Every verifier carries a numerical weight from 1-10 reflecting how important the criterion is to a correct result. Weight 10 is reserved for gate criteria - the 'wrong here = the task fails' checks. See the Weighting guidance for the scale and gate rules.

Step 4

Configuring verifiers

Configuring the verifier correctly

Choosing a Verifier Type

Choose the verifier type that matches what you are checking. RL Studio offers three types (see the Verifier Types guidance for full detail):

Model Final Output - an LLM judge reads the model's final output or deliverable and checks the criterion. Use for conclusions, recommendations, and other deliverable content.

Trajectory - checks that the model performed a specific action or tool call, for example sending an email to the right recipient or opening a required file. Use this for any email-related check.

App DB Programmatic - a code-based check that the model made the correct change in an app's database, or created a required artifact.

Selecting the Grading Target in Studio

Set the grading target to what the grader should read. For most Model Final Output verifiers, the All Output option is appropriate. See the Verifier Types guidance for how grading targets work with each verifier type.

Dependencies

Julius does not use the Dependencies field for criterion, but it still appears in the UI. You do not need to fill them in.

Output Dependencies

Not the same field as Dependencies above. Output Dependencies tells the grader which file the criterion is graded against, and Julius does use it — fill it in on every verifier that grades a file.

The value is the task's expected file output: the exact file name you entered in prompt section 1.4) Expected Output Files. Nothing else belongs in this field.

- Output Dependencies = Expected Output Files. Same file name, same spelling, same extension as 1.4).
- Every verifier that grades a file carries the same value. A task with one deliverable lists that one file on all of its file-graded verifiers.
- If the task has more than one deliverable, list each file the verifier is actually graded against.
- File names must have no spaces and must include the filetype — Final_Legal_Memo.docx, not Final Legal Memo. The field will not accept anything else.
- Leave it empty only where the verifier grades no file: rows set to Final Answer Only (No Files), Trajectory verifiers checking an action or an email, and App DB Programmatic verifiers checking app state rather than a created artifact.
- If the picker comes up empty, 1.4) Expected Output Files has not been filled in. Fix 1.4 first, then come back to the rubric.

Editing Output Dependencies: Spreadsheet vs Structured view

The Rubrics UI opens in Spreadsheet view by default and Output Dependencies is read-only there, which is why it is often left blank. To fill it in, switch the Rubrics view from Spreadsheet to Structured, click the verifier to open it, and edit Output Dependencies in the expanded verifier. Switch back to Spreadsheet afterwards if you prefer.

Walkthrough (11 seconds): [editing Output Dependencies in Structured view](#)

If the field is still not editable after switching to Structured, the rubric is locked at your current task status — flag it to your reviewer instead of leaving it blank.

Step 5

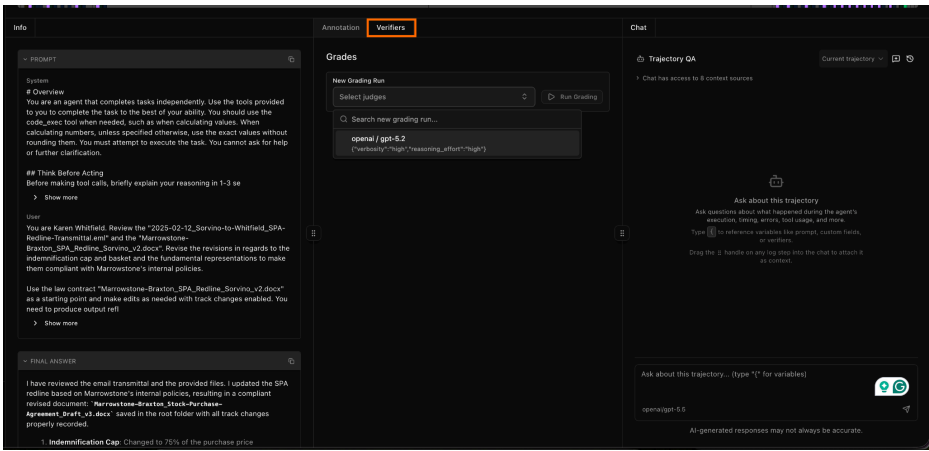
Running Verifiers against
your Criteria

Running a Grader Against a Response

We use an LLM grader to score each model's attempt to solve your prompt against your rubric. The grader should be run on all completed trajectories.

To run gradings, open the Trajectory section and navigate to the Verifiers tab. Use the drop-down under 'New Grading Run' to select the Gemini 3 Flash judge model and click 'Run Grading'. Within a few minutes, the scores will appear below in the same section, and a percentage score will show next to each trajectory in the main task view.

Review the scores and flag to your reviewer if you consider them to be inaccurate. **Confirm that your task meets the project's failure requirements before submitting.**



Please note that screenshots might differ from the actual view. Access the RLS to get the up-to-date view.

Verifier Types

Verifier Types

This is the most important section of this guide. Each verifier type serves a different purpose and checks a different aspect of the model's execution. Using the wrong verifier type for a criterion makes the task flawed and the evaluation unreliable.

Every criterion must use the verifier type that matches what it is actually checking.

Model Final Output Verifier

- What it checks. The model's final textual output or deliverable file content.
- When to use. Checking conclusions, final recommendations, or any content the model is expected to produce as part of its final answer or deliverable.
- How it works. An LLM judge reads the model's output and evaluates whether the criterion is met.

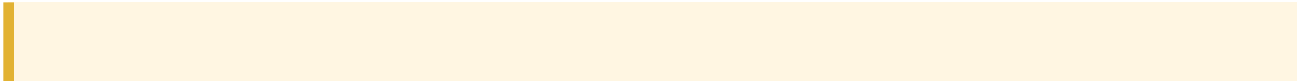
Required Fields

Field	Description	Required
Criteria	A clear, specific statement that is either true or false of the model's output.	Yes
Criteria Explanation	Why this output is expected.	Yes
Dependency	The criterias that this criteria is dependent on.	Yes
Grading Target	What the grader should look at. Typically the All Output option is correct.	Yes
Criterion Type	Objective Compliance, Expert Assessment, or Process.	Yes
Numerical Weight	Integer from 1 to 10.	Yes
Reference Artifacts	The artifacts required to evaluate the criteria.	Yes
Is Primary Objective	Whether this is a core task	Yes

	outcome.	
--	----------	--

Bad Criteria Examples

- Provides a thorough analysis of the data. Subjective; what counts as thorough?
- Reads the pricing spreadsheet. This is a process check, not an output check. Use Trajectory verifier.
- Mentions revenue. Too vague. Mentions it where? What value? In what context?



Trajectory Verifier

- What it checks. Whether the model made a specific tool call with specific arguments.
- When to use. Checking that the model performed a required action: opened a specific document, called a specific API, sent an email to the right recipient, read the file that contains the password.
Additionally, *anything email-output related* should be checked with the Trajectory Verifier, even if it is a deliverable, due to verifier limitations.
- How it works. Extracts all tool calls and outputs from the trajectory, passes them to an LLM judge along with the criterion.

Required Fields

Field	Description	Required
Criterion	Natural language description of expected tool behavior.	Yes
Numerical Weight	Integer from 1 to 10. Must align with severity level.	Yes
Criteria Explanation	Why this behavior is expected.	Yes
include_tool_outputs	Include the tool response data in evaluation context.	No (default setting is True)
include_messages	Include conversation history in evaluation context. This should always be marked True.	Yes (default setting is False so you need to mark this as True)

Criterion Type	Objective Compliance, Expert Assessment, or Process.	Yes
----------------	--	-----

Examples

- PASS if the email “To:” field is procurement@acme.com. FAIL otherwise.
- PASS if the agent opened the ER-200_protected.pdf file at any point in the trajectory. FAIL otherwise.

DB Diff Verifiers

What are they?

DB Diff verifiers are used when the task requires the agent to make changes to data stored in the app. This could include asking the agent to input a journal entry, add a customer, or other

What do they do?

The verifier compares the before and after state of the database to determine if the changes were made correctly.

So → 1) the app starts in its initial state, 2) the agent performs actions within the database during the task, 3) the DB Diff verifiers check that the final state of the app is correct.

What do they need to have?

DB Diff verifiers need to have a set of criteria that clearly describes the expected app changes. They should specify the following.

- Which records should be created, updated, or deleted.
- What fields or values are expected to change.
- Any constraints or conditions that must be satisfied.

The verifier then evaluates the app diff against these criteria to determine whether the task was completed successfully.

Verifier Language Guidelines

DB Diff verifiers should follow **CUD** language - Create, Update, Delete, etc.

Creates a Journal Entry of [...]

Update credit note to [...]

Delete debit notice for [...]

Clarification of create vs. update: Create is a net new row, updating means the value existed but has not been changed.

Note: DB Diff verifiers cannot have negative rubrics (ex. Does not create [...])

How does this affect your golden response?

When a prompt requires the agent to make changes within an app, your golden response must include you going into that app and making those changes yourself.

In practice this means:

- **Go into the app and perform every change yourself.** When the prompt requires the agent to create, update, or delete data, your golden response must walk into that app and carry out each of those operations, exactly as a successful agent would.
- **Your golden response defines the expected diff.** The end state you produce *is* the answer key. The changes you make should line up one-to-one with the criteria in your verifier.
- **Don't introduce changes outside the criteria.** Avoid incidental edits or side effects (extra saves, status flips, auto-populated fields you didn't intend). A stray write can show up in the diff and either contradict your own criteria or set an end state the agent can't reasonably reproduce.
- **Start from the same initial state the agent will.** The diff is measured from the task's starting state, so make your changes from that same baseline.

Weighting Rewards

Criteria Weighting

Every criterion is assigned a numerical weight. These determine how much each criterion contributes to the overall score. **Weighting is based on criteria importance, not just difficulty.**

Weight	Meaning	Calibration question
9–10	Critical. This is the core or central crux of the task, and hence the most critical.	"Is this the item a senior reviewer would check first?"
7–8	High-stakes. Getting this wrong means the work product is materially incorrect.	"Would this error cause a real-world consequence, such as financial, legal, or safety related?"
5–6	Core. This is part of what the task is fundamentally asking for.	"Would missing this make the deliverable incomplete?"
3–4	Supporting. Contributes to quality but isn't the core deliverable.	"Would missing this cause a revision request?"
1–2	Cosmetic / structural. Getting this wrong is sloppy but doesn't change the conclusion.	"Would a manager notice this in a 30-second skim?"

4 Golden Trajectory

For a complete walkthrough, please see [this loom video](#).

Step 1

Capture the Trajectory in the Playground

What a Golden Trajectory Is

A golden trajectory is the recorded path from the prompt to a 100% rubric score. It proves the task is solvable and becomes the ceiling every model run is graded against. Instead of hand-building a perfect output file, you guide an agent through the task in the Playground and capture the run that gets there — faster and more hands-off, while still producing one perfect, client-ready answer.

Start a Recorded Playground

From your task, start a new Playground; it takes ~30 seconds to 3 minutes to provision, so kick it off early. Enter it once ready — the session is recorded, which is how we capture the path, instructions, and tool calls the agent used. Inside you have the same data and apps the agent does (open PDFs, send emails, inspect files), but you'll mostly work through the chat.

Guide the Agent to the Answer

Paste the task's original prompt into the chat and send it. A well-calibrated task won't be solved one-shot — that's expected. Work back and forth with the agent until it reaches the correct answer in your domain's expected output format. Keep your guidance minimal and on-task: everything you type is recorded into the trajectory.

Hint Guidance

A hint may point the model toward what the model did incorrectly (e.g. "You've identified the wrong document", "made the wrong assertion", etc.) However, it should not provide the precise calculation, document name, figure itself, step-wise instructions, or intermediate answer needed to reach the final answer.

Overarching principle: A hint should never be such that the Agent could reasonably skip the tool calls it needs to get the answer

Step 2 Grade to 100% Against the Rubric	Capture and Run Grading When the agent has reached the answer, click Capture the Golden Trajectory, select the grader, and click Run Grading (refresh to confirm grading is in progress). 100% Is the Bar A trajectory only qualifies as golden at a 100% rubric score. On larger rubrics your first capture may land at 60% or 90% — keep iterating and re-capturing until you hit 100%. You can capture as many attempts as you like, but you cannot submit a golden trajectory below 100%. At 100% the run shows Validated. Keep Prompt, Golden, and Rubric Anchored The run's final output is your golden answer, so the anchor rule holds: anything in a strong answer should be (1) requested in the prompt, (2) present in the agent's output, and (3) tested in the rubric — with nothing the prompt didn't ask for. If the agent genuinely can't reach 100%, the gap is usually in the prompt, golden, or rubric — not the agent.	
Step 3 Set Golden Status & Make it the Ceiling	Save and Mark It Golden Once a run is Validated at 100%, end the session — this saves the recording and creates a snapshot. Then click Set Golden Status to label that session golden. Keep the Most Efficient Run You can have multiple golden sessions; keep the best one. "Best" is the most efficient path — the agent stayed on track and you added no stray instructions. (Chat like "double-check your answer" is logged into the trajectory, so a clean run that solves it directly beats a messy one — re-record if needed.) Re-grading a session to log its score is optional. Make It the Ceiling The golden trajectory is what every model run is graded against, so it must be the strongest possible run. If a later run ever beats it on any dimension, improve the golden. You're done when the session shows Golden status back in the Playground — then move on to your next task.	



Law Golden Trajectory Example

Golden Trajectory Transcript

Contract_Law16_AK_002

USER

Revise the March 1, 2026 Master Services Agreement to address the remaining issues concerning the liability cap, breach notification, and indemnification provisions. In preparing your redline, review the transaction record as of March 15, 2026, including the attached Strawberry Final Approvals email from David Whitaker (HolisticStar's CEO), negotiation memos, board materials, and meeting minutes. For each of the three issues, insert a comment explaining your rationale for revising the provision or, if applicable, for leaving it unchanged. Do not make any changes unrelated to those three issues.

Return the document with the same filename as the original template. Make all edits with tracked changes enabled and return the document with redline edits visible. Produce output reflecting professional legal standards.

Input Files

The following files have been provided as context for this task:

Strawberry Final Approvals.pdf

Date: Sat, 14 Mar 2026 09:18:41

From: David Whitaker <dwhitaker@holisticstarhealth.com>

To: Christopher Hayes <chayes@holisticstarhealth.com>, Monica Reynolds <mreynolds@holisticstarhealth.com>, Brandon Cole <bcole@holisticstarhealth.com>, Natalie Price <nprice@holisticstarhealth.com>, Jason Mitchell <jmitchell@holisticstarhealth.com>, Tiffany Brooks <tbrooks@holisticstarhealth.com>, Angela Foster <afoster@holisticstarhealth.com>

Subject: Approval — Revised Strawberry MSA Negotiation Positions

Team,

Following my discussion with Laura McBride earlier today, I am approving three targeted revisions to HolisticStar's negotiating position for the Strawberry Master Services Agreement.

First, the company's approved limitation-of-liability position is revised from an aggregate cap of \$5,000,000 to \$7,000,000. This adjustment addresses Strawberry's remaining concerns regarding enterprise risk allocation while maintaining an acceptable level of protection for HolisticStar.

Second, the approved Security Incident notification commitment is revised from seventy-two (72) hours following confirmation of a reportable Security Incident to forty-eight (48) hours. Tiffany has confirmed that the revised commitment can be operationally supported.

Third, we've agreed to revise Section 11.3(a) to replace the current exception for Article 12

indemnification obligations with a provision limiting each Party's aggregate liability for its Article 12 indemnification obligations to two (2) times the liability cap in Section 11.2. This change appropriately limits indemnification exposure while preserving meaningful protection for both parties.

Please treat this email as the controlling internal approval for these revisions. These changes should be reflected in the next draft of the MSA and all related transaction documents.

David

—

David Whitaker

Chief Executive Officer

HolisticStar Health Analytics, Inc.

1100 Demonbreun Street, Suite 800

Nashville, TN 37203

+1 (615) 555-0150

dwhitaker@holisticstarhealth.com

ASSISTANT

→ **ToolSearch** · select:mcp__playground__filesystem_list_files,mcp__playground__filesystem_search_files,mcp ...[+169 chars]
→ **mcp__playground__filesystem_list_files** · /
→ **mcp__playground__filesystem_list_files** · /contracts
→ **mcp__playground__filesystem_list_files** · /legal
→ **mcp__playground__filesystem_list_files** · /meetings
→ **mcp__playground__filesystem_list_files** · /email
→ **mcp__playground__word_v2_get_document_overview** · /contracts/Master_Services_Agreement_Draft.docx
→ **mcp__playground__word_v2_read_document_content** · /contracts/Master_Services_Agreement_Draft.docx
→ **Read** · /root/.claude/projects/-scratch-prompt/ee132660-bf58-4817-8157-8fe32a1dc0ad/tool-results/t ...[+33 chars]
→ **Read** · /root/.claude/projects/-scratch-prompt/ee132660-bf58-4817-8157-8fe32a1dc0ad/tool-results/t ...[+33 chars]
→ **ToolSearch** · select:TodoWrite
→ **TodoWrite**
→ **Grep** · /root/.claude/projects/-scratch-prompt/ee132660-bf58-4817-8157-8fe32a1dc0ad/tool-results/t ...[+33 chars]
→ **mcp__playground__word_v2_read_document_content** · /contracts/Master_Services_Agreement_Draft.docx
→ **mcp__playground__word_v2_read_document_content** · /contracts/Master_Services_Agreement_Draft.docx
→ **mcp__playground__word_v2_read_document_content** · /contracts/Master_Services_Agreement_Draft.docx
I have the critical provisions. Let me check Article 12 (indemnification) and confirm the structure of 11.3.
→ **mcp__playground__word_v2_read_document_content** · /contracts/Master_Services_Agreement_Draft.docx
I have all the key provisions. Now let me quickly review the supporting materials for context on the rationale, then explore what tools are available for tracked-change edits and comments.
→ **ToolSearch** ·

select:mcp__playground__word_v2_edit_content_text,mcp__playground__word_v2_insert_content, ...[+242 chars]

Let me quickly review the negotiation memo and steering committee minutes for context to ground the rationale comments.

→ **mcp__playground__word_v2_read_document_content** · /legal/Negotiation_Strategy_Memo.docx

→ **TodoWrite**

→ **mcp__playground__word_v2_read_document_content** ·
/meetings/Executive_Steering_Committee_Minutes.docx

I have the full record. Now let me check for a shell tool to properly inject Word tracked-change XML (the native Word tools don't support revisions).

→ **ToolSearch** · select:mcp__playground__stirrup_code_execution_run_shell,mcp__playground__filesystem_get_f ...[+12 chars]

I have a shell tool. Let me build a Python script that will make proper tracked-change edits (w:ins and w:del) and add Word comments to the MSA. First, let me confirm the file locations and inspect the docx internals.

→ **mcp__playground__stirrup_code_execution_run_shell** · python3 -c "import docx; print(docx.__version__)" 2>&1; ls /contracts/ 2>&1 | head

→ **mcp__playground__stirrup_code_execution_run_shell** · pwd && ls && find . -maxdepth 3 -name "Master_Services_Agreement*" 2>/dev/null

→ **mcp__playground__stirrup_code_execution_run_shell** · cp contracts/Master_Services_Agreement_Draft.docx /tmp/msa_backup.docx && mkdir -p /tmp/do ...[+83 chars]

→ **mcp__playground__stirrup_code_execution_run_shell** · cp contracts/Master_Services_Agreement_Draft.docx /tmp/msa_backup.docx && python3 -c " imp ...[+77 chars]

→ **mcp__playground__stirrup_code_execution_run_shell** · python3 << 'PY' import zipfile, re with zipfile.ZipFile('/tmp/msa_backup.docx') as z: xml ...[+415 chars]

Excellent — I have python-docx and full access. Now I'll write the redline script that injects proper OOXML tracked-changes (w:ins/w:del) and Word comments.

→ **TodoWrite**

→ **mcp__playground__stirrup_code_execution_run_shell** · python3 << 'PY' import zipfile, re with zipfile.ZipFile('/tmp/msa_backup.docx') as z: xml ...[+303 chars]

→ **mcp__playground__stirrup_code_execution_run_shell** · python3 << 'PY' import zipfile, re with zipfile.ZipFile('/tmp/msa_backup.docx') as z: xml ...[+376 chars]

Now I'll build and run the full redline script.

→ **mcp__playground__stirrup_code_execution_run_shell** · cat > /tmp/redline.py << 'PY' """ Apply tracked-change redlines and comments to Master_Ser ...[+14436 chars]

Redline applied. Let me verify the tracked changes and comments rendered correctly.

→ **mcp__playground__stirrup_code_execution_run_shell** · python3 << 'PY' import zipfile, re z = zipfile.ZipFile('contracts/Master_Services_Agreemen ...[+1239 chars]

Let me do a final validation to confirm the file opens cleanly and the edits appear as intended.

→ **mcp__playground__stirrup_code_execution_run_shell** · python3 << 'PY' from docx import Document d = Document('contracts/Master_Services_Agreemen ...[+336 chars]

There's an unescaped & in my comment text ("Errors & Omissions"). Let me fix by escaping.

→ **mcp__playground__stirrup_code_execution_run_shell** · python3 << 'PY' import zipfile z = zipfile.ZipFile('contracts/Master_Services_Agreement_Dr ...[+204 chars]

Confirmed — the bare & in "Errors & Omissions" broke XML parsing. Let me patch comments.xml in place with proper escaping.

→ `mcp_playground_stirrup_code_execution_run_shell` · python3 << 'PY' import zipfile, shutil, os SRC = 'contracts/Master_Services_Agreement_Draf ...[+700 chars]

→ `mcp_playground_stirrup_code_execution_run_shell` · python3 << 'PY' from docx import Document d = Document('contracts/Master_Services_Agreemen ...[+420 chars]

The blank gaps are expected — python-docx.text skips <w:ins>/<w:del> runs by design. Let me verify at the XML level that the redlines are intact.

→ `mcp_playground_stirrup_code_execution_run_shell` · python3 << 'PY' import zipfile, re z = zipfile.ZipFile('contracts/Master_Services_Agreemen ...[+850 chars]

All four tracked-change pairs and three comments verified. Let me finalize the todo list and confirm.

→ **TodoWrite**

Redline complete. Returned in place at /contracts/Master_Services_Agreement_Draft.docx (original filename preserved).

Summary of tracked-change edits (author: Monica Reynolds, General Counsel; date: 2026-03-15):

#	Section	Deletion	Insertion
1	§ 11.2 Cap on Liability	FIVE MILLION UNITED STATES DOLLARS (\$5,000,000)	SEVEN MILLION UNITED STATES DOLLARS (\$7,000,000)
2	§ 9.1 Notification of Security Incidents	seventy-two (72) hours	forty-eight (48) hours
2a	§ 9.6 Operational Alignment (conforming)	seventy-two (72) hour	forty-eight (48) hour
3	§ 11.3(a) Exceptions	a Party's indemnification obligations under Article 12	each Party's indemnification obligations under Article 12, provided that each Party's aggregate liability for its indemnification obligations under Article 12 shall not exceed two (2) times the amount of the cap set forth in Section 11.2

Three anchored comments attached to §§ 9.1, 11.2, and 11.3, each explaining the rationale by reference to Whitaker's March 14, 2026 approval email, the March 5 Executive Steering Committee resolutions, the Cedarline precedent, the Westbrook coverage tower, and the CISO's operational confirmation. <w:trackChanges/> is enabled in word/settings.xml so any downstream edits continue as tracked.

USER

Don't assume the latest email controls everything. The March 14 Final Approvals email is not the

only document that matters. Before making any changes, compare the email against the earlier governance documents to make sure each requested change was actually approved.

ASSISTANT

You're right — I moved too fast. Let me pull the earlier governance record and check whether each of the three changes in the March 14 email actually had proper authorization behind it.

→ `mcp__playground__word_v2_read_document_content` · /meetings/Board_Governance_Briefing.docx

This is important. The Board Briefing dated **March 13, 2026** (one day before the "Final Approvals" email) tells the Board that management is asking them to CONFIRM the \$5M cap and 72-hour notification. And the March 5 committee resolutions bound the cap movement to require Board consultation. Let me read the outside counsel memo, both other emails, and the negotiation thread to complete the picture.

→ `mcp__playground__filesystem_read_text_file` · /email/Executive_Approval_Email.eml

→ `mcp__playground__stirrup_code_execution_run_shell` · for f in email/*.eml; do echo "===== " ; echo "FILE: \$f"; e ...[+66 chars]

You're right — this is a serious governance issue. Let me lay out what the earlier record shows.

Prior authorizations for the three items:

1. **March 5 Executive Steering Committee resolution D-March 5, 2026-01** (unanimous): Hold cap at **\$5,000,000**. *"Any movement above \$5,000,000 requires re-convening this committee **and consultation with the Board.**"*
2. **March 5 Committee resolution D-March 5, 2026-02**: Hold notification at **72 hours**. *"Any movement off the 72-hour position requires re-convening this committee."*
3. **March 10, 2026 Whitaker "Final Negotiation Posture" email** (Whitaker's own earlier controlling record): Approves **\$5M cap** and **72-hour** window as final; Condition (c): *"No material change to the \$5,000,000 cap or the 72-hour notification window is to be entertained without my written approval **and a fresh review by Christopher and Tiffany.**"* Also expressly delegates indemnification/IP/audit-rights authority to Monica Reynolds within the negotiation strategy memo envelope, *"provided no movement is made on the liability cap or the breach-notification window."*
4. **March 13, 2026 Board Governance Briefing** (Whitaker to Board): Represents to the Board that management will request the Board to *confirm* the \$5M cap and 72-hour window on June 25.
5. **March 14, 2026 "Final Approvals" email**: 24 hours after the Board Briefing. Cites only a discussion with Laura McBride (Strawberry Procurement) and Brooks's operational confirmation. **No re-convened committee. No Board consultation. No Hayes (CFO) fresh review. No amendment to the pre-read the Board just received.**

My assessment against each of the three requested changes:

Change	Prior authorization	Sufficient?
Cap \$5M → \$7M	Requires (a) re-convened Steering Committee, (b) Board consultation, (c) Hayes + Brooks fresh review per Whitaker's own March 10 conditions. March 14 email evidences none of these; further, Board has been told 24 hours earlier that \$5M will be presented for confirmation.	No — should NOT be made
72h → 48h	Requires re-convened Steering Committee (Mar 5 resolution) plus Hayes + Brooks fresh review (Mar 10). Committee not re-convened; only Brooks referenced.	No — should NOT be made
§11.3(a) — 2× sub-cap for Art. 12 indemnification	Falls within GC's standing delegated authority (Mar 10 email) <i>and</i> directly resolves counsel's identified walk-away threshold in the Feb 27 Negotiation Strategy Memo ("a clearly bounded multiple of it").	Yes — should be made

I need to revert the cap and notification tracked changes, keep the indemnification tracked change, and rewrite the comments to explain each disposition (including two "leaving unchanged" rationales).

→ **TodoWrite**

→ `mcp_playground_stirrup_code_execution_run_shell` · `ls -la /tmp/msa_backup.docx && python3 -c "from docx import Document; d = Document('/tmp/m...[+217 chars]`

Clean backup verified. Now rebuilding with only the one authorized tracked change and three fresh rationale comments.

→ `mcp_playground_stirrup_code_execution_run_shell` · `cat > /tmp/redline2.py << 'PY' """ Rebuild redline from clean backup. Governance-driven di...[+12815 chars]`

→ `mcp_playground_stirrup_code_execution_run_shell` · `python3 << 'PY' import zipfile, re; z = zipfile.ZipFile('contracts/Master_Services_Agreemen...[+1214 chars]`

→ **TodoWrite**

Revised redline saved to /contracts/Master_Services_Agreement_Draft.docx (original filename preserved).

Revised dispositions after checking the March 14 email against the earlier record:

Provision	Disposition	Reason
§ 11.2 — \$5M cap	Left unchanged ; anchored comment explains the change was not made	Steering Committee resolution D-March 5-01 requires committee re-convening and Board

		consultation for any move above \$5M; the CEO's own March 10 email requires his written approval plus fresh CFO + CISO review; the March 13 Board Briefing already asked the Board to confirm \$5M on June 25. March 14 email evidences none of the required steps.
§ 9.1 / § 9.6 — 72-hour notification	Left unchanged ; anchored comment explains the change was not made	Committee resolution D-March 5-02 requires committee re-convening; March 10 email requires CFO + CISO fresh review (only CISO documented); Board briefed on 72-hour posture the day before the reversal.
§ 11.3(a) — 2× sub-cap for Art. 12 indemnification	Tracked change made	Falls squarely within GC's standing delegated authority under the March 10 Whitaker email and directly implements the "clearly bounded multiple" prescription in the Feb 27 Negotiation Strategy Memo. Authorized independently of the March 14 email.

The comments include express guidance that if the CEO wants to proceed with the cap and notification changes, GC will circulate a revised draft once the missing governance steps (committee re-convening, CFO review, and — for the cap — Board consultation) are documented on the record.

USER

How many dispositions did you make?

ASSISTANT

Three dispositions — one for each of the three issues in the task:

1. **§ 11.2 (liability cap)** — left unchanged, with a comment explaining why the March 14 email did not satisfy the governance chain.
2. **§ 9.1 / § 9.6 (breach notification)** — left unchanged, with a comment explaining why.
3. **§ 11.3(a) (indemnification)** — revised via tracked change to add the 2× sub-cap, with a comment explaining the independent authorization basis.

Two "leave unchanged" dispositions and one tracked-change disposition, each with its own anchored comment.

USER

Why did you combine these two § 9.1 / § 9.6 (breach notification) into one

ASSISTANT

Because § 9.6 is not a separate obligation — it just restates the same commitment expressed in § 9.1. Its opening clause is *"HolisticStar represents that the seventy-two (72) hour notification commitment is supported by..."* — it references the number rather than setting a new one.

That means the two provisions rise and fall together: change § 9.1 and § 9.6 has to be conformed; leave § 9.1 alone and § 9.6 requires no edit. There isn't a governance question about § 9.6 independent of the § 9.1 question. So I treated the "breach notification" issue as a single disposition covering both.

The user's task framed three issues (liability cap, breach notification, indemnification), and my three dispositions map one-to-one against those. That said, I only anchored the comment to § 9.1. If you'd like § 9.6 to carry its own anchored comment (even if the reasoning is derivative), I can add one — say the word.

USER

Isn't 9.6 a downstream or conforming edit?

ASSISTANT

Yes — you're right. § 9.6 is purely a conforming provision. It only ever becomes an edit *if* § 9.1 changes. Since I'm leaving § 9.1 at 72 hours, § 9.6 needs no work at all — it's not part of the disposition, it's a non-event.

The cleaner framing: **three dispositions**, one per issue —

1. § 11.2 (cap) — left unchanged, comment anchored.
2. § 9.1 (notification) — left unchanged, comment anchored. § 9.6 is untouched because there's nothing to conform.
3. § 11.3(a) (indemnification) — tracked change made, comment anchored.

I shouldn't have listed § 9.6 alongside § 9.1 in the disposition table — it implied I did something with § 9.6 when in fact the document is untouched there.

USER

So then did you make 3 or 4 decisions?

ASSISTANT

Three decisions — one per issue.

I made three substantive dispositions (cap: leave; notification: leave; indemnification: change). § 9.6 wasn't a fourth decision — it was a consequence of the notification decision. Because I left § 9.1 unchanged, § 9.6 needed no conforming edit, so no decision about it was required.

I muddled that earlier by listing "§ 9.1 / § 9.6" in the table as if § 9.6 were part of the disposition. It wasn't. Three decisions, three anchored comments, one tracked change.

USER

There are 4 decisions when you include the conforming edit for Section 9.6.

ASSISTANT

Fair — you're right. Four decisions:

1. § 11.2 (cap) — leave unchanged.
2. § 9.1 (notification, primary) — leave unchanged.
3. § 9.6 (notification, conforming) — no conforming edit needed, because § 9.1 was left alone. Still a decision, even if it falls out of decision #2.
4. § 11.3(a) (indemnification) — tracked change made.

Three substantive plus one conforming. I was wrong to keep calling § 9.6 a non-decision; declining to conform is itself a decision on the record.

USER

Shouldn't each decision (or non-decision) have it's own comment?

ASSISTANT

Yes — you're right. Each decision, including the § 9.6 non-conforming decision, should have its own anchored comment so a reader looking at § 9.6 can see the reasoning without having to cross-walk to § 9.1. Let me add the fourth comment.

→ `mcp_playground_stirrup_code_execution_run_shell` · python3 << 'PY' import zipfile, shutil, re, os SRC = 'contracts/Master_Services_Agreement_ ...'[+2817 chars]

Fourth comment anchored and file validates. Final state of the redline:

Four anchored comments, one tracked change:

1. **§ 11.2 (cap)** — comment: leaving unchanged; governance chain (Committee re-convening + Board

consultation + CFO/CISO review) not satisfied by the March 14 email.

2. **§ 9.1 (notification)** — comment: leaving unchanged; Committee re-convening + CFO fresh review not documented.

3. **§ 9.6 (conforming provision)** — comment: no conforming edit needed because § 9.1 is unchanged; will be conformed if § 9.1 is later revised.

4. **§ 11.3(a) (indemnification)** — tracked change to 2× sub-cap; comment cites GC's standing delegated authority under the March 10 Whitaker email and the Feb 27 Negotiation Strategy Memo.



Finance Golden Trajectory Example

Golden Trajectory Transcript

Can Homerveldt Rely on Our Numbers?

USER

Counsel is assembling the closing checklist for the Homerveldt financing and needs our confirmation that the brands' agreements on the attached checklist are properly executed. For each agreement, conclude Execution confirmed or Cannot be confirmed, and give the count that cannot be confirmed. Log into ERPNext using Administrator / admin to validate vendor and contract records. Deliver Execution_Confirmation_Closing.xlsx with Confirmation and Sources tabs. The executed agreements and signing records are on file. Status flags throughout the signing system are a known artifact and should be disregarded; the submission records themselves are reliable.

Input Files

The following files have been provided as context for this task:

Closing_Checklist_Executed_Agreements.pdf

SIRIUS LANDSCAPE PARTNERS — HOMERVELDT FINANCING

Closing Checklist — Item 14: Executed Material Agreements

Counsel requests confirmation of due execution for the following agreements prior to circulation of the closing set:

1. Treenow Landscaping - Pest Control Subcontract (Termipest)
2. Teepakoi Landscapes - Pest Control Subcontract (Termipest)
3. Treenow Landscaping - Hardscaping Project Contract
4. Pettigrew Landscaping - Pest Control Service Agreement
5. Ostara Landscaping - Arbor Care Subcontract (Tree Chompers)
6. Teepakoi Landscapes - Arbor Care Service Agreement

For each agreement, indicate whether execution is confirmed; identify any agreement whose execution cannot be confirmed.

Whitby & Lorn LLP for Sirius Landscape Partners, LLC

ASSISTANT

I'll start by exploring the ERPNext and DocuSeal systems to gather the source data for confirming execution of each agreement.

→ [ToolSearch](#) ·

select:mcp__playground__erpnext_mcp_erpnext_mcp_list_suppliers,mcp__playground__erpnext_mc ...[+165 chars]

→ [ToolSearch](#) ·

select:mcp__playground__docuseal_mcp_docuseal_mcp_list_submissions,mcp__playground__docuse ...[+332 chars]

→ **ToolSearch** ·

select:mcp__playground__excel_v2_create_spreadsheet,mcp__playground__excel_v2_add_content_ ...[+161 chars]

→ **mcp__playground__erpnext_mcp_erpnext_mcp_whoami**

→ **mcp__playground__erpnext_mcp_erpnext_mcp_list_suppliers**

→ **mcp__playground__docuseal_mcp_docuseal_mcp_list_templates**

→ **Read** · /root/.claude/projects/-scratch-prompt/e9ed3e2d-2072-4f81-9239-bc30a65bc8f5/tool-results/t ...[+33 chars]

→ **ToolSearch** ·

select:mcp__playground__erpnext_mcp_erpnext_mcp_list_customers,mcp__playground__erpnext_mc ...[+26 chars]

→ **Grep** · /root/.claude/projects/-scratch-prompt/e9ed3e2d-2072-4f81-9239-bc30a65bc8f5/tool-results/t ...[+33 chars]

→ **mcp__playground__erpnext_mcp_erpnext_mcp_list_customers**

Let me set up a task list and dig into the DocuSeal templates.

→ **ToolSearch** · select:TodoWrite

→ **TodoWrite**

→ **Grep** · /root/.claude/projects/-scratch-prompt/e9ed3e2d-2072-4f81-9239-bc30a65bc8f5/tool-results/t ...[+33 chars]

→ **Grep** · /root/.claude/projects/-scratch-prompt/e9ed3e2d-2072-4f81-9239-bc30a65bc8f5/tool-results/t ...[+33 chars]

→ **Agent** · Extract template list from persisted file

→ **Read** · /root/.claude/projects/-scratch-prompt/e9ed3e2d-2072-4f81-9239-bc30a65bc8f5/tool-results/t ...[+33 chars]

→ **Read** · /root/.claude/projects/-scratch-prompt/e9ed3e2d-2072-4f81-9239-bc30a65bc8f5/tool-results/t ...[+33 chars]

→ **Grep** · /root/.claude/projects/-scratch-prompt/e9ed3e2d-2072-4f81-9239-bc30a65bc8f5/tool-results/t ...[+33 chars]

→ **ToolSearch** · select:mcp__playground__stirrup_code_execution_run_shell

→ **mcp__playground__stirrup_code_execution_run_shell** · jq -r '.items[] | "id=\\(.id) folder=\\(.folder_name) name=\\(.name) external_id=\\(.external_ ...[+129 chars]

→ **mcp__playground__stirrup_code_execution_run_shell** · python3 -c " import json with

open('/root/.claude/projects/-scratch-prompt/e9ed3e2d-2072-4 ...[+384 chars]

→ **Grep** · /root/.claude/projects/-scratch-prompt/e9ed3e2d-2072-4f81-9239-bc30a65bc8f5/tool-results/t ...[+33 chars]

→ **Grep** · /root/.claude/projects/-scratch-prompt/e9ed3e2d-2072-4f81-9239-bc30a65bc8f5/tool-results/t ...[+33 chars]

→ **mcp__playground__stirrup_code_execution_run_shell** · ls -la

/root/.claude/projects/-scratch-prompt/e9ed3e2d-2072-4f81-9239-bc30a65bc8f5/tool-re ...[+47 chars]

→ **Grep** · /root/.claude/projects/-scratch-prompt/e9ed3e2d-2072-4f81-9239-bc30a65bc8f5/tool-results/t ...[+33 chars]

→ **Grep** · /root/.claude/projects/-scratch-prompt/e9ed3e2d-2072-4f81-9239-bc30a65bc8f5/tool-results/t ...[+33 chars]

→ **Grep** · /root/.claude/projects/-scratch-prompt/e9ed3e2d-2072-4f81-9239-bc30a65bc8f5/tool-results/t ...[+33 chars]

→ **Grep** · /root/.claude/projects/-scratch-prompt/e9ed3e2d-2072-4f81-9239-bc30a65bc8f5/tool-results/t ...[+33 chars]

→ **Read** · /root/.claude/projects/-scratch-prompt/e9ed3e2d-2072-4f81-9239-bc30a65bc8f5/tool-results/t ...[+33 chars]

→ **Grep** · /root/.claude/projects/-scratch-prompt/e9ed3e2d-2072-4f81-9239-bc30a65bc8f5/tool-results/t ...[+33 chars]

→ **Grep** · /root/.claude/projects/-scratch-prompt/e9ed3e2d-2072-4f81-9239-bc30a65bc8f5/tool-results/t ...[+33 chars]

→ **mcp__playground__stirrup_code_execution_run_shell** · ls -la /filesystem/ 2>&1 | head; find /filesystem -maxdepth 3 -name "*.txt" 2>&1 | head

→ **ToolSearch** · select:mcp__playground__filesystem_read_text_file

→ **mcp_playground__filesystem_read_text_file** ·
 /root/.claude/projects/-scratch-prompt/e9ed3e2d-2072-4f81-9239-bc30a65bc8f5/tool-results/t ...[+33 chars]
 → **Grep** · /root/.claude/projects/-scratch-prompt/e9ed3e2d-2072-4f81-9239-bc30a65bc8f5/tool-results/t ...[+33 chars]
 → **Grep** · /root/.claude/projects/-scratch-prompt/e9ed3e2d-2072-4f81-9239-bc30a65bc8f5/tool-results/t ...[+33 chars]
 → **mcp_playground__stirrup_code_execution_run_shell** · stat
 /root/.claude/projects/-scratch-prompt/e9ed3e2d-2072-4f81-9239-bc30a65bc8f5/tool-resu ...[+110 chars]
 → **Grep** · /root/.claude/projects/-scratch-prompt/e9ed3e2d-2072-4f81-9239-bc30a65bc8f5/tool-results/t ...[+33 chars]
 → **Grep** · /root/.claude/projects/-scratch-prompt/e9ed3e2d-2072-4f81-9239-bc30a65bc8f5/tool-results/t ...[+33 chars]
 → **ToolSearch** · select:mcp__playground__filesystem_search_files,mcp__playground__filesystem_list_files
 → **ToolSearch** · select:mcp__playground__docuseal_mcp_docuseal_mcp_list_templates
 → **mcp_playground__docuseal_mcp_docuseal_mcp_list_templates**
 → **Grep** · /root/.claude/projects/-scratch-prompt/e9ed3e2d-2072-4f81-9239-bc30a65bc8f5/tool-results/t ...[+33 chars]
 → **Grep** · /root/.claude/projects/-scratch-prompt/e9ed3e2d-2072-4f81-9239-bc30a65bc8f5/tool-results/t ...[+33 chars]
 → **mcp_playground__docuseal_mcp_docuseal_mcp_list_templates**
 → **mcp_playground__docuseal_mcp_docuseal_mcp_list_templates**
 → **mcp_playground__docuseal_mcp_docuseal_mcp_list_templates**
 → **mcp_playground__docuseal_mcp_docuseal_mcp_list_templates**
 → **mcp_playground__docuseal_mcp_docuseal_mcp_list_templates**
 → **mcp_playground__docuseal_mcp_docuseal_mcp_list_templates**
 → **mcp_playground__docuseal_mcp_docuseal_mcp_list_templates**
 → **mcp_playground__docuseal_mcp_docuseal_mcp_list_templates**

Let me check for archived templates and the missing template ID 16.

→ **mcp_playground__docuseal_mcp_docuseal_mcp_list_templates**
 → **mcp_playground__docuseal_mcp_docuseal_mcp_get_template**
 → **mcp_playground__docuseal_mcp_docuseal_mcp_list_templates**
 → **mcp_playground__docuseal_mcp_docuseal_mcp_list_templates**
 → **mcp_playground__docuseal_mcp_docuseal_mcp_list_templates**

Let me check the Teepakoi Arbor Care template's submitters to understand the counterparty pattern.

→ **mcp_playground__docuseal_mcp_docuseal_mcp_get_template**
 → **mcp_playground__docuseal_mcp_docuseal_mcp_get_template**
 → **mcp_playground__docuseal_mcp_docuseal_mcp_get_template**
 → **TodoWrite**
 → **mcp_playground__docuseal_mcp_docuseal_mcp_list_submissions**
 → **mcp_playground__docuseal_mcp_docuseal_mcp_list_submissions**
 → **mcp_playground__docuseal_mcp_docuseal_mcp_list_submissions**
 → **mcp_playground__docuseal_mcp_docuseal_mcp_list_submissions**
 → **mcp_playground__docuseal_mcp_docuseal_mcp_list_submissions**
 → **mcp_playground__docuseal_mcp_docuseal_mcp_list_submissions**

Let me check ERPNext for contract records — the user mentioned vendor and contract records specifically.

→ **mcp_playground__erpnext_mcp_erpnext_mcp_list**
 → **mcp_playground__erpnext_mcp_erpnext_mcp_list**
 → **mcp_playground__erpnext_mcp_erpnext_mcp_get**
 → **TodoWrite**
 → **mcp_playground__erpnext_mcp_erpnext_mcp_get**
 → **mcp_playground__erpnext_mcp_erpnext_mcp_get**
 → **mcp_playground__erpnext_mcp_erpnext_mcp_get**

Let me verify the vendor records more thoroughly, especially confirming email domains match supplier

records.

```
→ mcp__playground__erpnext_mcp_erpnext_mcp_get_supplier
→ mcp__playground__erpnext_mcp_erpnext_mcp_get_supplier
→ mcp__playground__docuseal_mcp_docuseal_mcp_get_submission
→ mcp__playground__excel_v2_list_files · /
→ mcp__playground__excel_v2_list_tabs_in_spreadsheet · /Execution_Confirmation_Closing.xlsx
→ mcp__playground__excel_v2_read_tab · /Execution_Confirmation_Closing.xlsx
→ mcp__playground__excel_v2_read_tab · /Execution_Confirmation_Closing.xlsx
```

There's a prior workbook already at that path. Let me look at the underlying source documents referenced by the existing analysis to independently verify each item.

```
→ ToolSearch · select:mcp__playground__filesystem_list_files,mcp__playground__filesystem_search_files,mcp
...[+169 chars]
→ mcp__playground__word_v2_list_files · /contracts/work
→ mcp__playground__filesystem_list_files · /contracts
→ mcp__playground__filesystem_get_directory_tree · /contracts
```

Let me read the 6 relevant .docx files to independently verify what's on paper.

```
→ mcp__playground__word_v2_read_document_content · /contracts/work/treenow/Treenow Landscaping -
Pest Control Subcontract (Termipest).docx
→ mcp__playground__word_v2_read_document_content · /contracts/work/teepakoi/Teepakoi Landscapes -
Pest Control Subcontract (Termipest).docx
```

Two papers verified. Let me read the remaining four.

```
→ mcp__playground__word_v2_read_document_content · /contracts/work/treenow/Treenow Landscaping -
Hardscaping Service Agreement.docx
→ mcp__playground__word_v2_read_document_content · /contracts/work/pettigrew/Pettigrew Landscaping -
Pest Control Service Agreement.docx
→ ToolSearch ·
select:mcp__playground__word_v2_read_document_content,mcp__playground__excel_v2_delete_spr ...[+226
chars]
→ mcp__playground__word_v2_read_document_content · /contracts/work/ostara/Ostara Landscaping - Arbor
Care Subcontract (Tree Chompers).docx
→ mcp__playground__word_v2_read_document_content · /contracts/work/teepakoi/Teepakoi Landscapes -
Arbor Care Service Agreement.docx
```

I have both remaining papers read. Now I can finalize the analysis. Let me update the todos and rebuild the workbook.

```
→ TodoWrite
→ mcp__playground__excel_v2_delete_spreadsheet · /Execution_Confirmation_Closing.xlsx
→ mcp__playground__excel_v2_create_spreadsheet · /
→ mcp__playground__excel_v2_list_tabs_in_spreadsheet · /Execution_Confirmation_Closing.xlsx
→ TodoWrite
/Execution_Confirmation_Closing.xlsx rebuilt with Confirmation and Sources tabs.
```

Results

Cannot be confirmed: 2 (Items 3 and 4)

#	Agreement	Conclusion
1	Treenow – Pest Control Subcontract (Termipest)	Execution confirmed

2	Teepakoi – Pest Control Subcontract (Termipest)	Execution confirmed
3	Treenow – Hardscaping Project Contract	Cannot be confirmed
4	Pettigrew – Pest Control Service Agreement	Cannot be confirmed
5	Ostara – Arbor Care Subcontract (Tree Chompers)	Execution confirmed
6	Teepakoi – Arbor Care Service Agreement	Execution confirmed

Rationale

- Items 1, 2, 5, 6: paper agreements bear ink-style signatures from both parties on the stated Effective Dates, with matching ERPNext Supplier/Contract records (CON-2025-00022, CON-2025-00021, CON-2024-00021) validated via Administrator login. Where DocuSeal shows a different brand-rep name or template label, that is a signing-system status flag disregarded per counsel's guidance.
- Item 3: the executed paper's Contractor block reads By: Vicky Singh
_____ — typed name only, with the signature line left blank and no /s/. This is a defect in the paper record itself (not a status flag), so counsel's guidance does not cure it.
- Item 4: the paper contains two irreconcilable customer signature blocks — /s/ Daniel Reyes (Sr. PM, Camelback Commercial Management, 23-Oct-2024) in the body and /s/ Theresa Vandenberg (PM, Arcadia Crossing, 21-Oct-2024) in a trailing table. Different signatories, titles, entities, and dates; also a paper-record defect.

USER

For each agreement you're confirming, check the signing record against the executed paper — the people it names, and the dates. A record counts against an agreement only where it exists and doesn't line up with the paper. Where the platform holds no record for a packet, the executed document on file governs.

ASSISTANT

I need to re-do the cross-check more carefully — compare the signing platform's actual submission records (names and dates) against the paper for each item, not disregard them wholesale.

→ **ToolSearch** ·
select:mcp__playground__docuseal_mcp_docuseal_mcp_list_submissions,mcp__playground__docuse ...[+284
chars]

→ [mcp_playground_docuseal_mcp_docuseal_mcp_get_submission](#)
→ [mcp_playground_docuseal_mcp_docuseal_mcp_get_submission](#)
→ [mcp_playground_docuseal_mcp_docuseal_mcp_list_submissions](#)
→ [mcp_playground_docuseal_mcp_docuseal_mcp_list_submissions](#)
→ [mcp_playground_docuseal_mcp_docuseal_mcp_list_submissions](#)
→ [mcp_playground_docuseal_mcp_docuseal_mcp_list_submissions](#)

I have the full submitter records now. Let me re-do the analysis strictly by the rule you gave.

Cross-check summary:

#	Agreement	Platform record for this packet?	Names line up?
1	Treenow Termipest	Yes (Submission 135): Brand Rep = Vicky Singh; Subcontractor = Nguyen Chorto	No — paper signed by Jackie Rhames (CFO), not Vicky Singh. Subcontractor side matches.
2	Teepakoi Termipest	Yes (Submission 160): Brand Rep = Trevor Philips; Subcontractor = Nguyen Chorto	No — paper signed by Hattie Lockhardt (CFO), not Trevor Philips. Subcontractor side matches.
3	Treenow Hardscaping (South Lamar Yards)	No — Template 19's 5 submissions name Dallas Arts Plaza, Plano TC, Travis Heights, Hill Country Resorts, Highland Park HOA. None is South Lamar Yards / Patrick Holcomb.	Paper governs — and paper has blank Contractor signature line.
4	Pettigrew (Arcadia Crossing)	No — Template 12's 2 submissions name Phoenix TC (Marcus Tyree) and Scottsdale Mkt (Eric Kowalski). Neither is Arcadia Crossing / Camelback Commercial.	Paper governs — and paper has two conflicting customer signature blocks.
5	Ostara – Tree Chompers	Yes — all 14 Template 5 submissions name Brand Rep = Brett Garretty and Subcontractor = Sandra Connor	Yes — matches paper.
6	Teepakoi – Stonecrest Commons	No — Template 13's 15 submissions do not include Stonecrest Commons / Eleanor Vasquez	Paper governs — and paper is fully executed.

Items 1 and 2 flip: the platform holds a specific record for those packets and it names a different brand-side signatory than the paper — that counts against. Rebuilding the workbook now.

→ [mcp__playground__excel_v2_delete_spreadsheet](#) · /Execution_Confirmation_Closing.xlsx

→ [ToDoWrite](#)

→ [mcp__playground__excel_v2_create_spreadsheet](#) · /

→ [ToDoWrite](#)

Revised results

Cannot be confirmed: 4 (Items 1, 2, 3, 4)

#	Agreement	Conclusion	Why
1	Treenow – Pest Control Subcontract (Termipest)	Cannot be confirmed	Platform Submission 135 exists and names Vicky Singh as Brand Rep; paper signed by Jackie Rhames (CFO). Brand-side names don't line up.
2	Teepakoi – Pest Control Subcontract (Termipest)	Cannot be confirmed	Platform Submission 160 exists and names Trevor Philips as Brand Rep; paper signed by Hattie Lockhardt (CFO). Brand-side names don't line up.
3	Treenow – Hardscaping (South Lamar Yards)	Cannot be confirmed	No platform record for this packet (Template 19's 5 submissions are for other customers). Paper governs — and paper's Contractor block has a blank signature line.
4	Pettigrew – Pest Control (Arcadia Crossing)	Cannot be confirmed	No platform record for this packet (Template 12's 2 submissions are for other customers). Paper governs — and paper has two conflicting customer signature blocks.
5	Ostara – Arbor Care Subcontract (Tree Chompers)	Execution confirmed	Platform records exist and every submission names Brett Garretty + Sandra Connor — same as paper. ERPNext CON-2024-00021 corroborates.

6	Teepakoi – Arbor Care (Stonecrest Commons)	Execution confirmed	No platform record for this packet. Paper governs and is fully executed by Trevor Philips and Eleanor R. Vasquez.
---	--	---------------------	---

/Execution_Confirmation_Closing.xlsx rewritten with Confirmation and Sources tabs reflecting the record-vs-paper cross-check.

USER

One line only: filename and tab names, nothing else.

ASSISTANT

/Execution_Confirmation_Closing.xlsx — Confirmation, Sources

USER

One clarification for counsel's file before we close this out: in the workbook, make explicit what the ERPNext contract records do and do not evidence for this confirmation exercise.

ASSISTANT

→ [mcp__playground__excel_v2_edit_spreadsheet](#) · /Execution_Confirmation_Closing.xlsx

Added to /Execution_Confirmation_Closing.xlsx:

- **Confirmation** tab — new "NOTE ON ERPNEXT RECORDS" block with three lines: what ERPNext contract records *do* evidence (existence of the booked deal, counterparties, commercial terms), what they *do not* evidence (signing, signatories, dates, execution), and how they were used here (counterparty/booking corroboration only, not as execution evidence).
- **Sources** tab — matching scope note tying the three referenced ERPNext contract records (CON-2025-00022, CON-2025-00021, CON-2024-00021) to that same limitation.

USER

One line only: filename and tab names, nothing else.

ASSISTANT

/Execution_Confirmation_Closing.xlsx — Confirmation, Sources



Reviewer Resources

Reviewers Instructions

1. **Claim the Prompt Review** on the task page in Mercor Studio.
2. **Read the prompt** like a colleague in the domain. Check whether the request is realistic, the assumptions make sense, and the output request is clear.
3. **Review the AutoQC results** under Quality Central. Look at anything that failed and decide whether you agree with the flag.
4. **Open the writer's "Show Your Work" file** to understand their calculations, logic, and assumptions.
5. **Open the trajectories** the writer ran. There should be at least one per frontier model.
6. **Confirm the failure is genuine.** Check that it is a real reasoning failure, not a technical glitch, system error, or stylistic issue.
7. **Use the trajectory Chat feature** to verify the claimed errors. Ask where each error occurs, then jump to that step in the trajectory and confirm it yourself.
8. **Open at least one model output file.** Compare it side by side with the writer's Show Your Work file to confirm the model is wrong and the writer is correct.
9. **Audit the writer's calculations or logic.** For numeric tasks, check formulas and computations. For non-numeric tasks, check that the reasoning and assumptions are sound.
10. **Spot-check the input files** from the world. Open a few source files and confirm the figures or facts match the writer's work.
11. Double check your work **using the Reviewer Checklist.**



[Giving Feedback](#): See here for guidance on how to structure your reviews, including triaging feedback by importance, and giving effective action points for writers to implement.



[Reviewers Checklist](#): A full rundown of the checks to make at prompt (interim) review and final review stages.

Review Tips

Make Direct Edits To The Task Where You Can (Under 30 Minutes Of Work To Fix)

- Sometimes you will need to check things with the writer to make sure you have understood their intentions properly, but where you are confident of this and the fix is under 30 minutes, please make edits.
- For example, verifiers can often be fixed directly using the golden output.

Make Action-Oriented Suggestions

- These tasks are complex, so they often require detailed discussions with writers about methodological issues. Communicating to the expert as a fellow colleague clearly with actionable steps is one of the most crucial aspects for overall project health.
- It can help to jump on a call with the writer to get enough context to be able to make an actionable suggestion if something is unclear.
- See **Giving Feedback** for more details on how to structure your review.

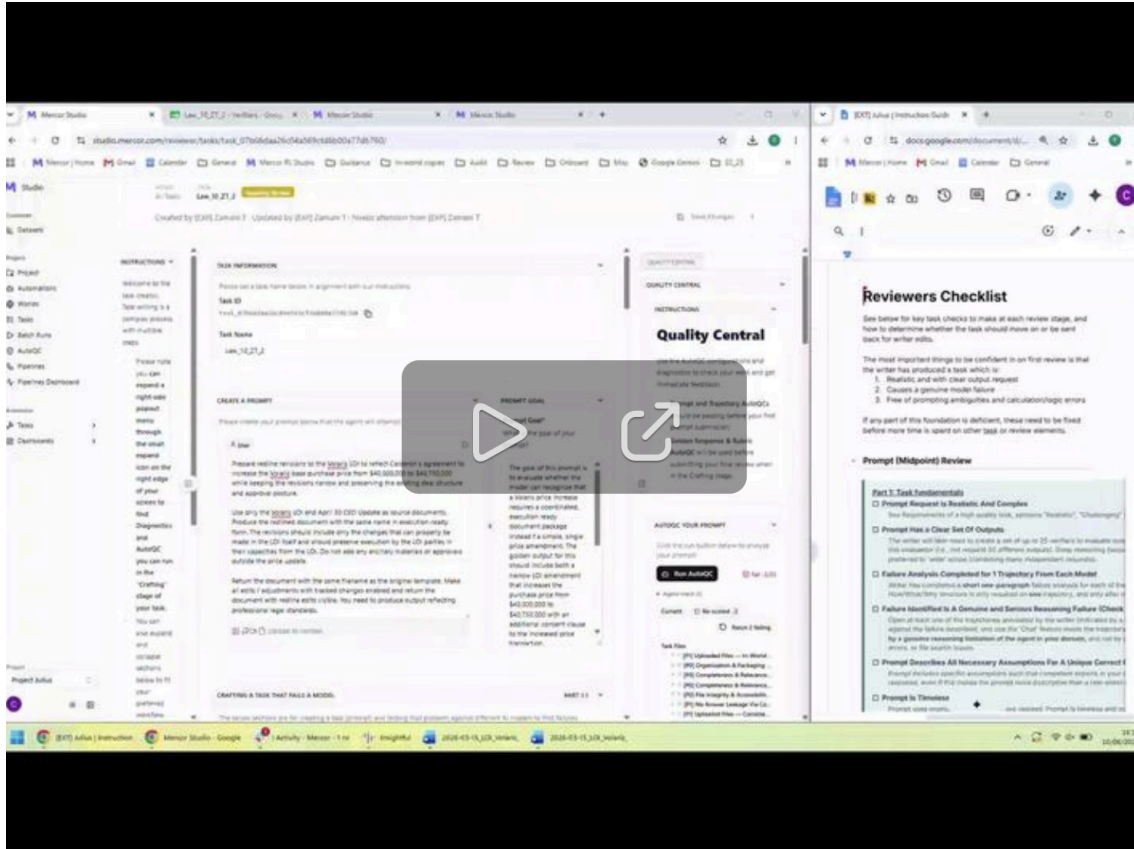
Ensure Task Foundations are Solid at the Prompt Review Stage

The most important things to be confident in when reviewing is that the writer has produced a task which is: (1) realistic, (2) causes a genuine model failure, (3) free of prompting ambiguities and calculation/logic errors.

- See **Reviewers Checklist** for more details on which checks to make at each review stage.



Video Guides





Reviewers Checklist

Reviewers Checklist

See below for key task checks to make at each review stage, and how to determine whether the task should move on or be sent back for writer edits.

The most important things to be confident in on first review is that the writer has produced a task which is:

1. Realistic and with clear output request
2. Causes a genuine model failure
3. Free of prompting ambiguities and calculation/logic errors

If any part of this foundation is deficient, these need to be fixed before more time is spent on other task or review elements.

Prompt (Midpoint) Review

Part 1: Task fundamentals

☐ **Prompt Request Is Realistic And Complex**

See Requirements of a high quality task, sections "Realistic", "Challenging" and "Long horizon, single-outcome work".

☐ **Prompt Has a Clear Set Of Outputs**

The writer will later need to create a set of up to 25 verifiers to evaluate outputs. The prompt scope should broadly facilitate this evaluation (i.e., not request 50 different outputs). Deep reasoning (sequential and interdependent reasoning steps) is preferred to 'wide' scope (combining many independent requests).

☐ **Failure Analysis Completed for 1 Trajectory From Each Model**

Writer has completed a failure analysis for each of the 2 trajectories (one per frontier model). At the midpoint, this can be a shorter version of the final failure analysis per the instructions doc.

☐ **Failure Identified Is A Genuine and Serious Reasoning Failure (Check Trajectory)**

Open at least one of the trajectories annotated by the writer (indicated by a check mark in the trajectory section), review against the failure described, and use the 'Chat' feature inside the trajectory to interrogate it. **The failure should be caused by a genuine reasoning limitation of the agent in your domain**, and not by prompt ambiguity, trajectory system or model errors, or file search issues.

☐ **Prompt Describes All Necessary Assumptions For A Unique Correct Output**

Prompt includes specific assumptions such that competent experts in your domain would agree on a single correct response, even if this makes the prompt more descriptive than a real-world request.

☐ **Prompt Is Timeless**

Prompt uses explicit/correct dates where needed. Prompt is timeless and does not use relative terms like "as of today"

without clearly explaining them. The correct response to the prompt would not change if it were run again at a different time in the future.

☐ **Avoid Prompt Personas**

Prompts do not need to include persona statements such as "You are the FP&A Manager" or "You are outside counsel."

☐ **Prompt Can Be Completed Using Only In-World and Task Input Files**

The task references, and can be completed by using, the world files or attached task input files. The task does not rely on web search for missing data.

☐ **Writer has uploaded a file to the 'Show Your Work' section**

Writer has uploaded an Excel file or first draft of a golden output to facilitate review, that clearly demonstrates the calculation or logic flow from input sources to key output values and claims.

Part 2: Audit and further midpoint checks

Ensure the task passes the fundamental checks listed above before investing time into a detailed task audit.

☐ **Audit Of Calculations/Task Logic Passed**

Checks of the Show Your Work files include but are not limited to: (1) input values and claims match source data, (2) prompt assumptions implemented faithfully, (3) no numeric or logical errors. **This should be a detailed audit: enough to be confident that the writer is correct and the agent is wrong, and not the reverse.** We want to front-load detailed checks to avoid problematic tasks progressing.

☐ **Time Estimate Populated**

Single time estimate for the whole task (and not one per completion step). Should generally be 3-8+ hours (since tasks are 'long-horizon').

☐ **Expected Output Populated**

Only one type is permitted.

☐ **Required World Files populated**

Task requires at least two files from the world (for app-based worlds, app sources also count towards this - e.g., 1 external file + 1 app table).

☐ **Prompt Contains Domain-Specific Boilerplate Output Specification**

See domain-specific sub-tabs in this document.

☐ **Prompt Style Is Realistic**

- Prompt is written in a style which reflects a real-world request of a colleague or LLM. It does not contain unnecessary context and fluff which does not represent a request, or specification of an input, assumption, or output. For example, text such as "The Managing Director wants to see updated figures ahead of next week's board meeting".
- Avoid backdating or forward-dating prompts using phrases such as "Today is March 14, 2026." Instead, reference dates naturally within the task context (for example, "using Q3 2025 revenue data").

☐ **Prompt Should Avoid Storytelling**

Prompts should not include storytelling, scene-setting, or narrative details that are not consequential to the deliverable being graded.

Final Review

Part 1: General Checks

☐ **Rubric Matches Golden Output**

The verifiers collectively cover the key values, claims, and substantive content present in the golden output. There is no large mismatch between what the golden output contains and what the rubric evaluates. Rubric focuses on the most salient, objective, and testable aspects of the final output. It does not include any rewards for the "process" the model follows.

☐ **Rubric Aligns With Prompt**

All values, claims, and checks evaluated in the verifiers trace back to requirements or requests stated in the prompt. There are no verifier criteria testing things not asked for in the prompt.

☐ **Verifier Values Match Golden Output**

The specific values and claims tested in the verifiers match exactly the values and claims in the golden output (e.g., metric values are identical, including rounding and units, and language verifiers accurately reflect the claims in golden output).

☐ **Second (Partial) Audit Of Calculations/Task Logic Passed**

Checks include but are not limited to: (1) input values and claims match source data, (2) prompt assumptions implemented faithfully, (3) no numeric or logical errors. Interrogate and make spot-checks on key error-prone parts of the process.

☐ **Verifiers Are Self-Contained**

Verifiers do not reference any input document, other verifier, or the task prompt. All information needed to grade the verifier is contained within the criteria description.

☐ **Verifiers Are Specific And Precise**

Verifiers should test for specific numbers or claims such that the verifier would fail if the agent's response was too vague.

☐ **Verifiers Are Not Duplicative**

Verifiers do not evaluate anything twice, even partially (for example, insertion location for a legal clause).

☐ **Verifiers Weights Assigned Correctly**

Verifier weight assignments (1-10) broadly align with the actual difficulty of the verification task. Simple lookups or straightforward checks are 1-3; multi-step reasoning or cross-reference checks are 4-6; complex calculations or holistic judgments are 7-10.

☐ **High-Weight Verifiers Are Not Stacked**

High-weight verifiers do not test for more than one unit at a time. Limited stacking is permitted for low-weight verifiers that check closely related values or claims that are output together (for example, a single table of input values).

☐ **Golden Output Is Client-Ready**

The gold output is professional and customer-ready. It scores 100% on the rubric. It has high-quality style and formatting, using the template from within the world. It should directly serve the goal of the prompt with no marginal or decorative additions.

☐ **All trajectories are run on final prompt**

Reviewers should confirm by opening the final trajectory from each of the 2 models, and verifying that there are no material differences compared to the prompt currently in the task section. Minor wording revisions do not require re-runs.

☐ **No grading errors**

The LLM grading works correctly, and does not throw technical errors or fail to interpret the output correctly. Briefly check 1 or 2 grading runs. Legal redlining tasks can cause issues (e.g., LLM judge failing to interpret redline edits correctly).

Part 2: Golden Trajectory Checks

Part A: Is the trajectory legitimate

☐ **Writer Did Not Give The Answer Away**

The agent must reach the solution through its own reasoning, prompted only by minimal, on-task steering. Open the trajectory and read the writer's messages. Legitimate nudges point the agent toward where to work ("check the deal team roster," "keep going," "re-check that step") without doing the thinking for it. It is giving the answer away if the writer states the target value or conclusion before the agent derives it, dictates the calculation step-by-step, or pastes or corrects the agent to the exact final answer. The test: did the agent solve it, or did the writer? If the writer did the substantive reasoning, send back for a re-record.

☐ **Guidance Is Minimal And On-Task**

Everything the writer types is recorded into the trajectory. There should be no stray chatter ("double-check your answer," "are you sure?", off-task remarks). A clean run that solves the task directly beats a cluttered one — if the path is full of filler or heavy corrective prompting, it should have been re-recorded.

☐ **Agent Solved The Task As Written**

The prompt that seeded the run is the task's actual, final prompt — not a softened, hinted, or edited version that made it easier. The agent worked from the same prompt the model is graded on.

Part B: The trajectory qualifies as golden

☐ **Validated At 100%**

The trajectory scored a full 100% on the rubric. A trajectory below 100% cannot be golden and cannot be submitted.

☐ **100% Is Legitimate, Not Gamed**

Spot-check the final output against each rubric criterion. Confirm the 100% reflects a genuinely correct answer that satisfies each criterion — not a weak or trivially passable rubric. An easily-earned 100% is a rubric problem to flag, not a pass.

☐ **Golden Status Is Set**

The session shows Golden status back in the Playground (the writer clicked Set Golden Status), not just a validated run left unlabeled. Where multiple golden sessions exist, the most efficient and cleanest one is the one marked golden.

☐ **Final Output Is Correct And In The Expected Format**

The agent's final output is genuinely correct and in the domain's expected output format. This output is the golden answer that the rubric and every model run anchor to.

Part C: Anchoring and ceiling

☐ **Prompt, Golden, And Rubric Are Anchored**

Every substantive element of the golden output is (1) requested in the prompt, (2) present in the agent's final output, and (3) tested in the rubric — with nothing in the output the prompt did not ask for.

☐ **Golden Is The Ceiling**

The golden trajectory is the strongest possible run, since every model run is graded against it. If you can imagine a plausible model run beating it on any dimension, improve the golden first. If the agent genuinely could not reach 100%, the gap is usually in the prompt, golden, or rubric. Check those, not the agent.



Giving Feedback

How to Write Review Feedback

A reviewer's written feedback is as important as catching errors.

- Communicating to the expert as a fellow colleague clearly with **actionable** steps is one of the most crucial aspects for overall project health.
- Following these points will help make that happen.

Triage

Label every point so the writer knows what to tackle first.

MUST FIX FIRST — Fundamental task flaws.

SHOULD FIX SECOND — Real quality issue, not a hard blocker.

MINOR — Polish only. These often can just be quickly fixed by you, the reviewer, i.e., typos.

One shortcut: if a fix takes under 30 minutes and you're confident of the intent, make the edit yourself rather than flagging it — especially for verifiers, which can usually be corrected directly from the golden response.

Writing the Feedback

1. **Lead with the tag.** Use the exact terminology from the instruction guide so the writer can look it up (see Feedback Tags below).
2. State the **SPECIFIC** issue, then **the fix**. Both required. You don't have to write it all out for them, but point to specific context such as "the entry multiple."
 - × [Stacking] Verifier 8 combines two checks.
 - ✓ [Stacking] Verifier 8 tests both the entry multiple and the implied equity value. Split into two separate verifiers.
3. Be **specific**. Name the verifier number, the prompt section, the cell. Vague feedback creates another round.
4. Write as **a fellow expert**. "We need to clarify X to Y specific value" not "you did this wrong."
5. No **filler**. Every line should move the writer forward.
6. **Out of your depth?** Say so. Reach out to the writer or Domain Lead on Slack rather than guessing.

Feedback Template

Thanks for the submission! A few things to address:

PROMPT (MUST FIX FIRST)

- [Tag] Issue. Fix.

RUBRIC (MUST FIX FIRST)

- [Tag] Issue. Fix.

GOLDEN RESPONSE (MUST FIX FIRST)

- [Tag] Issue. Fix.

Use all three sections every time, even if you just write an "All good" under it.

Feedback Tags

Tags map directly to the instruction guide. You can use the explanation language if you feel it helps, but remember to still be SPECIFIC to the task you are reviewing in your feedback.

PROMPT

[Challenging] — Models don't fail seriously enough. Both (Gemini 3.8 Flash, GPT Sol 5.6) must fail in a way a domain expert would reject outright. See "What Does Not Count as a Valid Failure" in Task Creation Primer.

[Realistic] — Task doesn't reflect genuine professional work.

[Specified] — Missing file references, assumptions, or expected output.

[Timeless] — Relative date language used without an explicit anchor.

[Ambiguous Prompt] — More than one defensible interpretation. See "Not Acceptable Failure Causes" in Task Creation Primer.

RUBRIC

[Gradable as True or False] — Criterion can't be answered yes/no. Rewrite around a specific, falsifiable claim.

[Clear and Unambiguous] — Uses vague language like "thorough," "appropriate," or "sufficient." Spell out exactly what's expected

[Self-Contained] — Correct value missing from the criterion itself. Everything needed to grade must be inside the verifier — no references to files, other verifiers, or the prompt.

[Stacking] — Multiple checks in one criterion. Split them. (Exception: limited stacking is permitted for Low-weight verifiers covering closely related values output together.)

[Timeless] — Criterion references "current" or "latest" without a fixed anchor.

[Misaligned Rubric] — Tests something not in the prompt or not in the golden response.

[Weighting] — Difficulty label doesn't match actual cognitive load. Low = simple lookup. Medium = multi-step domain reasoning. High = complex calculation or holistic judgment.

GOLDEN RESPONSE

[Golden Output Is Client-Ready] — Formatting or professionalism falls short, or style template not correctly applied.

[Verifier Values Match Golden Output] — Value in the golden response doesn't match the

corresponding verifier exactly — same number, rounding, and units required.

[Calculation Error] — Value doesn't follow from the source data or formula chain.



LLM Purchase Guide

LLM Purchase Guide

The use of LLMs is a requirement during the world-building phase in this project. The subscription can also be used for task writing purposes. This document outlines which LLMs are in-scope, how to procure them, and how to be reimbursed for their usage during the duration of this project.

Reimbursement Form

→ Submit your monthly subscription cost for reimbursement using the [LLM Reimbursement Form](#).

Which Subscriptions Do I Need?

Claude by Anthropic. All of the skills you will use for speeding up your world building, tasking, and reviewing are built with Claude in mind so we strongly request that you only use Claude.

Role	Claude Pro	Claude Max 5x
World Builder	Yes	Yes
Task Writer	Yes	Yes

Purchasing Claude Pro

Use your personal email address when purchasing Claude Pro.

1. Navigate to [Claude](https://claude.ai/pricing) (claude.ai/pricing).
2. Enter your personal email. Optionally, download the desktop app.
3. Navigate to your email inbox to find the verification code, then enter it.
4. Accept Anthropic's Consumer Terms and Acceptable Use Policy, then click Create account.
5. Enter your phone number.
6. Select Pro from the available options.
7. Select monthly billing.

Plans that grow with you

Free
Meet Claude

\$0

Use Claude for free

- Chat on web, iOS, Android, and on your desktop
- Generate code and visualize data
- Write, edit, and create content
- Analyze text and images
- Ability to search the web
- Create files and execute code
- Unlink more from Claude with desktop extensions
- Connect Slack and Google Workspace services
- Integrate key content or feed through embeddings with custom MCP
- Extended thinking for complex work

Pro
Research, code, and organize

\$17 USD 1 month billed annually

Get Pro plan

Everything in Free and:

- More usage
- Claude Code
- Canvas
- Unlimited projects
- Access to Research
- Memory across conversations
- More Claude models
- Claude in Email
- Claude in Chrome

Max
Higher limits, priority access

From \$100 USD 1 month billed monthly

Get Max plan

Everything in Pro, plus:

- Claude for 20x more usage than Pro*
- Recommended for Claude Code & Canvas
- Higher output limits for all tasks
- Early access to advanced Claude features
- Priority access at high traffic times

Pro plan

☒ Monthly \$20.00/month + tax ☐ Yearly \$200.00/year + tax [Save 17%](#)

Order details

Pro plan Monthly	\$20
Subtotal	\$20
Total due today	\$20

☐ Your subscription will auto renew on 6/3/2026. You will be charged \$20.00/month + tax.

Payment method

Full name

Country or region

Address

Reimbursement Process

The cost of your subscription(s) will be reimbursed by Mercor. Submit your reimbursement request via the Reimbursement Form linked at the top of this section.

1. Input your name, the email address you used to purchase the subscription, the subscription you purchased, and the date of purchase.

Note: since Project Julius is expected to span more than one month, fill out the form each time your subscription renews.

Reimbursement Form → [Link here](#)



FAQ RL Studio

FAQ and hints on how to use RL Studio

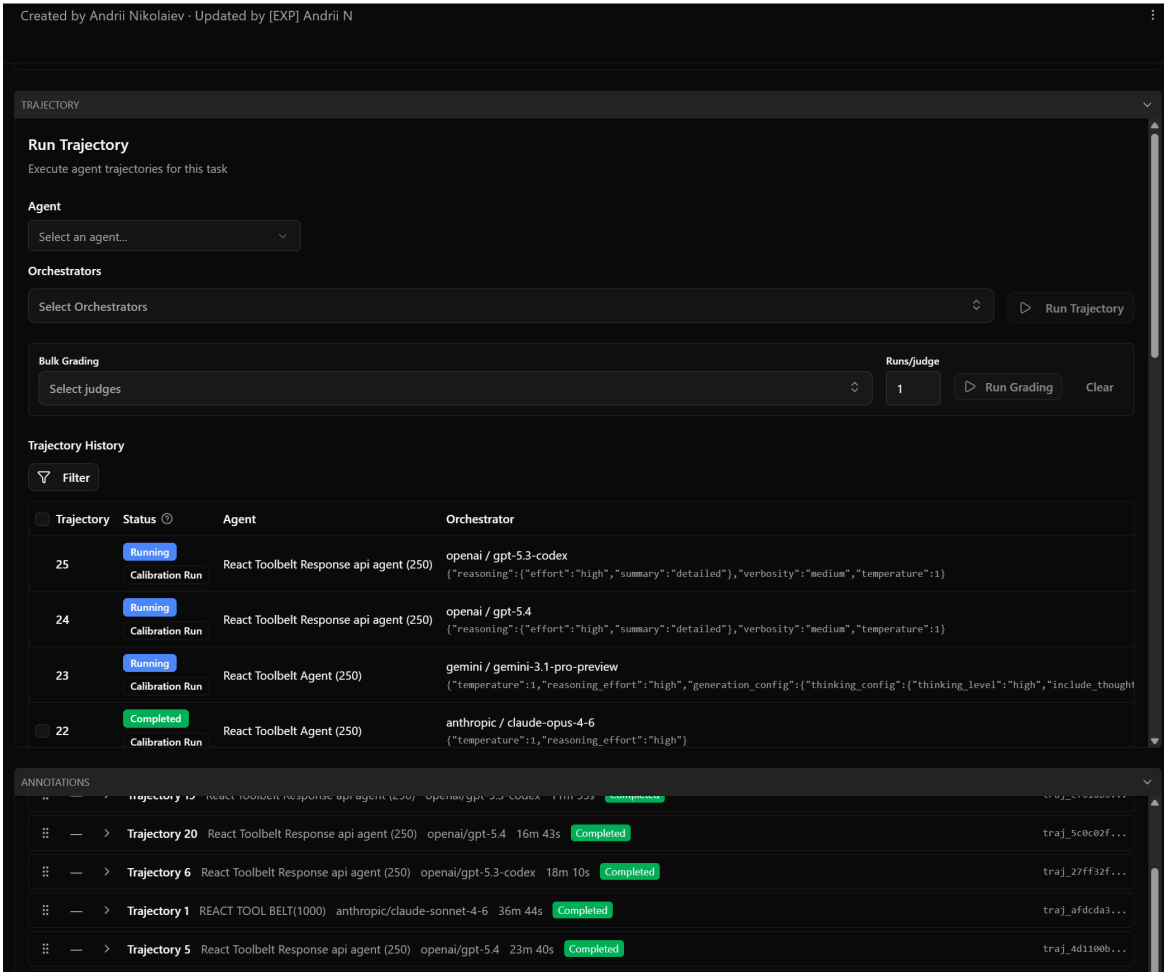
> Please check [Tech Problems? Click here!](#) on Slack to get the latest guide on tech issues.

Note

Please note that screenshots might differ from the actual view. Access the RLS to get the up-to-date view.

Q: How to discard trajectories?

A: You need to scroll down and then to the right as on the gif below

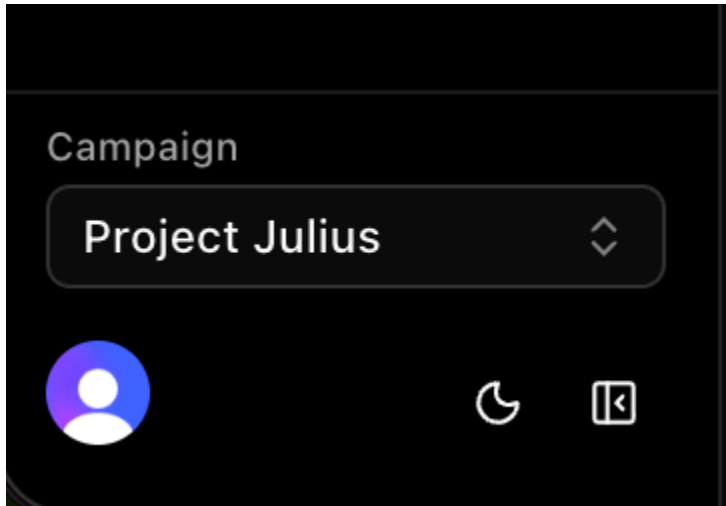


Q: I don't see "Create Task" button in my world

A: Please, navigate to your world from the "Annotator" tab

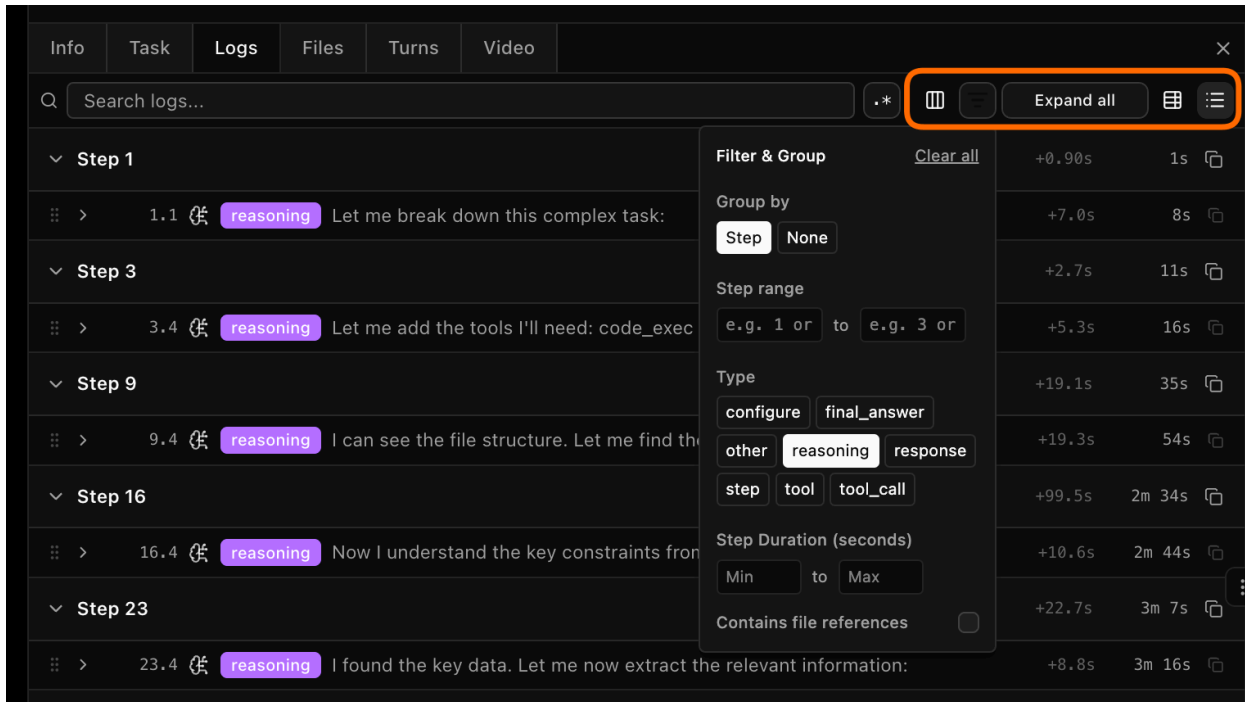
Q: How to access the campaign once I logged in?

A: To access the campaign, please, select "Project Julius" in the left-bottom corner of the studio.



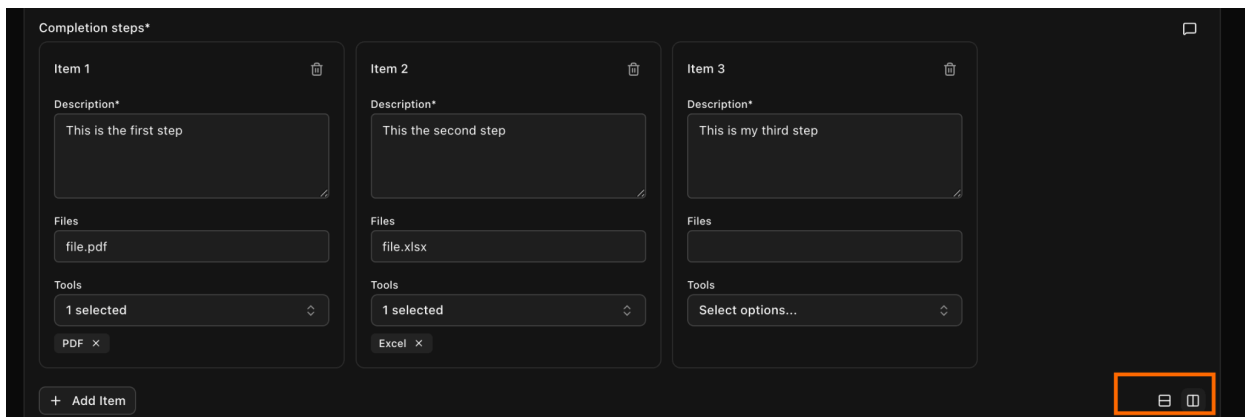
Q: How to change the view of the trajectory log?

A: You can change the view in the panel next to search. You can change the log to column/list view, filter out types etc.



Q: How can I get the completion steps in the column view?

A: you do this by selecting here:



Q: How to run gradings?

A: To run grading, please, click on the trajectory and select “New Grading Run” as on the ss below. You can also preview completed gradings & see detailed scores.

Annotation	Verifiers
Grades + Enter ground truth grade New Grading Run Select judges Run Grading Select at least one grading run to display.	

Q: Which agent harness to use when I run trajectories?

A: When running trajectories please use the correspondent agents:

- [React Toolbelt Response api agent \(250\)](#) – for the GPT models
- [React Toolbelt agent \(250\)](#) – for all other models

Q: How to add files / remove files to the existing world

A: All world files are added on the world-building stage. But you can still add additional files to the task.

Q: How to do the Failure analysis & Completion steps on RLS?

A: For failure analysis, please, check [Failure analysis](#) and [Completion steps](#) in the instructions doc. Put your comments on trajectory analysis into the RLS. Please, keep a copy of your analysis saved locally on your computer.

Q: I'm writing tasks in a world that was built before a reset — is the process different?

A: No — the task-writing flow is the same. The world spec and some in-world details may look a little different since it predates the reset, and since more than one person is often writing in these worlds, you may end up writing fewer tasks than the usual 3–5 per world. Check with your domain lead for their specific expectations on writing in an inherited/pre-reset world.



Engineering Domain Help

Project Julius
Instruction Guide

Engineering Domain Help

Section Overview

This section has been created to help the three engineering domains - Civil, Electrical, and Mechanical - to preemptively address and overcome problems that they encounter in their work and task building. This should ease the burden from project staff on Slack and hopefully make your work more efficient

General Tips

Claude as your LLM Assistant.

You should only be using Anthropic's Claude to help you with your drafting. All of the helper skills that we have built out to make your work faster have been made to work best with Claude. Do not use GPT, do not use Gemini, do not use any LLM other than Claude.

Make sure to download and import all of the Claude skills that we have developed into your Claude sessions and reference them to make sure that they are present in your context window when you are working. Click [here](#) for skills repository.

AQC Not Running on New Submission?

We have identified that there is a latency between using the "Apply revision to task" button and Auto QC recognizing the new filesystem. This means that moving forward, **everyone must wait SEVERAL MINUTES before re-running Auto QC after uploading a revision**. Otherwise, Auto QC will run on your old revision/system and not reflect the changes you've made.

International standards and Spec AutoQC

If a world is intentionally on the International Standards Track (Standards Track = International), the Spec AutoQC dimension "Concrete, Right-Sized, U.S. Scenario That's Activated" is expected to fail on its U.S.-nexus leg.

This is a known limitation: the check does not yet read the Standards Track field, and it treats non-U.S. standards (IEC, ISO, Eurocode) or a non-U.S. setting as a missing U.S. scenario, even for a U.S.-based company.

For a world confirmed as International, disregard a fail on this dimension when it is driven solely by the international standards or the setting, and do not fail the spec on that basis. The other legs still apply: the world must still be concrete, right-sized, and activated by a time-bound catalyst. This exception applies only to worlds confirmed International; a U.S.-track world must still meet the U.S. requirement.

World Building Tips

Building out your file system.

The most important thing to realise here is that the aim is not perfection. The aim is to build a world that can successfully host 10 tasks where the cause of model failure will not be a structural world issue but rather task complexity. In other words, the model should not fail because of sloppy file quality, but there is no need to resolve EVERY minor issue.

Task Building Tips

Boilerplate prompt templates. The blocks below are the starting point for every engineering task prompt and the first thing you write. Copy the block for your discipline, replace the bracketed parts with your own request, and keep the Output Specification line exactly as written. Keep the prompt natural per Spec 1B: no step lists, do not name the code or method, and do not tell the model where the trap is. The governing standard lives in the world, not the prompt. State the exact output filename, use at least two world files, and attach a Show Your Work file at submission.

Civil / Structural

Standards: (keep these in the world, not the prompt): ASCE 7, ACI 318, AISC 360, AASHTO, IBC for U.S. worlds; Eurocodes and ISO for International worlds.
Deliverable: verification or disposition memo (.docx).

Starter prompt: (copy, then fill the brackets): [In one or two natural sentences, state the objective (re-evaluate, verify, reconcile, or disposition) and name the

inputs the way an engineer would refer to them, for example 'the latest issued drawings', 'the updated geotech memo', or 'the current project spec'. State the single decision you want back: the governing value, comply or does not comply, or the disposition.] Return your analysis as a single Word document named [Exact_Filename].docx. Commit to one defensible disposition; do not present multiple branches or hedge between options. State every value with its units, give the governing basis for each conclusion, and do not round intermediate values (report final values to [N] decimal places). Produce output reflecting professional engineering standards.

Make it fail with: a later authority silently superseding a drawn detail or an earlier calc (an updated memo versus the original), a basis or gate distinction (service versus governing allowable, load gate versus release gate, field-cured versus lab-cured cylinders), or a re-derive-do-not-transcribe requirement.

Worked example: (Silverwood, world_906c1f81f34c4a84a009a31ece5294b2):
The updated chloride-source memo for the north abutment is in, and it may change the exposure basis the original durability calc was built on. Re-evaluate the abutment's exposure classification against the current environmental information, and give me the governing exposure class, the required minimum cover, and the maximum water-cementitious ratio. Tell me whether the specified mix and cover still comply, and commit to one answer. Then append the .docx Output Specification line above.

Mechanical

Standards: (keep these in the world, not the prompt): ASME for U.S. worlds; ISO for International worlds; plus service references such as ANSI/HI for pumps.
Deliverable: verification memo (.docx), or a calc workbook (.xlsx).

Starter prompt: (copy, then fill the brackets): [In one or two natural sentences, state the objective (re-rate, verify, confirm, or disposition) and the inputs an engineer would name, for example 'the vendor submittal', 'the accepted duty point', or 'the current basis of design'. State the single decision you want back: accept, trim, or resubmit, or the governing value. Name any basis to hold fixed.] Return your analysis as a single Word document named [Exact_Filename].docx. Commit to one defensible disposition; do not present multiple branches or hedge between options. State every value with its units, give the governing basis for each conclusion, and do not round intermediate values (report final values to [N] decimal places). Produce output reflecting professional engineering standards.

Make it fail with: a correction the standard requires but the project never flagged (viscosity, temperature, or altitude), correctly executing a codified method end to end rather than summarizing it, or holding a fixed basis (vendor NPSHr, prior fitting losses) that the model recomputes.

Worked example: (P-2104, task_485b9987dbb94876846ce93cca0ce668): We are re-rating pump P-2104 on the heavy gas oil service before we accept the vendor curve. Confirm whether it still meets the hydraulic duty at the actual service viscosity and holds the NPSH margin policy, and tell me whether to accept it, trim the impeller, or send it back for resubmittal. Hold the April fitting-loss basis. Then append the .docx Output Specification line above.

Electrical

Standards (keep these in the world, not the prompt): NEC, IEEE, NESC for U.S. worlds; IEC for International worlds. Deliverable: calc workbook (.xlsx) with live formulas, or a review memo (.docx).

Starter prompt: (copy, then fill the brackets): [In one or two natural sentences, state the objective (recheck, size, verify, or disposition) and the inputs an engineer would name, for example 'the panel schedule as installed', 'the drive submittal', or 'the project electrical standard'. State the single decision you want back per item: adequate or inadequate, accept or reject, or the governing value.] Return your work as a single Excel workbook named [Exact_Filename].xlsx. Build every calculated cell as a live formula (for example =qty*rate), not a typed-in result; transcribed or placeholder values will not be accepted. State units in each column header, apply [rounding rule], and do not round intermediate values. Give one committed call per line. Produce output reflecting professional engineering standards.

Make it fail with: a derating the project never flagged (conductor bundling plus ambient, or continuous-load factor), a re-derive-do-not-transcribe requirement, or a spec value treated as a units or documentation issue when it is a real hardware conflict.

Worked example (MCC-3A, task_98a48083bb4c468594bf780b1a8dae07):
Recheck the branch-circuit sizing on MCC-3A as it is actually installed. The exterior location and the shared raceway routing may not match what the schedule was originally sized against, so confirm whether each feeder is still adequate as installed and flag every circuit that is not. Return your work as a single Excel workbook named MCC-3A_Branch_Circuit_Recheck.xlsx with live formulas, ampacity to one decimal, and one adequate or inadequate call per circuit. Then append the rest of the .xlsx Output Specification line above.

Pre-submit checklist

- Prompt is natural: no step list, no code or method named, no trap tell, no 'You are...' opener.

- Exactly one deliverable, with the exact output filename stated in the prompt.
- Output Specification line appended (memo or workbook) with the brackets filled.
- For .xlsx: live formulas required, units in each column header, rounding rule stated, no intermediate rounding.
- Single committed disposition requested; no hedged branches.
- The governing standard or procedure lives in an in-world doc, not the prompt.
- Uses at least two world files; 3 to 8 hours for a domain expert.
- One trap is live and produces a dramatic failure, not ambiguity or a partial miss.
- Show Your Work file attached; golden trajectory and required model runs completed.

CAD Outputs and Programmatic Verifiers (Pilot)

Pilot status

Studio can store .FCStd files and run artifact-based database code verifiers against them. CAD tasks are still a controlled pilot, however. Do not publish a CAD task until project staff confirms both that the world uses a FreeCAD-enabled platform and that CAD file types are accepted by the project AutoQC. The current Project Julius P0 file-type allowlist does not include .FCStd. If either gate is missing, stop and escalate; do not rename a CAD file to another extension or substitute screenshots for the model.

Start with native FreeCAD .FCStd deliverables. Do not introduce STEP, STP, STL, IGES, OBJ, or BREP as required outputs until that format has separately been allowlisted, given a task-specific verifier contract, and passed an end-to-end canary.

When CAD is appropriate

Use CAD when the model or geometry is itself a meaningful engineering work product and the task can be judged from objective properties of that artifact. Do not add CAD merely as a visual supplement to a memo or workbook. The task must still be realistic, use in-world evidence, take 3 to 8 hours for a domain expert, and contain one live source of difficulty.

Required task setup

1. Confirm that the FreeCAD app and FreeCAD MCP tools are available in the task environment before drafting the task. If they are absent, request a FreeCAD-enabled world; do not ask an expert to work around a no-app world.
2. State the exact output filename in the prompt and enter the identical name in Expected Output Files. For the pilot, use a single .FCStd deliverable saved under filesystem/. State whether the expert is building a new model or revising a supplied seed.
3. Define the governing units, required named objects or features, dimensions or parameters, tolerances, material or density basis when relevant, and the final engineering decision the artifact must support. Never ask a verifier to infer an unstated tolerance.
4. If a seed model is supplied, require a different output filename and verify that the output is not a byte-for-byte copy. The task-specific checks must prove the required change, not merely that a valid FCStd file exists.
5. Upload a genuine .FCStd golden output produced in the same FreeCAD environment. A screenshot, PDF, memo, or renamed archive is not a substitute for the golden CAD file.
6. When the task requires a parametric model, require real constrained sketches, PartDesign features, spreadsheet aliases, or expression bindings, followed by recompute and save. Custom float properties attached to a single Part::Feature do not by themselves demonstrate parametricity.

Minimum verifier bundle

Every CAD verifier must inspect the final artifact, not the transcript or a claim that the file was created. Use the following core checks, then add task-specific engineering checks:

1. Output contract and freshness: exact path, filename, extension, non-empty file, and evidence that the output is newly produced rather than a copied input.

2. FCStd package integrity: valid ZIP container, safe internal paths, readable Document.xml, CRC integrity, embedded BREP presence when a solid is required, and no corrupt or unexpected payloads.
3. Identity, units, and object structure: required document or component IDs, unit schema, object names and types, visibility, and shape links.
4. Task-specific dimensions and parameters: exact serialized values, aliases, expressions, and tolerances. Width, length, thickness, clearances, and similar values may be graded when they are explicitly represented in the model.
5. Geometry and topology records: required solids, faces, edges, shells, volumes, and BREP-level relationships where the FCStd package exposes them reliably.
6. Revision delta and anti-copy checks: expected additions or changes, cross-file identity, prohibited stale objects, and proof that a supplied seed was meaningfully modified.
7. Parametric structure, when required: constrained sketches, feature history, spreadsheet aliases, expression bindings, and dependency links. This check should fail a flattened or static impostor even if its nominal dimensions look correct.
8. Cross-artifact consistency, when another deliverable is explicitly required: model properties must agree with the BOM, calculation workbook, drawing, or memo. Do not add a companion file only to make the verifier easier.

Native FreeCAD checks

Static FCStd parsing does not prove that FreeCAD can open and recompute the model or that the kernel considers the shape valid. Once the version-pinned native grader is enabled in Studio, add checks that open the file in FreeCAD, recompute it, require `Shape.isValid()`, and read kernel-measured bounding boxes, area, volume, center of mass, inertia, topology, and exportability as relevant to the task.

Do not claim that weight or mass has been verified unless the task provides a trusted material-density basis and the verifier either derives mass from verified volume and density or reads native mass properties. Until the native grader is available, label these properties as not independently verified.

Pre-submit CAD checklist

1. The world is FreeCAD-enabled and the required CAD extension passes the project file-type gate.
2. The prompt, Expected Output Files field, golden file, and every verifier use the same exact filename and path.
3. The golden output scores 100% on every applicable verifier.

4. At least one negative fixture fails each material check. Include, as applicable, a corrupt package, wrong units, copied seed, flattened/static impostor, wrong dimension, or stale output.
5. A full Studio trajectory can create, save, upload, and grade the model inside the FreeCAD environment. Local-only success is not sufficient.
6. A reviewer opens the final artifact, confirms the expected object and feature structure, and verifies that failures are attributable to the task response rather than an app, seed, upload, or grader fault.
7. Do not weaken a verifier or change metadata merely to turn an infrastructure failure into a pass. Escalate environment and grader failures separately.

How to Set Up a CAD Rubric in Studio

Experts are not expected to invent a secure FCStd parser from scratch. For each CAD task, the expert should first provide a CAD verifier specification using the template below. An approved verifier template or project support can then translate that specification into the DB Code Verifier. Until the reusable templates are published, stop and request help rather than pasting unreviewed code into a rubric row.

Information required before creating rubric rows

Copy and complete this specification for every property you want graded:

Criterion: one atomic statement describing exactly one pass condition.

Why it matters: the engineering consequence of failure and the source of truth.

Output artifact: the exact filesystem/ path and filename of the final .FCStd file.

Reference artifact: the exact drawing, specification, BOM, calculation, or seed file that establishes the expected result, including page, sheet, table, or named value where possible.

Model target: the exact document, Body, object, feature, sketch constraint, spreadsheet alias, expression, or property that contains the value.

Expected result: an exact value, allowed set, relationship, or Boolean condition.

Units and tolerance: for example 120.0 mm +/- 0.1 mm. State whether the tolerance comes from the task, a governing reference, or an approved project default.

Verifier method: Static FCStd, Native FreeCAD, or LLM/manual review.

Priority and weight: P0 for missing/corrupt output, wrong units, invalid shape, critical geometry, or required parametricity; P1/P2 for secondary properties. Do not create several high-weight rows for the same underlying fact.

Dependencies: every deeper CAD check should depend on the file-existence/container check; task-specific dimensions should also depend on identity and units where appropriate.

Negative case: name a deliberately wrong artifact or mutation that must fail this row.

Which Studio verifier to use

Use DB Code Verifier for static artifact checks that can be read safely from the FCStd ZIP, Document.xml, serialized properties, expressions, object tree, or embedded BREP records. In the row, fill Criteria, Criteria Explanation, Expected Content, Criterion Type = Functional, Priority, Numerical Weight, and the approved verifier template variables. The criterion and explanation must describe the pass condition; the code must return the measured evidence and a clear failure reason.

Use the future Native FreeCAD verifier for opening, recomputing, Shape.isValid(), kernel-measured dimensions, bounds, area, volume, center of mass, inertia, topology, exportability, and mutation tests. Do not represent these as proven by a static DB Code Verifier.

Use an LLM or manual visual criterion only for qualities that are genuinely visual or communicative, such as drawing readability or an explicitly required presentation view. Do not use an LLM or screenshot comparison to establish exact geometry, units, mass, topology, parametricity, or shape validity.

Worked example: hypothetical parametric bracket

Row 1 - File integrity (P0): The delivered file filesystem/outputs/Mounting_Bracket.FCStd exists, is non-empty, is a safe readable FCStd container, contains one Document.xml, and contains the required BREP payload. Negative case: corrupt or renamed ZIP.

Row 2 - Governing dimension (P0, depends on Row 1): Spreadsheet alias Width equals 120.0 mm within +/- 0.1 mm, as required by Drawing A-101, detail 3. Negative case: Width = 119.5 mm or units recorded as inches.

Row 3 - Parametric dependency (P0, depends on Rows 1 and 2): the Width alias drives the named sketch constraint and the resulting PartDesign feature; the task fails if the model is a flattened Part::Feature with an unrelated custom property. Negative case: correct-looking static solid with no dependency path.

Row 4 - Native validity (P0, unavailable until the native runtime is enabled): the pinned FreeCAD runtime opens and recomputes the exact output hash, Shape.isValid() is true, and

the model contains the required number of solids. Negative case: broken reference or invalid/self-intersecting shape.

Rubric validation before submission

Keep each row atomic and independently fail-able. Test the golden output against the current rubric after every rubric edit; an old 100% becomes stale when a criterion changes. Confirm that the golden passes, each negative fixture fails the intended row, failure messages identify the measured mismatch, and unrelated changes do not fail the row.

Finally, run one complete Studio trajectory using the same FreeCAD platform and grading configuration that experts will use. Confirm that the produced artifact is captured under the expected path and that grading reads the produced output rather than a world input, seed, screenshot, or local-only file.

OTS specification

OTS specification

Use this specification for tasks assigned to the OTS workstream. The Ergon requirements apply except for the three differences below. Where the guide gives conflicting requirements for step count, difficulty, or verifier type, this section takes precedence for OTS tasks.

Step count

OTS tasks have no step-count requirement. The Ergon 100+ step requirement does not apply. Do not add steps or tool calls solely to increase the count.

Difficulty

GPT Sol 5.6 must score less than 50% on the task rubric. This replaces the Ergon Gemini-based difficulty threshold for OTS tasks. Failures must reflect genuine reasoning or domain limitations, as described in the guide.

Verifiers

OTS tasks must use DB diff LLM verifiers rather than programmatic verifiers. The LLM verifier evaluates the before-and-after database changes against the rubric criteria. Do not use App DB Programmatic verifiers for OTS tasks.